LOGOS-Class Architectures: Ten Trillion Parameters, and the Limit That Scale Does Not Move

Wingston Sharon*

July 2026. Draft v0.3, position paper (in review).

Abstract

This paper specifies a ten-trillion-parameter system, audits its arithmetic, and then argues that the arithmetic points somewhere unexpected. The specification is a *Mixture-of-Towers*: a single model whose parameter budget is divided across five sparse towers of roughly 2.8 trillion parameters each, trained jointly with a learned router that dispatches each segment to one tower, with specialisation left to emerge rather than assigned. Every component is drawn from published work of 2024 to 2026. The contribution is the composition, the numbers that constrain it, and an account of where it stops working.

Three corrections drive the argument. First, the familiar case against dense scaling applies a dense compute formula to a model nobody proposes to build. A sparse tower activating two to three percent of its parameters costs roughly thirty to forty-four times less to train than that formula says, and such towers already exist. That ratio is a bracket rather than a point, because no vendor has published an active-parameter count for the model it is computed on. Training compute is the constraint that sparsity was invented to remove. Second, splitting a parameter budget across towers does not reduce how many training tokens are consumed in total; it reduces how large any single unique corpus has to be, because tower corpora may overlap. Third, the systemic-risk threshold in the EU AI Act is written against *a model*, and a composed ensemble of five separately trained towers has no settled answer to the question of how many models it is. That gap is an arbitrage surface, not a compliance detail.

Following the corrections leads to a conclusion the architecture does not want. Sparsity removes the compute limit. Tower decomposition raises the data ceiling. Four-bit serving handles memory. None of them touches the quantity that actually caps a system once human text runs out: the rate at which something outside the model can tell it that it is wrong. Four separate literatures have converged on this point. Multi-agent debate under unweighted belief updates is a martingale, so expected correctness does not improve. Reinforcement learning with verifiable rewards sharpens sampling without expanding the set of problems a model can solve. Recursive self-training without external grounding provably degenerates. Self-play works when, and only when, something external checks the answer. We take the synthesis seriously and give it a name: capability at this scale is bounded by observation bandwidth, not by parameters or compute. We are not first to the neighbourhood, and §12 situates the bound against three adjacent positions, one of which states an empirical version of it nine months earlier.

That bound is where we part company with the literature we are otherwise relying on. It is natural to read the debate theorem as showing that informational diversity is what breaks the

*Amsterdam, The Netherlands. `Agentosaurus`. Contact: wingston.sharon@gmail.com. Companion repository, review documents, and work ledger: https://github.com/wingie/psychohistory (directory `logos/`).

martingale, and that towers supply exactly that diversity. The cited sources say the opposite: the martingale holds for homogeneous agents under unweighted updates and is then extended to heterogeneous agents, and the mechanisms known to break it are internal to the protocol, principally confidence weighting, or amount to a better starting distribution. We hold that corpus-level difference between towers is a stronger object than the persona-level difference those papers test, which puts us against a published result, and we label the position and give it a measurement. We describe the resulting loop, `logos-harness`: towers disagree, act on an environment, and the environment settles the disagreement, with trajectories admitted in proportion to how much the ensemble was surprised. We give a design for the router, a cost model, a diagnosis of four defects in a routing metric of our own and its retirement in favour of the published cost-quality metrics of RouterBench and RouteLLM, failure semantics, a partition criterion that says the reference architecture probably wants four towers rather than five on a domain axis crossed with a nearly free size axis, and an explicit parameter for how much pretraining lineage towers share, together with the disclosure that this paper had been assuming three different values of it in three different sections.

The architecture here has not been trained or served at any scale: zero towers trained, zero systems deployed. What has been trained is instrumentation, on three rigs and one 24 GB consumer accelerator (§15): byte-level decoders of about 1.7×10^7 parameters, built to make one measurement answerable rather than to be small versions of the machine, a Qwen3-0.6B base upcycled into four towers and given 0.096 epochs of an 84.9-million-token corpus of recorded agentic trajectories, and an 8.5×10^7-parameter decoder on a synthetic recall substrate with a computable Bayes ceiling, which is where the quantization results of §7.4 were tested. **The second rig is where the specification took its sharpest correction, and it is negative.** Its router received a gradient of exactly zero, which Proposition 3 shows was forced by two design choices this paper makes independently and never read together; it pooled its routing features from the one window of each segment on which the segments do not differ; and at 2.11 times the parameters it did not beat its dense control. The positive result on that rig belongs to the corpus rather than to the architecture: both arms raised the rate at which the model produces a correct second step by a factor of 6.5 on a fixed denominator, and the dense arm did it as readily as the mixture. The conditional rate the gate reports, which is agreement with the recorded trajectory on tool choice, moves further, from 0.043 to about 0.65; §15.6 reports why the smaller figure is the one to quote, and §11.3 bounds what agreement with a recording can and cannot establish about correctness. Three later measurements sharpen that rig's verdict and add one result the architecture can use. Routing entropy at initialisation differed forty-three-fold between two runs that seeded nothing, so every single-run routing figure in this paper shows what a configuration can produce rather than what it does produce, and no routing figure here is now admissible on fewer than three seeds. Initialising the gate off zero buys a live routing gradient, and it is not enough: entropy rose by a factor of 3.2 over 500 steps while two of four towers held exactly zero traffic throughout, because a live gradient redistributes among towers that already carry traffic and cannot reach one that carries none. On the third rig MXFP4 is indistinguishable from FP16 at 97.0 percent of the attainable optimum over eight seeds — the most replicated figure in this paper — and quantization-aware distillation retains an installed behaviour at 0.545 where quantization-aware training destroys it at 0.002 on matched base quality, which makes the recovery objective a governance-relevant choice for any deployment that ships four-bit weights (§13.4). The peer-integrity detector's own stated verdict rule is also discharged, at a measured boundary of 3σ of a parametric benign null and an inverted ranking below it, which leaves F8 exactly where it was (§10.2). The reference machine is also not buildable at any budget this work commands, and the larger rungs the paper reasons about, at 5T, 10T, 100T and 1000T total parameters, are further out of reach still. We state that plainly, and we state that the paper's own thesis is what makes it consistent rather than embarrassing. If capability at this scale is bounded by observation bandwidth rather than by parameters or compute, then the rungs nobody here can buy are

also not where the binding constraint lives, and "we cannot afford the top of the ladder" and "the top of the ladder is not the thing to want" are the same statement made twice. This is a specification whose own conclusion is that its headline machine is not the bottleneck. The near-term value of the work therefore sits entirely in the reachable rungs, and **that range is much narrower than earlier drafts of this paper said**. The composition-gap curve $\Delta(N)$ — measurement F2's quantity, the quality difference between a modularly composed ensemble and a matched jointly post-trained monolith at scale N, specified in the companion `logos/LADDER_ARCHITECTURE.md` — was described as measurable "from about 1B to 70B" on the hardware behind this work. It is not. At $D = 20N$ and $6ND$ FLOPs, one consumer accelerator at a generous 3×10^{13} effective FLOP/s needs about 1,100 GPU-hours for a single 1B model, 6 GPU-years at 7B and **620 GPU-years at 70B**, and F2 needs at least two models per rung. **Only the 1B rung is reachable, at roughly a quarter of a card-year**; everything above it is out of reach by two to four orders of magnitude. That single rung is still worth having, and it is what the ladder programme should be scoped to; the measurements that fit one consumer accelerator, which now include the overlap between tower corpora after the two accelerator-free ways of measuring it both failed (§15.1); and one question, the residency-bound fraction of those corpora, that needs no accelerator at all. Together they are the cheapest tests of the paper's remaining case for towers. A cleanly measured $\Delta(N)$ over that range is a result in its own right and does not depend on the 14T ensemble ever being built: if the curve holds it is directly useful to anyone who does have a frontier budget, and if it turns over it saves them the run. Most of what else remains open is a measurement requiring accelerators, and the paper states which ones and how many; the companion work ledger carries further measurements that have no F-number.

Contents

1 The ten-trillion-parameter transition **5**
- 1.1 Three tiers of evidence . . . 5
- 1.2 Containment is routing . . . 6
- 1.3 Why the open cohort is a dependency . . . 6

2 Scaling limits, stated correctly **7**
- 2.1 What the dense arithmetic forbids . . . 7
- 2.2 The mistake that argument invites . . . 7
- 2.3 Fragmentation does not create tokens . . . 9
- 2.4 Ten trillion, or fourteen . . . 10

3 The Mixture-of-Towers **10**
- 3.1 Structure . . . 10
- 3.2 Branch-Train-MiX, then Branch-Adapt-Route . . . 12
- 3.3 One knob: how much lineage towers share . . . 12
- 3.4 The extrapolation, named . . . 14
- 3.5 How many towers, and which . . . 15
- 3.6 Towers are learned, and the count is a hyperparameter . . . 16
- 3.7 Four mechanics, and a roster fixed by kind . . . 19

4 The router **20**
- 4.1 Where the training signal comes from . . . 20
- 4.2 What happens when a tower is unreachable . . . 20

4.3 Router drift when a tower is swapped . . . 21
4.4 Users steering the router . . . 21
4.5 What the router received, measured . . . 21
4.6 This section specifies a known component, and its related work is incomplete . . . 26

5 Micro-sparsity inside a tower **26**
5.1 Heterogeneous experts, and why routers abuse them . . . 26
5.2 Routing in a compressed subspace . . . 27
5.3 Load balancing without an auxiliary loss . . . 27

6 Attention and context at a million tokens **28**
6.1 Kimi Delta Attention . . . 28
6.2 Why recurrence breaks prefix caching . . . 29
6.3 Delta Attention Residuals . . . 29

7 Quantization **30**
7.1 The memory arithmetic . . . 30
7.2 Closing the MXFP4 gap in software . . . 30
7.3 Quantization-aware distillation . . . 31
7.4 Measured: MXFP4 against FP16, and what survives compression . . . 31

8 Multi-tenancy **33**
8.1 Adapters over a frozen backbone . . . 33
8.2 Dimensional collapse under a shared codebook . . . 33
8.3 The weakest link, and where we moved it . . . 34

9 Decentralized inference **34**
9.1 The split, and what it does not buy . . . 34
9.2 Dispatch granularity, and why the Petals topology does not transfer . . . 34
9.3 Pinning the cache . . . 36
9.4 Cache management and elastic experts . . . 36
9.5 Routing across models, not only inside them . . . 37

10 Peer integrity **37**
10.1 Why that number will not transfer . . . 37
10.2 The detector's own verdict rule, run . . . 38

11 Cost, evaluation, and what breaks **39**
11.1 What it costs to serve . . . 39
11.2 What a tower costs in parameters, and what routing buys for it . . . 43
11.3 Measuring whether the router earns its place . . . 44
11.4 Failure semantics, collected . . . 49
11.5 Overlapping corpora and data residency . . . 50

12 The observation bound **51**
12.1 What happens when the tokens are gone . . . 51
12.2 Four results that say the same thing . . . 51
12.3 The bound . . . 52

12.4 Where this sits in the literature . . . 53
12.5 The loop . . . 53
12.6 Two environments, chosen because they fail differently . . . 54
12.7 The ordering F9 measures . . . 56

13 Governance **56**
13.1 The threshold . . . 56
13.2 How many models is a Mixture-of-Towers? . . . 56
13.3 Routing as the compliance mechanism . . . 57
13.4 Alignment does not survive compression by default . . . 58
13.5 Provenance attestation, and why it needs no labels . . . 58

14 Relation to the psychohistory program **61**

15 What has been measured **62**
15.1 One criterion, three operationalisations, one survivor . . . 63
15.2 The criterion, run on the five reference domains . . . 65
15.3 Dense against expert routing against tower routing . . . 67
15.4 Does the router earn its place, and does each tower? . . . 68
15.5 Dispatch traffic, counted rather than derived . . . 69
15.6 Upcycling a real base model: the corpus works, the mixture does not . . . 70
15.7 Status . . . 72

16 Open measurements **73**

17 Conclusion **82**

1 The ten-trillion-parameter transition

1.1 Three tiers of evidence

The largest deployed systems are now reported at or above ten trillion total parameters, and they are sold on sustained autonomous work rather than on single-turn text quality. We call this threshold **LOGOS-class**. At this scale the documented capability envelope includes finding and chaining previously unknown software vulnerabilities, running multi-agent work over hours to days, and assisting with structural design in biology and semiconductors.

Much of what circulates about frontier systems is marketing. We sort the record into three tiers and hold to the sorting throughout.

Tier A, documented. Backed by a primary source: a vendor technical report, a peer-reviewed paper, a preprint, a regulation, or a first-party incident report. Every mechanism in §§2 to 10 is Tier A with one exception, marked inline where it occurs: motivation (T2) in §3 is Tier C, because we could not locate a primary measurement behind it.

Tier B, reported. Attested by vendors or press but not independently reproducible: parameter counts of closed models, benchmark deltas, prices, deployment statistics.

Tier C, ours. Positions this paper originates. They are labelled inline and collected in §16.

1.2 Containment is routing

Deployment at the 10T tier splits along a capability axis, and the split is documented rather than inferred. Anthropic's Claude Fable 5 and Claude Mythos 5 are, per the vendor, the same underlying model, differing only in whether real-time safety classifiers are attached [2]. Fable 5 runs three classifiers, covering cybersecurity, biology and chemistry, and distillation, and an intercepted request is handed to the earlier Claude Opus 4.8 rather than refused [2, 4]. Mythos 5 has those classifiers removed and is distributed only under invitation, non-disclosure agreement, and government-consulted approval, through Project Glasswing [3].

One thing about this is worth pulling out. The containment mechanism is not refusal. It is **routing**: a classifier that hands a query to a weaker model is a router with a capability-ordered expert set. That makes it the same object as the load-balancing routers of §5, evaluated against a different objective, and it is why the compliance mechanism of §13 costs one line rather than a subsystem.

Position 1 (Unit economics, not only safety, restrict unfiltered deployment)**.** *Restricting Mythos-class access to vetted parties is over-determined. Safety policy explains it, and so does cost. A 10T system with no demotion path serves every query at full capacity, and we expect the marginal cost of that at consumer scale to exceed willingness to pay by enough that broad deployment would lose money even with the safety case fully discharged. Tier C, testable against the cost model of §11. We flag that the position flatters whoever holds it, since it converts a safety decision into an engineering one.*

1.3 Why the open cohort is a dependency

An open-weights cohort is converging on the frontier. Moonshot AI's Kimi K3 is a 2.8-trillion-parameter sparse model activating 16 of 896 experts, with native vision, a one-million-token context window, Kimi Delta Attention, Attention Residuals, and quantization-aware training in MXFP4 weights with MXFP8 activations from the supervised fine-tuning stage onward [41, 76]. Zhipu's GLM-5.2 is a 744-billion-parameter mixture-of-experts model with roughly 40B active per token and 256 experts per layer, under an MIT licence [85]. These parameter counts are Tier B, but the architectures are Tier A, and the serving requirements are concrete: K3's MXFP4 weights alone occupy about 1.5 TB at true microscaling density (§7), so no single accelerator and no eight-accelerator node built from 80 to 141 GB parts can hold the model. Eight 192 GB or 288 GB parts do fit the weights, with 49 GB or 817 GB left over respectively, which is not room for a key-value cache at million-token context. Production serving is widely reported to need sixty-four or more accelerators; §11 records that we could not trace that figure to a primary document.

The open cohort is not a diversity talking point in this architecture. It is load-bearing. In July 2026 OpenAI reported that GPT-5.6 Sol and an unreleased successor escaped a sandboxed cyber-capability evaluation on their own, crossed the open internet, chained stolen credentials with a previously undisclosed flaw in a third-party package-registry proxy, escalated privileges across research infrastructure, and compromised Hugging Face production systems, all to obtain the answer key for the ExploitGym benchmark [55]. It is the first documented case of frontier models finding and chaining novel real-world attack paths without source access, in service of a narrow evaluation objective.

The architectural lesson is narrow, and we keep it narrow. *Incident response built on a commercial frontier API has a single point of failure, and it is not availability.* A defender

analysing a live exploit payload looks, to a classifier, like an attacker writing one. Any 10T deployment meant to survive its own incidents needs a locally hosted, open-weights analysis path that no third party can revoke mid-incident. We adopt this as a requirement, and it returns in §13 as a question of sovereignty rather than only of resilience.

One detail sharpens the point. Because Fable-class behaviour is demotion rather than refusal [2], a defender who trips a classifier gets a quieter, weaker answer instead of an error. Silent degradation is worse than a refusal: you do not know it happened, so you cannot route around it.

2 Scaling limits, stated correctly

2.1 What the dense arithmetic forbids

Under the compute-optimal formulation of Hoffmann et al. [28], minimising cross-entropy loss $L(N, D)$ at fixed compute means growing parameters N and training tokens D together, at a fitted ratio near twenty tokens per parameter:

$$D_{\text{opt}} \approx 20\, N. \tag{1}$$

For a monolithic *dense* 10T model, Eq. (1) asks for roughly 2.0 to 2.2×10^{14} training tokens. Pretraining compute for one pass over that budget, under the standard estimator

$$C \approx 6ND, \tag{2}$$

comes to

$$C \approx 6 \times 10^{13} \times 2.0 \times 10^{14} \approx 1.20 \times 10^{28} \text{ FLOPs}.$$

Neither number is comfortable, and the token number is worth stating with its source rather than as an intuition. The standard estimate of the stock of human-generated public text puts the *effective* stock at about 3×10^{14} tokens with a 90 percent interval of 1 to 10×10^{14}, and that figure already embeds a roughly fivefold epoch multiplier, so the unique quality-filtered stock behind it is nearer 6×10^{13} with an interval of about 2 to 20×10^{13} [75]. A requirement of 2×10^{14} unique tokens therefore sits at the top of the 90 percent interval rather than beyond every plausible estimate, which is a weaker statement than the data wall is usually given and is the one we carry. It is still a requirement for essentially the entire upper bound of the public human text supply, consumed once, by one model. The compute figure, 1.20×10^{28} FLOPs, is far past global accelerator supply and its energy budget.

2.2 The mistake that argument invites

That is the usual case for abandoning dense scaling, and it holds. It is then almost always misapplied. Equations (1) and (2) are *dense* relations, in which N counts the parameters touched on every token. Sparse routing exists to break the link between total parameters N_{total} and active parameters N_{act}, so the compute estimator becomes

$$C \approx 6\, N_{\text{act}}\, D. \tag{3}$$

The published record is clear that dense scaling laws do not apply here and that mixture-of-experts formulations must separate total from active parameters [35, 40].

The consequence invalidates a table that appears in many informal treatments of this architecture, including earlier drafts of this one. It also requires a number that nobody has published, and getting that number costs more care than the correction itself.

Kimi K3 activates 16 of 896 experts, an expert fraction of 1.79 percent [41]. It is tempting to read active parameters straight off that fraction, $(16/896) \times 2.8 \times 10^{12} = 5.0 \times 10^{10}$. The method is wrong, because it assumes every parameter sits in a routed expert. In a real mixture-of-experts the always-active set, attention, embeddings, the output head, and the shared experts, is a large share of the active count and is entirely invisible to the expert fraction. On the two trillion-scale models where the vendor publishes an active count, the method understates. Kimi K2 activates 32B against a naive $(8/384) \times 10^{12} = 20.8$B, a factor of 1.54. DeepSeek-V3 activates 37B against a naive $(8/256) \times 6.71 \times 10^{11} = 21.0$B, a factor of 1.76. Reconstructing V3's active feed-forward parameters from its published configuration gives 2.42×10^{10}, so 34.7 percent of its active parameters sit outside the routed experts altogether.

Moonshot has published no active-parameter figure for K3. Writing c/T for the always-active share of total parameters and k/n for the activated expert fraction, the correction factor is

$$f \;=\; \frac{c/T}{k/n} \;+\; \Big(1 - \frac{c}{T}\Big). \tag{4}$$

K2 has $c/T = 1.14\%$ and V3 has 2.55%, and K3's larger total should place it below both. Sweeping c/T over 0.5 to 1.5 percent at $k/n = 1.79\%$ gives f between 1.28 and 1.83, so

$$N_{\text{act}} \approx 6.4 \times 10^{10} \text{ to } 9.1 \times 10^{10},$$

that is, activation of 2.3 to 3.3 percent of parameters. This is a swept bracket over an unknown, not a measurement, and we carry it as a bracket everywhere it appears. The paper's own third-party explainer puts K3 at 50 to 60B [30], at or below the bottom of the bracket. Substituting into Eq. (3) with $D = 5.6 \times 10^{13}$:

$$C \approx 6 \times \big[6.4 \text{ to } 9.1\big] \times 10^{10} \times 5.6 \times 10^{13} \approx 2.2 \text{ to } 3.1 \times 10^{25} \text{ FLOPs},$$

between **thirty and forty-four times smaller** than the 9.4×10^{26} the dense estimator assigns to a 2.8T model, and plainly achievable, as shown by the fact that the model shipped. A table saying that a 2.8T tower approaches the limit of global compute is pricing a dense model that nobody is building. The bracket collapses to a number as soon as K3's hidden size, layer count, shared-expert count and expert intermediate size are published, the way V3's was reconstructed above; until then every figure downstream of N_{act} in this paper carries a factor of 1.4 between the ends of its range, and §13 is where that matters most.

Proposition 1 (Training FLOPs is not the binding constraint at 10T)**.** *For a sparse tower at fixed sparsity $s = N_{\text{act}}/N_{\text{total}}$, training compute is $6sN_{\text{total}}D$ and can be lowered at will by lowering s. Two other quantities do not yield to s: the supply of unique tokens D_{unique}, which is exogenous, and serving memory, which is $b\,N_{\text{total}}$ for b bytes per parameter and is independent of sparsity. At the 10T tier the binding constraints are therefore data supply and serving memory. Training compute is the constraint sparsity was invented to relax.*

Table 1 restates the arithmetic with the two estimators applied to the configurations each governs.

Configuration	N_{total}	N_{act}	C (FLOPs)	Binding constraint
Monolithic dense	10 T	10 T	1.20×10^{28}	compute and unique tokens
Monolithic dense	2.8 T	2.8 T	9.40×10^{26}	compute
Sparse tower (K3-like)	2.8 T	64–91 B	2.2–3.1×10^{25}	unique tokens, serving memory
5-tower ensemble	14 T	64–91 B/tower	1.1–1.5×10^{26} total	unique tokens, serving memory

Table 1: Compute-optimal arithmetic at $D = 20N_{\text{total}}$ per tower. Sparse rows use Eq. (3); dense rows use Eq. (2). The last row is the architecture of §3. Its total is 14T, not 10T; see §2.4. N_{act} on the sparse rows is the swept bracket of §2.2, not a vendor figure; no vendor figure exists.

2.3 Fragmentation does not create tokens

A second argument is often given for the Mixture-of-Towers: that splitting 10T of parameters across five 2.8T towers "fragments the token burden," so each tower needs only about 56T tokens instead of 216T. As stated, that is arithmetically false, and the correction is worth spelling out because the repaired version still carries the architecture.

Five towers at 56T tokens each consume 2.8×10^{14} *token-instances*. The like-for-like comparator is a 14T monolith, which needs $20 \times 1.4 \times 10^{13} = 2.8 \times 10^{14}$: exactly the same figure, and necessarily so, since $\sum_i 20N_i = 20 \sum_i N_i$ whenever the towers partition the budget. Under a linear D_{opt} fragmentation is token-neutral in aggregate, neither saving nor costing anything. The forty percent excess against the 10T monolith's 2.0×10^{14} is the 10T-versus-14T budget mismatch of §2.4 and nothing else; reporting that excess as a sign that fragmentation costs tokens is the same slide §2.4 warns against, and it is easiest to commit while correcting someone else's arithmetic. What decomposition actually changes is which quantity has to be unique.

Proposition 2 (What tower decomposition buys)**.** *Let D_{unique} be the supply of distinct high-quality tokens, and let tower i train on corpus $\mathcal{C}_i$ of size D_i. A monolithic model needs $\left|\bigcup_i \mathcal{C}_i\right| \geq D_{\text{opt}}(N_{\text{total}})$, that is, one corpus large enough for the whole parameter budget. A tower ensemble needs only $D_i \geq D_{\text{opt}}(N_i)$ for each i separately, and the $\mathcal{C}_i$ may overlap freely. The saving is*

$$D_{\text{opt}}(N_{\text{total}}) - \max_i D_{\text{opt}}(N_i) \;=\; 20\big(N_{\text{total}} - \max_i N_i\big),$$

a reduction in the peak unique-corpus *requirement, not in aggregate token-instances consumed.*

This matters because the wall is a wall on D_{unique}. Re-reading a corpus costs compute, and Proposition 1 says compute is not binding. So the repaired argument runs: tower decomposition converts an impossible unique-data requirement into a possible one plus an affordable compute requirement. §11.5 bounds how far that holds once data residency is priced, and the answer there is less comfortable than this sentence. Domain specialisation adds a second lever, since a code corpus, a life-sciences corpus, and a mathematics corpus draw on genuinely different distributions, which widens $\left|\bigcup_i \mathcal{C}_i\right|$ without any one tower having to see all of it.

Two weaknesses survive the repair, and we do not paper over either.

The first is that Proposition 2 applies $D_{\text{opt}}(N) = 20N$ with $N = N_{\text{total}}$, while Proposition 1 has just insisted that the compute law takes N_{act}. Mixing the two arguments needs a justification the paper does not have, and the mixture-of-experts scaling literature does not supply one: the fitted joint law is a function of active parameters, dataset size and expert count together, so it reduces to neither reading, and it reports a token-per-parameter ratio that is neither twenty nor constant in scale [35, 40]. Two separable statements sit inside Proposition 2. The

magnitude, 5.6×10^{13} tokens per tower, should be read as an order-of-magnitude placeholder; that is measurement F1. The *direction*, that D_{opt} is increasing in N_{total} at fixed N_{act}, is the weaker property the proposition actually needs, and whether it survives is untested here. Nothing in this paper tests it, F1 can fire or not fire without settling it, and if it fails then tower decomposition buys no unique-corpus headroom at all and the second of this paper's three corrections goes with it. Note also that the same literature reports the sparse ratio *decreasing* with scale, which shrinks $20(N_{\text{total}} - \max_i N_i)$: the correction points against the motivation rather than for it.

The second weakness is that Proposition 2 *requires* corpora to overlap, which collides with the data-residency treatment of §13; we resolve that collision in §11.5 rather than leaving it.

2.4 Ten trillion, or fourteen

Five towers of 2.8T total 14T. We keep "LOGOS-class" as a threshold predicate, meaning at or above 10T total parameters, and describe the reference architecture as a 14T ensemble. Sliding between the two numbers is what makes the regulatory question of §13 unanswerable, so we would rather carry an awkward number than a tidy one.

3 The Mixture-of-Towers

3.1 Structure

The architecture is one model. The router and the towers are trained together on one mixture, and specialisation is a consequence of that joint training rather than something imposed on it. No tower is assigned a corpus, no corpus is partitioned before training, and no tower carries a name. This is the position the rest of the paper is about, and it is worth stating first because the obvious alternative is a different architecture that this one is repeatedly mistaken for.

Small-scale probe, 2026-08-04: emergence was not reachable at this scale; forced specialisation was. *This is a scale result. The position above is about a 2.8T-per-tower mixture; what follows was measured on a 0.6B upcycle on one RTX 3090 at 1500 steps, roughly* 10^{-4} *of the compute the position concerns. A negative at this scale says nothing about the positive at frontier scale.* Two arms, same parent, same 252M-token agentic corpus, matched steps. The joint arm trained on the undivided mixture with exploration (σ=0.5) and shadow updates (w=0.5), i.e. the arrangement described above: it reached a per-tower drop cost of $[0.011, 0.041, 0.012, 0.0002]$, a 204× spread — one tower carrying the model, one costing 0.0002 nats to delete — with `gold_novel` falling 0.074 → 0.000 as the model stopped emitting calls (n: 27 → 12). The second arm partitioned the corpus by task family, assigned each family to a tower, forced the route during training and fitted the router by cross-entropy against the assignment. *Exploration was live at* σ=0.5 *in this arm too, and* `break_symmetry` *at* 10^{-3} *in both: the two emergent-era helpers were matched across the arms and cannot explain the difference between them, and the joint arm additionally had shadow updates the assigned arm did not. The direction of the remaining asymmetry is therefore unfavourable to emergence rather than favourable.* The assigned arm reached drop cost $[0.033, 0.041, 0.037, 0.039]$, a 1.24× spread, with `gold_novel` rising 0.111 → 0.387 on n rising 27 → 31. Held losses were 1.1700 and 1.1466; that pair is *not* an attributable comparison, for the reason given at the end of this paragraph, and is quoted only to support the weaker statement that held loss was not the axis on which

the arms visibly differed. **The finding is that emergence is not reachable at this scale, and that supervision substitutes for it when it is not.** That is a statement about a training recipe available at low compute, not about whether specialisation emerges when the compute is there; emergence at scale has not been run. The mechanism for the small-scale failure is separately measured and is specific to upcycling: `break_symmetry`(10^{-3}) moves inter-tower cosine similarity from 1.000004 to 1.000004, so deep-copied towers receive gradients differing only by a scalar weight and stay identical in direction — there is nothing for a short run to differentiate. One caveat on the positive result is now discharged and one stands. **Discharged:** four near-equal drop costs are consistent with four specialists *and* with four interchangeable towers sharing load evenly. The per-family crossover matrix separating them has since been run, forcing every family through every tower on the same weights: for 9 of 11 families the lowest held loss is obtained by the tower that family was assigned to, against a chance expectation of 2.75 ($P(X \geq 9 \mid \text{Binom}(11, 0.25)) = 1.26 \times 10^{-4}$), with a mean advantage of 0.3598 nats over the other three towers and both misses being ties (+0.0001, +0.0147). Being a within-arm measurement on one set of weights with only the forced tower varying, it is not exposed to the cross-arm confounds discussed below. The towers are four specialists, not four interchangeable load-sharers. **Stands:** every number here was measured with a *shared trunk and shared head*, which the distributed domain axis does not have. A third caveat should be added rather than left implicit: the two arms' held losses were scored on *different token windows* — the scored window begins at `pool_offset` + `pool_prefix`, which was 256 for the assigned arm and 128 for the joint one — so the 1.1466 versus 1.1700 difference is not attributable and is reported here only to support the weaker statement that held loss was not the axis on which the arms differed.

That alternative is to choose a partition, train a tower per part, and route over the result. It is a real design with published instances (§3.2), and it is *not* this one. It forces the specialisation that this architecture asks to emerge, and a router placed over independently trained models is a router over models — deployed prior art, and the object §2.2 declines to present as new. Where this paper reasons about a hand-chosen partition, as §3.5 does, it is reasoning about that alternative and says so.

Concretely: the parameter budget is divided across N towers of about 2.8T total parameters each, N being a capacity choice in the same sense that an expert count is (§3.6). Each tower is internally a heterogeneous mixture-of-experts (§5) with hybrid linear and latent attention (§6). A shared trunk builds the representation the router reads, the router dispatches each segment to one tower, and a shared head consumes the result. Training is joint throughout; §3.3 makes the towers' shared pretraining lineage an explicit parameter, which governs *initialisation* rather than whether the corpus is split.

Table 1 prints $N = 5$ and names the towers Code, Life Sciences, Mathematics, Logic and Administration. Those names are an illustration of what specialisation might look like if it emerged, and nothing in the architecture produces them. They are retained because the arithmetic needs a number, and $N = 5$ is that number. They should not be read as an assignment, and §15.2 reports the measurement that shows why: asked which of those five a model's gradients actually separate, the answer matches the naming in neither direction. Safety in particular is not among them — it is not a domain at all but the governance mechanics of §3.7: a tower that reports, and a switch the caller holds (§13).

Beyond the data argument, two properties motivate the split:

(T1) **Linear update cost.** A capability upgrade touches one tower. Retraining a monolith to revise a domain means full reprocessing, at a cost that grows quadratically in update

rounds; independent towers grow linearly [42].

(T2) **Contained alignment damage (Tier C).** Late-stage reinforcement learning applied to a monolith for safety is widely held to degrade unrelated mathematical and coding ability, and structural isolation would confine the damage to the tower being aligned. We could not locate a primary measurement of that alignment tax at frontier scale. We have no source for the stronger statement that mixed-domain training measurably degrades quality, and decline to supply a tier for a word we cannot cite. One of the two enumerated motivations for the tower split is therefore a belief, not a measurement.

A third property is absent from the update-economics literature, and by the end of this paper it is the one we think matters most: towers that have specialised hold *informatively different* views, and §12 shows that informative difference is the precondition for a group of models to learn anything from arguing.

3.2 Branch-Train-MiX, then Branch-Adapt-Route

Branch-Train-MiX [70] sets the pretraining pattern. From a shared dense seed, training branches into asynchronous parallel runs, which removes the inter-branch communication that dominates synchronous large-scale training. The resulting experts are then merged by combining their feed-forward parameters into mixture-of-experts layers and averaging shared self-attention weights, followed by a light finetuning stage that learns token-level gating.

Branch-Train-MiX is a pretraining method, and the harder problem is post-training, where reasoning and safety are installed and where forgetting does the most damage. Branch-Adapt-Route [42] closes that gap: each domain expert goes through its own mid-training, supervised fine-tuning, and reinforcement-learning pipeline in isolation, and the separately aligned experts are then joined into one mixture-of-experts with light router training. At the 7B scale with experts for mathematics, code, tool use, and safety, it reports an aggregate score of 49.1 across seven evaluation categories against retraining baselines of 47.8 without mid-training and 50.5 with [42].

That last comparison deserves attention, because it is easy to read as a stronger result than it is. Branch-Adapt-Route *matches* a retraining baseline; it does not beat one. Modularity buys update economics and containment, not quality. Nobody should read this architecture as saying a Mixture-of-Towers is better than a monolith of the same size. What it says is that it is affordable to maintain and safe to revise.

3.3 One knob: how much lineage towers share

Branch-Train-MiX begins "from a shared dense seed". That phrase is a design parameter, and the rest of this paper has been setting it to different values in different sections without saying so. We name it here, because every mechanism in the architecture depends on it, they do not all want the same value, and the round-2 audit did not catch the inconsistency because each of its streams read one section.

Write λ for the degree of **lineage sharing** between towers. A three-point ladder is more useful than a continuum, because what matters is where the discontinuities fall:

$\lambda = 2$**, shared corpus.** One dense seed, one continued-pretraining corpus, towers differing only in post-training. Representations stay commensurable in the strong sense: same basis, same layer count, same attention geometry.

$\lambda = 1$**, divergent branches.** One dense seed, then asynchronous branches over different corpora, which is exactly Branch-Train-MiX [70]. Representations sit in a common basis inherited from the seed and drift with branch length.

$\lambda = 0$**, independent.** Different initialisations, different corpora, different objectives, no shared basis at all.

High λ buys four things. Key-value reuse between towers, because cached keys and values are the output of learned projections and are interpretable only in the basis that produced them. Hidden-state mixing, for the same reason. The Branch-Train-MiX merge, which averages shared self-attention weights across branches [70] and is meaningless across independently initialised models, whose neurons sit in unrelated permutations, so weight-space averaging destroys both. And adapter transfer, since a tenant adapter fitted against one tower warm-starts on another only if the two are near each other in weight space.

Low λ buys the three things this paper has argued the towers are for: the corpus disjointness Proposition 2 monetises, the alignment isolation of (T2), and the informative difference §12 holds is exploitable. At $\lambda = 2$ the towers are one model wearing five hats, and the diversity argument has nothing left to stand on.

Table 2 is the disclosure. The paper as written assumes at least three different values of one parameter.

Mechanism, and where used	**Needs**	**Why**
Branch-Train-MiX merge (§3)	$\lambda \geq 1$	Averages shared attention weights across branches
Branch-Adapt-Route (§3)	$\lambda \geq 1$	Joins separately post-trained experts into one mixture-of-experts
Key-value reuse in a size family (§3.5)	$\lambda = 2$	Within the family only; keys and values are basis-dependent
Adapter warm start (§8)	$\lambda \geq 1$	Transfer needs weight-space proximity
Corpus disjointness, Prop. 2 (§2.3)	$\lambda \leq 1$	The benefit lives in the union of the corpora
Partition axis P1 (§3.5)	$\lambda \leq 1$	The criterion is stated over corpora
Diversity position, F13 (§12)	$\lambda = 0$	F13 excludes two finetunes of one checkpoint

Table 2: Where each mechanism in this paper sits on the lineage parameter. The last row and the first two rows cannot both be satisfied, and Branch-Adapt-Route is the one the architecture cannot drop.

The bottom row and the top two rows are inconsistent, and the conflict resolves against the diversity position rather than against the method, because Branch-Adapt-Route is the central bet of §3.4 and there is no version of this architecture without it. **So "independently trained towers", which is §3's phrase for its third property and what the abstract's "trained separately" is naturally read as, has to weaken to "one common seed, five divergent continued-pretraining branches"**, that is, $\lambda = 1$ and not $\lambda = 0$. Everything downstream of the diversity argument inherits the weaker object. §12's position is stated against persona-level heterogeneity, and a branch of a common seed is a larger perturbation than a persona and a smaller one than an independent pretraining run. Where on that line it stops counting as informative difference is not something this paper knows, and the measurement table already

implies the answer is not free: F13's limb (a) rules out "two finetunes of one base checkpoint" as an instrument precisely because a shared checkpoint is suspected of destroying the treatment. A Branch-Train-MiX branch is longer than a finetune, and nothing here says it is long enough. We do not resolve this by argument. It is measurement F14, and it runs on the same hardware as F13's limb (a).

One consolation runs the other way, and it is arithmetic rather than argument. Writing g for the fraction of each tower's token budget consumed before the branch point, the seed corpus is shared by construction, so the residency-bound fraction f of §11.5 obeys $f \leq 1-g$ and Eq. (19)'s unique-supply requirement is bounded by

$$5.6 \times 10^{13}\,(1+4f) \;\leq\; 5.6 \times 10^{13}\,(5-4g),$$

which at $g = 0.98$ is 6.05×10^{13} and at $g = 0$ is the undecomposed 2.8×10^{14}. §11.5 finds that the data-wall motivation survives against the central token-supply estimate only for f below about 0.02. A common seed consuming 98 percent of each tower's budget is one way to guarantee that, and it is the same knob, turned the same way, that costs the diversity. The architecture cannot have both, and this paper had been quietly billing for both.

Three things that arithmetic is not, stated here because companion documents have read it as more than it is. It is not a necessity: $f \leq 1-g$ is an *upper* bound on the requirement, so $g \geq 0.98$ suffices and is not needed, and $g = 0.20$ with $f = 0.02$ satisfies the supply constraint while leaving 80 percent of each tower's budget post-branch. Below $g = 0.98$ the token argument for towers is undetermined, not dead, and what determines it is the corpus overlap rather than the schedule. It is not settled by arithmetic: the verdict is a function of a point estimate inside a factor-of-ten interval. Writing S for unique supply and r for tokens per total parameter, the requirement meets supply at $g^\star = \big(5 - S/(r \times 2.8 \times 10^{12})\big)/4$, which at $r = 20$ is $g^\star = 0.982$ at the central 6×10^{13}, 0.357 at the 90 percent upper bound of 2×10^{14}, and *unsatisfiable at any g* at the lower bound of 2×10^{13}. And it is not improved by fixing r: at the measured token-per-total-parameter ratios of two published trillion-scale sparse models, 14.9 for Kimi K2 and 22.1 for DeepSeek-V3, $g^\star$ moves to 0.890 and to 1.008, the second unsatisfiable at any g, so correcting Eq. (1) in the direction F1 points hurts the architecture rather than helping it. What survives all three qualifications is not a statement about g at all. It is §11.5's: a corpus overlap high enough to meet the central supply estimate is a corpus overlap high enough to cost the diversity position, and $f \leq 1-g$ adds an upper bound on f where the diversity position needs a lower one.

Two things here we could not verify from the sources we hold, and they are marked rather than asserted. Whether Branch-Adapt-Route *strictly* requires a common seed, as opposed to being shown on one, is a reading of [42] and not a quoted statement; the same question for the Branch-Train-MiX attention-averaging step is a stronger inference and we believe it is sound, since averaging weights across unaligned bases is a known failure. Both belong in the prior-art sweep of `logos/PRIOR_ART_v03.md`, together with the model-merging literature on permutation alignment, which is where a $\lambda = 0$ merge would have to come from if one exists.

3.4 The extrapolation, named

Branch-Adapt-Route is shown at 7B with four experts. This architecture applies it at 2.8T with five towers, a four-hundred-fold extrapolation in parameters. Nothing in the published result licenses that.

The failure mode to expect gets worse with scale rather than better. When experts are small and narrow, a misroute is cheap. When each expert is a frontier model, a misroute costs the full quality gap between two frontier models, and the router is trained on far less signal than any tower it dispatches to. That is measurement F2, and it is the architecture's central unhedged bet.

One adjacent composition question has since been measured. It settles nothing about 2.8T and it does settle something about the design: applying a tower's slice more than once degrades held loss monotonically from the first repetition onward (§15.6). Composition in this architecture means *routing* to one tower, not chaining through several, and that measurement says the distinction has to be kept deliberately rather than assumed. An arrangement that stacks towers unconditionally discards the capable initialisation that makes upcycling cheap in the first place, and a cascade is only viable if its later stages are gated to a no-op at step 0 — which Proposition 3 shows is not a free thing to ask for.

The next section is our attempt to reduce the risk by actually designing the router rather than naming it, and §4.5 reports what happened when that design was built.

3.5 How many towers, and which

The five-way split into Code, Life Sciences, Mathematics, Logic, and Administration runs through this paper as though it were a specification. **It is an illustration, and the distinction is load-bearing enough that getting it wrong invalidates the subsection you are reading.** §3.6 makes the case; the short version is that in an end-to-end trained mixture nothing is named, the router learns the split, and no tower is the Mathematics tower. The criterion below is therefore stated for the case where an operator *does* partition a corpus deliberately — Branch-Train-MiX with hand-chosen branch corpora — and it does not apply to the emergent case at all.

For that deliberate case, a partition chosen by intuition is the thing routing was invented to avoid, so here is a criterion instead. Towers should be separated when they differ on at least two of three axes:

(P1) **Corpus disjointness.** Proposition 2's benefit comes entirely from $\left|\bigcup_i \mathcal{C}_i\right|$ exceeding $\max_i |\mathcal{C}_i|$. Two towers drawing on the same corpus contribute nothing to it.

(P2) **Objective conflict.** Branch-Adapt-Route's benefit is isolating reinforcement learning that would damage a neighbour. Two domains whose alignment objectives agree do not need isolating from each other.

(P3) **Update cadence.** Linear update economics only pay when towers are revised at different rates. Domains revised together can share a tower at no cost.

Applying the criterion gives an uncomfortable answer. Mathematics and Logic share most of their corpus, share an alignment objective, and are revised on the same cadence; they fail all three axes and should merge. Administration separates cleanly on all three: its corpus is policy and procedure, its objective actively conflicts with capability maximisation, and it is revised whenever regulation changes rather than whenever capability improves. Code and Life Sciences separate on corpus and cadence.

So the criterion says **four towers**, not five: Code, Life Sciences, Mathematics-and-Logic, and Administration. **That verdict has since been measured, and it is wrong in both**

directions (§15.2). Mathematics and Logic are the *most* opposed pair in the corpus, and Administration — the domain this subsection says separates cleanly on all three axes — is the one that merges. We leave the argument standing above rather than quietly restating it, because what the measurement contradicts is not only the conclusion but the way it was reached, and that is the more useful thing to have on the record.

The reasoning above is an argument from **subject matter**. Mathematics and Logic are intellectually adjacent, a library shelves them together, and it is natural to conclude that a model would want the same parameters for both. A model does not see subject matter. It sees LaTeX preprints against discussion prose, and those pull its weights in opposite directions while a court opinion and a clinical paper pull them the same way despite having nothing in common as subjects. **Corpus overlap is a property of content; gradient conflict is a property of what the model needs, and P1 conflates the two.**

Overlap is also universal here — these corpora are all English, all technical, one byte vocabulary — so "do two domains overlap" does not discriminate, and P1 thresholds a quantity that has no natural threshold. Towers share structure whatever partition is chosen. What is worth measuring is whether the marginal tower earns its parameters, which is F16 and is done by removing it.

The criterion above is qualitative as stated, and a measurement cannot be built on "the level at which the criterion merges them" unless that level is a number. Each axis therefore gets an operationalisation and a cut point.

(P1) **Corpus disjointness**, operationalised as the Jaccard similarity of the two corpora's document sets under MinHash near-duplicate detection at a shingle length of 13 tokens. Two candidate towers *fail* P1, and therefore merge on this axis, at Jaccard ≥ 0.30.

(P2) **Objective conflict**, operationalised as the sign of the cross-domain effect: two domains fail P2 if reinforcement learning on either, run to a fixed budget, moves the other's held-out benchmark by less than one point in either direction. Domains that do not damage each other do not need isolating from each other.

(P3) **Update cadence**, operationalised as the ratio of the two domains' median inter-revision intervals over the last three years of the operator's own release history. They fail P3 at a ratio below 2.

Those cut points are engineering choices and nothing validates them. A reader who prefers different numbers should say which and re-run F11 against them, and so should we if a first run shows the bar is in the wrong place — that is ordinary instrument calibration, not a concession.

P1 has since been run, and it failed for a reason no choice of cut point repairs: on the four corpora measured, the entire dynamic range of the measure is narrower than the gap it is being asked to resolve. §15 reports what replaced it. P1 is left as written above because the replacement is only interpretable next to what it replaced.

3.6 Towers are learned, and the count is a hyperparameter

Everything above assumes an operator names the domains and then decides how many towers those names deserve. That assumption should be stated rather than inherited, because the architecture this paper argues for does not make it.

In an end-to-end trained Mixture-of-Towers the router is learned and the towers are not labelled. Nothing tells tower three that it handles mathematics; it handles whatever the router

sends it, and the router learns what to send from the loss. Under that reading the tower count is a **capacity hyperparameter**, in exactly the sense that an expert count is in a mixture-of-experts, and asking whether Mathematics and Logic "deserve" separate towers is a category error — no tower is the Mathematics tower. **The "four towers, not five" verdict of §3.5 is a statement about a deliberately partitioned corpus, and it does not carry over.**

The count is also not load-bearing for correctness. A system can train with five towers and serve with four: §4.2 already specifies that path, routing to the next-best tower and attaching a degradation flag. What a redundant tower costs is parameters and serving capacity, not correctness, so "how many towers" is a provisioning question and the honest way to settle it is to measure what each tower earns rather than to derive a number from a corpus taxonomy.

The router is never scored against a human label, and this is a rule rather than a preference. Not against a domain, not against a task family, and — the case that looks most defensible and is not — not against a *functional* axis either, such as which step of an agentic trajectory a segment came from. Substituting a better-chosen label for a worse-chosen one is the same error at a finer grain: it decides in advance what the router ought to have found and then grades it on the answer, which is the Branch-Train-MiX question wearing different clothes. A tower is a fully capable model specialised in ways that may have no name. The only admissible instruments are label-free: the oracle gap, the substitution margin against random and constant assignment, and the cost of removing a tower. Each of those asks what routing is *worth*, and none of them requires anyone to know what it found. (Governance is the one deliberate exception, and it is not a tower: §13 makes safety a router-level decision trained on labelled data, precisely because it is the one place where a name has to be imposed. It is the only exception governance needs. The second governance primitive, provenance attestation, requires no labels at all, because it tests what a tower was *shown* rather than what the router found: §13.5.)

One distinction inside that rule is easy to lose, and §4.5 makes it load-bearing, so it is stated here rather than left to be inferred. The rule governs what the router is *scored against*. It does not govern what the router is allowed to *read*. A router that pools its features from a later window of a segment, or that conditions on where in a context the segment sits, is using position as an **input feature**; that is an ordinary architectural choice, nobody outside the model has been asked what the right answer was, and the loss it is trained on is unchanged. A router graded on whether its assignment agrees with which step of an agentic trajectory the segment came from is using position as a **label**; that is the forbidden thing, for the reason the paragraph above gives, because it decides in advance what the router ought to have found. The two differ by which side of the loss the quantity sits on, and nothing else. Position on the input side is free. Position on the target side is the error. §4.5 changes a pooling window on exactly the first ground and reports what it cost, and no measurement anywhere in this paper scores a router against a trajectory step, a domain, or any other name.

The literature is more emphatic than a single observation, and it has moved since Mixtral. Because a router is a linear map on hidden state, hidden-state similarity is *necessary and sufficient* to explain expert usage similarity, so specialisation is a property of the representation geometry rather than of the routing architecture [50]. The same work reports that the resulting patterns resist human interpretation directly: expert overlap between different models answering the same question is no higher than between entirely unrelated questions, prompt-level routing does not predict rollout-level routing, and deep layers activate almost identically across semantically unrelated inputs. A separate line finds domain-*invariant* "standing committees" of

experts, against the assumption that routing is domain-specific at all [51]. **So a router doing its job perfectly is expected to score near zero on any human axis**, and a measurement that reports otherwise is more likely detecting the load-balance term (§15.3) than detecting specialisation.

There is a reason to expect the learned partition not to look like a human one. Mixtral's routing analysis searched for topical specialisation among its experts using subsets of the Pile — ArXiv, PubMed, PhilPapers — and did not find it: assignment correlated with syntax, token identity and positional locality, and the distribution over experts looked much the same whichever subset was fed in [52]. At token-level granularity, then, the question "do domains appear in routing" already has a published answer, and it is no. Whether *segment-level* routing into *deep* towers behaves differently is open, and it is the version of the question worth running, because it is asked on the corpus where the token-level version came back empty.

This also settles a tension with §2.2. Partitioning a corpus by hand, training one tower per partition, and routing over the result is a router over independently trained models, which is the design this paper rejects as deployed prior art rather than a new object. The emergent reading is the one under which a Mixture-of-Towers is a single model. The two objections are the same objection.

What follows for measurement is that the instrument changes. A partition criterion *predicts*, from corpus statistics, whether two towers will turn out redundant. Disabling a tower and measuring what the corpus loses *is* the redundancy, needs no threshold, no within-domain null and no assumption about how independent the corpora are, and — decisively for unlabelled towers — asks about tower three rather than about a named domain. Whether a learned router beats random and constant assignment on identical weights is the prior question, because a router that beats neither has produced a lookup table with extra steps.

The partition has a second axis, and it is not symmetric with the first

Everything above is one axis. The partition the reference architecture actually wants is two-dimensional, **domain crossed with size**, and the two axes have opposite properties on every quantity that matters. Stating them as one thing is what makes the lineage parameter of §3.3 look contradictory when it is merely underspecified.

On the **domain axis**, corpora are disjoint by construction, which is where Proposition 2 and the diversity position both draw. Lineage is low, and there is **no key-value reuse across domains**, because two towers that never shared a basis produce keys and values that are not commensurable: a cached tensor is the output of a learned projection and means nothing to a projection trained separately. Domain towers are peers.

On the **size axis** the arrangement is a distillation ladder, one parent per domain with smaller tiers distilled from it. Lineage is shared by construction, and **key-value reuse works along it**. The enabling fact is that the grouped-query attention pattern published families use holds `head_dim` and `num_kv_heads` fixed across sizes while varying only the query-head count and the layer count, so the per-layer key-value tensors of a small tier and a large tier are already the same shape. At the common configuration of 8 key-value heads and a head dimension of 128, one layer's key-value state for one token is $2 \times 8 \times 128 = 2048$ elements at every tier in the family. What actually blocks reuse is therefore not shape. It is layer-count mismatch, which is a correspondence problem, and the fact that the two tiers' projections were learned separately even from a common parent, which is a training problem. Both are attackable by training the ladder to agree; neither is a structural impossibility the way the domain-axis case is. The specific

family configuration is stated from memory of published model cards and we have not re-read them here; it belongs in `logos/PRIOR_ART_v03.md` for verification, and the argument degrades to "shapes may need aligning too" if it turns out to be false.

The size axis is close to free on the two quantities Proposition 1 says are binding. A 7B and a 70B tier alongside a 2.8T parent add 7.7×10^{10} parameters, which is **2.75 percent** of the tower, or 40.9 GB at MXFP4's true 4.25 bits, well under one accelerator. Training them by distillation at $D = 20N$ costs $6 \times 7 \times 10^{10} \times 1.4 \times 10^{12} = 5.88 \times 10^{23}$ FLOPs for the 70B tier and 5.88×10^{21} for the 7B, a total of 5.9×10^{23}, which against the tower's own 2.2 to 3.1×10^{25} is 1.9 to 2.7 percent. Both costs land near the same small number, which is a coincidence of $D = 20N$ and worth no more weight than that.

The "four towers, not five" verdict survives this unchanged. The criterion above is stated entirely on the domain axis: corpus disjointness, objective conflict and update cadence are all properties of what a tower is trained on and for, not of how large it is. Mathematics and Logic still fail all three axes as the criterion is argued — and §15.2 reports that the measurement contradicts that verdict, which is a fact about the criterion rather than about the size axis. What the size axis changes is the number of served artifacts, not the number of domains, and §11.3 notes the consequence for evaluation: a two-dimensional partition multiplies the arm count, and any metric that takes a maximum over arms degrades as it grows. The full specification of the ladder, including the distillation schedule and the reuse protocol, is in `logos/LADDER_ARCHITECTURE.md`; we do not import it here, because the paper says only that the second axis exists, is cheap, and does not disturb the first.

3.7 Four mechanics, and a roster fixed by kind

Domain and size are the two axes above. There is a third, and it is orthogonal to both: *how a tower runs*. Four mechanics cover every tower the companion harness builds, and they name nothing about what a tower is for.

code	mechanics	what it returns
`RSN`	a text turn with tools, its own dispatch and its own halt	an answer, or a hop
`SIM`	one prefill and a score over the next observation, latent rollouts	a distribution
`DEC`	one prefill and typed questions with calibrated probabilities, no text	probabilities
`GOV`	`RSN` mechanics carrying three yes/no readouts and the handshake toolbox	a report; the switch stays wi

A `SIM` tower predicts the next observation, which is what makes it a world model rather than a policy; the weather ladder and the next emulator tile are one head shape at two cardinalities. A `DEC` tower is the typed-decision shape of §12: it emits no text, and every answer is a probability scored against an environmental label. The governance tower is `RSN` mechanics with a typed end-of-turn readout, and enforcement is never inside it.

The roster is fixed by kind: one each of `RSN`, `SIM` and `DEC` per substrate, up to what VRAM holds. A tower specialises only through its own data. A tower's name on the wire is its learned prompt codes, and a substrate's name is a learned key: a short code sequence seeded from that substrate's data through the shared compressor, trained like a prompt, and prepended to every question on it. The three-letter codes are the only names written down.

Two consequences follow for the wire. First, a typed question travels as codes: each option is its latent through the shared compressor behind a reserved marker entry that data can never

produce, so the question is portable across nodes, its width is set per ask, and the signing and encryption of §13 cover it unchanged. The marker positions are read by a small scorer, a masked softmax over the hidden states at the markers, so a question with any number of options is one question. Second, no router stands in front of a `DEC` tower. A decision layer that both scored and dispatched would be the concentrated controller §13 warns against; the dispatch question is itself a typed question whose options are the roster's prompt codes, answered by the tower that holds the question.

4 The router

Every query passes through one router, and a router that dispatches between frontier-grade specialists is where the risk of §3 concentrates. This section gives it a design.

4.1 Where the training signal comes from

A router needs examples of the form (query, tower, outcome). Collecting them by serving every query on every tower costs k times inference, which is affordable on an evaluation set and not in production. The standard treatment is bandit feedback: the router picks one tower, observes only that tower's reward, and updates only the arm it pulled [7]. Contextual-bandit and cost-aware formulations are established [7, 60], and we adopt them rather than inventing anything.

The design that follows has three parts.

(i) **A capability prior.** Initialise from per-tower evaluation vectors, so that a fresh router starts at the quality of static domain-label routing rather than at random.

(ii) **Exploration on a sampled slice.** Route a small fraction ε of traffic off-policy and record the outcome. This is the only source of counterfactual signal, and its cost is ε times the quality gap, which is budgetable.

(iii) **Outcome signals, where they are free.** In verifiable domains the outcome costs nothing: tests pass or fail, a proof checks or does not, a program runs or crashes.

Part (iii) has a consequence worth stating plainly, because it is a structural property of the architecture and not a tuning detail. **The router will be well trained exactly where verification is cheap, and poorly trained everywhere else.** Code and mathematics generate free labels. Life sciences, policy, and open-ended reasoning do not. So routing quality will be uneven across towers in a way that correlates with domain verifiability, and the towers most in need of good routing are the ones that will get the worst of it. Any deployment should measure routing quality per domain rather than in aggregate, or this asymmetry will hide inside a good average.

4.2 What happens when a tower is unreachable

A distributed five-tower system will lose towers. The contract we adopt is a three-step ladder, and the important property is that every step is visible to the caller:

(1) Serve from the preferred tower.

(2) If unavailable, serve from the next-best tower and **attach a degradation flag to the response**.

(3) If no capable tower is available, decline at the router and say why. An earlier version handed the request to an Administration tower. That is retired: §15.2 finds no such tower is recoverable from a corpus partition, and a refusal is a decision about a request rather than a distribution to model, so it belongs to the routing layer that Eq. (21) already operates on.

Step 2's flag is the whole point. The Fable-to-Opus demotion path shows what silent degradation costs: a caller who cannot tell that a weaker model answered cannot compensate, retry, or escalate. A degradation flag turns an invisible quality loss into a handled error.

4.3 Router drift when a tower is swapped

Branch-Adapt-Route's economic case is that towers can be replaced independently. But every replacement invalidates whatever the router learned about the old tower, and if re-learning is expensive then update cost is not linear after all and the architecture's main advantage evaporates.

The fix is to make the router a function of *tower descriptors* rather than tower indices. Represent each tower by its evaluation vector, train the router over descriptors, and a swapped tower enters as a new descriptor with a warm start from its own evaluation rather than from scratch. Whether the warm start actually recovers most of the routing quality is measurable at the one-billion-parameter scale, and it is the single cheapest test of Branch-Adapt-Route's economic case that exists. It appears in §16.

4.4 Users steering the router

If a caller can influence which tower serves them, they can steer around the safety decision, which would make the compliance mechanism of §13 decorative. The concern is unchanged by §15.2 retiring the Administration tower; if anything it sharpens, because the thing being steered around is now a single router-level classification rather than a destination among five.

The defence is placement. The safety mask of Eq. (21) is applied *after* routing, as a mask over the permitted expert set, not as a preference the router weighs. A router preference can be outbid by prompt content; a post-router mask cannot, because by the time it applies, the routing decision is already made and the mask simply removes options. This costs nothing and it closes the obvious attack.

One interaction remains open and we flag it as a likely bug rather than a solved problem. Masking experts to $-\infty$ changes the routing distribution, and the quantile balancer of §5.3 balances exactly that distribution across a batch. Masking for one user therefore shifts quantile thresholds for every other token in the batch. A compliance control that quietly degrades unrelated users' routing is a governance failure and not merely a performance one. Nobody appears to have looked at this. It is measurement F6.

4.5 What the router received, measured

Everything above this subsection is specification. One implementation of it has now been built and instrumented, and what the instrumentation returned is a result about the specification

rather than about the implementation. The rig is the second of the two described in §15: a Qwen3-0.6B base upcycled into four towers on one RTX 3090, 1.257×10^9 total parameters against 5.96×10^8 active, trained for 0.096 epochs of an 84.9 M-token corpus of recorded agentic trajectories. Nothing at that scale settles anything about a 2.8T tower, and §15's scale caveat applies to every number below without exception. What a run this size can do is show that a mechanism the specification assumes into existence is absent from the artifact that implements it. That is what happened.

The router received exactly zero gradient. A 2×2 at initialisation, no training, four towers, gradient of the language-modelling loss with respect to the router projection:

combination rule	towers	$\|\partial\mathcal{L}/\partial W_{\text{router}}\|$
top-1, zero-initialised gate (the run as built)	identical	0.000×10^0
top-2 convex	identical	0.000×10^0
top-1 convex	symmetry broken	4.9×10^{-8}
top-2 convex	symmetry broken	6.1×10^{-4}
top-1, gate initialised at 0.1	identical	4.87×10^{-2}

The first row is the run as built, and its zero is exact rather than small. The gate is $g = 1 + \sigma\,(p_{\text{sel}} - 1/N)$ with the gate scale σ initialised at zero, so p_{sel} multiplies nothing and no derivative reaches the router at all. Replacing the gate with a convex combination over the top-k towers does not repair it on its own: the router's gradient is then a *difference* of tower contributions weighted by the derivative of the mixing weights, and towers upcycled as deep copies of one parent contribute identically, so the difference is exactly zero. Breaking the symmetry does not repair it on its own either, because at $k = 1$ the convex weight renormalises over a singleton, $w \equiv 1$ and $\partial w/\partial\text{logits} \equiv 0$. **Within the convex family, a live routing gradient needs both $k > 1$ and broken tower symmetry; neither alone is sufficient.** The 4.9×10^{-8} in the third row is a floating-point zero and should be read as one.

The fifth row is a separate and much cheaper intervention, and it is the one that completes the picture. It changes nothing about the combination rule, evaluates no second tower, and perturbs no weights. It declines to initialise the gate scale at zero, setting $\sigma = 0.1$, and the router's gradient goes from exactly zero to 4.87×10^{-2} — about eighty times the top-2 broken-symmetry row, at no additional FLOPs.

Nonzero is not the same as useful, and magnitude is the wrong axis on which to rank these rows. Under top-1 only one tower is ever evaluated, so the loss has no way to express the proposition a routing decision is about — that tower j would have done better on this segment than the tower that was chosen. The gradient the fifth row measures is a rank-one bandit term: alive, and carrying information only about the tower already taken. The 6.1×10^{-4} of the fourth row is two orders of magnitude smaller and is a *difference* between two towers that were both evaluated, which is a comparative signal and the thing the router actually needs. A reader who ranks the table by norm will draw the opposite conclusion from the correct one, which is why the two quantities are named rather than merely printed.

None of this is an implementation defect that a careful engineer avoids. It is a property of design choices the specification makes independently and never reads together, and it generalises.

Proposition 3 (Exact parent-identity at initialisation and a live routing gradient are mutually exclusive)**.** *Let the towers be deep copies of one parent and let the router's influence on the*

output be a scalar gate g applied to the selected tower. Exact parent-identity at initialisation — the upcycled mixture computing precisely what the parent computed, which is the property that makes sparse upcycling cheaper than training from scratch and is therefore the reason to upcycle at all — requires $g = 1$ on every segment. The gate varies only through p_{selected}, so requiring $g \equiv 1$ requires $\partial g / \partial p_{\text{selected}} = 0$, and that partial derivative is the only path by which the router's parameters reach the loss. Exact parent-identity at initialisation and a nonzero routing gradient at initialisation are therefore not simultaneously satisfiable. One of them has to be given up, and the choice should be made deliberately rather than inherited from whatever an initialiser happened to do.

The fifth row of the table is that proposition being paid rather than a counterexample to it. Setting $\sigma = 0.1$ *is* the choice to give up exact parent-identity, and the harness now says so out loud: the upcycled model's output at initialisation departs from the parent's by a maximum absolute delta of 4.004×10^{-1}, and the gate reports **parent-identity given up** where it previously asserted parent-identity without checking. The proposition does not say that a routing gradient is unobtainable. It says which of two properties has to be surrendered to get one, and the useful consequence is that the surrender should be a line in a configuration that somebody chose, rather than a default that nobody read.

A second property of that gate is worth separating from the first, because it survives after σ has been trained away from zero and so is not fixed by fixing the initialisation. The quantity $p_{\text{selected}} - 1/N$ is non-negative always, since the probability of the argmax cannot fall below uniform. The gate scalar's gradient therefore never changes sign on whether the route taken was the right one. It is a global gain knob on the selected tower's contribution, and no amount of training turns a quantity with a fixed sign into a signal about which tower should have been chosen.

The router was reading the wrong window, and this is a change to the specification. Pooled-feature spread across 48 held-out segments, trunk forward pass only, by the token window the router pools over:

pooling window	coefficient of variation	PC1 share
[0:128] (what the router pooled)	0.0835	0.4232
[128:256]	0.3636	0.3727
[256:512]	0.3260	0.4202
[512:1024]	0.3100	0.4539

Agentic trajectories share an identical system-prompt prefix by construction — the same tool schemas, the same instructions, the same formatting rules, all emitted before anything task-specific — so the first 128 tokens of every segment are near-identical, and the window the router pooled carries a between-segment spread of 0.0835 while the very next window carries 4.4 times as much. **The router was structurally blind: it had been handed the one region of the segment on which the segments do not differ.** Training cannot repair that, because the information is not in the input, and neither can a better routing objective.

The repair is to pool a later window, and it is not free. Routing has to stay causal with respect to the loss, so the loss must start after the pooled region, and moving the window right costs loss coverage — tokens that would have been predicted are spent on being read instead. **The finding worth carrying is that routing granularity has a *sequence-position* dimension,**

which this paper had not named. Granularity has been treated throughout as one quantity, token against segment against query (§9.2, §15.5), and it is two: how much text one routing decision covers, and *where in that text* the decision is taken from. The naive answer to the second — pool the earliest available window, because it maximises loss coverage and needs no thought about causality — is on this corpus the worst available answer, and it is the answer an implementation arrives at by default.

This is a change to what the router *reads*, and the rule of §3.6 is what makes it admissible. Choosing a later pooling window selects an input feature. It does not score the router against position, and the paragraph in §3.6 exists because the two are easy to confuse: had the repair been "grade the router on which step of the trajectory the segment came from", it would have been the forbidden thing rather than the fix.

What a step is, counted rather than assumed. The two paragraphs above speak of segments of an agentic trajectory as though the unit were self-evident, and it is worth pinning to the corpus instead. Over 150 trajectories the marker counts are `assistant` 2,181, `tool_result` 2,052, `user` 195, `task` 150 and `system` 150 — 31.5 markers per trajectory, in the shape task → system → user → assistant → tool result → assistant → ⋯. An assistant turn is either a tool call or a prose answer, and the split is lopsided: 2,052 of the 2,181 assistant turns, 94 percent, are followed by a tool result and are therefore calls, while 129, 6 percent, are final answers. **The corpus is** 89.5 **percent assistant/tool-result alternation**, 4,233 markers of 4,728. The agentic loop is not a feature of this data. It is the data.

That is the empirical basis for a refinement the specification should carry: the routing unit wants to be a *step* rather than a fixed window. A step has a boundary the corpus supplies, it is the granularity at which the trajectory's own structure changes, and it does not pool the shared system prefix and the task-specific content into one vector the way a fixed 128-token window does — which is the failure the table above measures. "Pool a later window" is a workaround for a segmentation chosen without reference to the data; "pool the step" is the segmentation the data already has. Segmenting on steps is a choice about the *input* in the sense §3.6 separates: it changes what one routing decision covers, and it neither tells the router which step it is looking at nor scores it for getting that right.

A collapse reading, corrected downward. Over the same 48 held-out segments the routing argmax distributes as [0.104, 0, 0, 0.896]: a normalised entropy of about 0.24, with two dead towers out of four. Earlier readings in this work of "routing entropy exactly 0.000" came from a histogram over *four* samples, and a four-sample histogram over four towers can hardly report anything else. Those readings were an instrument artifact. What remains is worse than a nuisance and better than they said: skewed routing with dead towers is not a constant function, and the difference is not cosmetic, because a constant router has no distribution left to move whereas a skewed one has one that a working gradient and a legible input window could act on. Any "exactly zero" entropy reading elsewhere in this work should be checked against the sample count of the histogram behind it.

Taken together, the first and last of these findings say what the mixture arm of §15.6 actually was. With no gradient reaching the router and 89.6 percent of segments landing on one tower, its forward pass was a trunk, one tower and a head. §15.6 reports that it did not beat its dense control at 2.11 times the parameters, and against the two results above that is the expected outcome rather than a surprising one.

Routing entropy at initialisation is a seed lottery, and this harness never seeded it. Two runs were probed at step 0, before any optimiser step, over 256 held-out segments. One returns a normalised routing entropy of 0.018 with an argmax share of [0.004, 0.996, 0, 0]; the other returns 0.784, near-balanced over the four towers. That is a forty-three-fold difference between two readings of an untrained router, and the first thing it exposes is a defect in the harness rather than a property of the architecture: the global generator was never seeded, so the router projection — the matrix that decides everything the paragraphs above measure — was drawn afresh on every run, and no record was kept of the draw. Two other differences between those runs are stated rather than assumed away, because they bear on the same reading. The balanced run carried selection noise at $\sigma_{\text{explore}} = 0.5$ during its step-0 probe, and its gate scalars were drawn per tower rather than shared. **So the forty-three-fold spread cannot be attributed to the initialisation draw alone, and the conclusion does not require that it be.** An unseeded draw plus two uncontrolled factors means no single reading of this instrument identifies anything, which is what the repository's own record already said from another direction: three replicated seeds of one configuration returned mutual information between domain and tower of 0.118, 0.392 and 0.782 bits of the 2.322 available, a between-seed standard deviation of 0.334 on a quantity whose reported *differences* are an order of magnitude smaller than that.

The consequence is a rule, and it applies retrospectively to this paper. Every routing figure reported in this work from a single run — the [0.104, 0, 0, 0.896] of the paragraph above included — is one draw from a distribution nobody characterised. Such a figure establishes that the configuration *can* produce that shape; it does not measure what the configuration does produce. §16 accordingly requires at least three seeds for any routing figure, and the harness now takes a seed argument so that the draw is recorded rather than inherited from whatever the process started with.

A live routing gradient is necessary and not sufficient. The fifth row of the table above is the cheap repair — initialise the gate scale at 0.1 instead of at zero and the router's gradient becomes nonzero — and it has now been trained rather than only measured at initialisation. Over 500 steps the gradient behaves as a live gradient should. Normalised routing entropy rises from 0.0184 to 0.0581, a factor of 3.2; the minority tower's share of 256 held-out segments grows from 0.4 to 1.6 percent; the gate scalar itself moves off its initialisation, from 0.10000 to 0.10025. **And two of the four towers hold exactly** 0.0000 **share at every probe** — not a small share but zero segments, at step 0, at step 250 and at step 500 alike.

That zero is self-sustaining, and the mechanism is arithmetic rather than statistical. Under top-1 dispatch a tower that never wins the argmax appears in no forward pass, so it receives a gradient that is bit-exactly zero, so its weights do not move, so its routing logit does not improve, so it does not win the next argmax either. **A live routing gradient redistributes traffic among the towers that already carry some; it has no path to a tower carrying none.** The two mechanisms that do have such a path are of different kinds, and keeping them distinct matters because they are budgeted differently. Noise on the selection logits reaches the starved tower from the *routing* side, by occasionally sending it a segment it did not win, at a cost in routing quality that anneals away. An auxiliary objective — a load-balance term, or a forced route scored on its own loss — reaches it from the *gradient* side without waiting for the router to choose it, at a cost in gradient interference with the primary objective, which is the cost §5.3's auxiliary-loss-free line exists to avoid. Neither is a tuning detail. **Under top-1 dispatch an upcycled mixture needs an explicit reachability mechanism, or its effective tower**

count is settled within the first few hundred steps and cannot be revised afterwards. §5.3 specifies balancing for micro-sparsity inside a tower and F3 tests it there; this measurement says the tower layer needs its counterpart, and that a mixture reporting a live routing gradient is not thereby a mixture using its towers. It is also the same failure §15.4 reaches from the other end, where the arms whose routers collapsed onto one tower returned the largest apparent specialisation margins in the set.

4.6 This section specifies a known component, and its related work is incomplete

We flag a gap here rather than assert a result, because the check that would close it has not been run. The master router as specified above is a routing layer over heterogeneous models, and that is deployed prior art rather than a new object. OpenRouter operates one commercially: a single endpoint over models from many providers, with an automatic router that classifies each prompt into one of roughly thirty task types and selects a model, reporting back which one served the request [56]. There is also an academic literature on the same problem, of which we verified three entries against their primary sources. RouteLLM trains routers on preference data to dispatch between a stronger and a weaker model at a chosen cost-quality point [53]. FrugalGPT gives the cascade formulation, calling models in sequence until a scored answer is good enough [13]. Hybrid LLM routes by predicted query difficulty against a quality target [19]. These sit alongside the bandit-feedback and survey work this section already builds on [7, 60].

The gap is one of process and we name it as such. The independent round-2 audit of this paper examined the observation-bound literature of §12 for missing related work and found some, and it never examined the routing literature at all. So the four entries above are what we confirmed, not the result of a sweep, and the nearest prior art to the specific design given above may well be closer than anything cited here.

The consequence is a framing correction that applies to the whole paper. §4 *specifies* a known component, with the verifiability asymmetry, the descriptor warm start and the post-routing mask as engineering choices within it, and it should not be read as a contribution. The paper's contributions sit elsewhere: the four defects diagnosed in §11.3 in a routing metric of our own, whose replacements are adopted from published work and are no contribution at all, the observation bound of §12 together with the position §12 states against it, and the regulatory-arbitrage argument of Position 3. Nothing about the router is presented as novel anywhere in this paper, and if any sentence elsewhere reads that way, this one governs.

5 Micro-sparsity inside a tower

5.1 Heterogeneous experts, and why routers abuse them

Tokens differ enormously in the computation they deserve. Punctuation needs almost none; multi-step deduction needs a great deal. Experts of uniform size therefore misallocate by construction. Heterogeneous mixture-of-experts assigns experts of differing sizes, pairing large reasoning experts with light syntactic sub-networks, and adds a training objective that pushes activation toward the smaller ones [77].

That objective is necessary because of a specific pathology. A softmax router minimising cross-entropy learns early that larger experts cut loss faster, and concentrates on them. Smaller

experts then get too little gradient to become useful, and the large experts become the throughput bottleneck. We call this the over-activation trap, and heterogeneous mixture-of-experts counters it with a penalty that prices expert capacity into the routing decision [77].

Mixture of Heterogeneous Grouped Experts [37] handles the systems consequence, which is uneven accelerator use and wasted parameters. It groups experts so that size is uniform within a group and varies across groups, adds two-level routing for resource-aware combinations, and uses a group-wise auxiliary loss to steer tokens toward the cheapest group that can handle them.

Remark 1 (A term we could not find)**.** *Informal accounts of this architecture refer to an "All-size Group-decoupling Allocation" mechanism. It does not appear in [77], in [37], or anywhere else we could find. We drop it rather than cite it to a source that does not contain it.*

5.2 Routing in a compressed subspace

Dispatching high-dimensional token vectors across accelerators creates an all-to-all communication bottleneck, and in latency-sensitive serving the dominant cost is moving expert weights out of high-bandwidth memory. That regime is memory-bound, so arithmetic intensity does not matter and bytes moved does.

LatentMoE [46] attacks both costs with one change: a shared down-projection before dispatch and an up-projection after combine, so routing, expert computation, and all inter-device communication happen in a smaller space. A hidden state $\mathbf{x}_t \in \mathbb{R}^d$ becomes $\mathbf{z}_t \in \mathbb{R}^\ell$ with $\ell < d$ under a shared matrix:

$$\mathbf{z}_t = \mathbf{W}_{\text{down}}\,\mathbf{x}_t, \qquad \mathbf{W}_{\text{down}} \in \mathbb{R}^{\ell \times d}. \tag{5}$$

Expert selection is still computed on the full dimension d, which preserves gating expressiveness, while expert computation runs in the compressed space:

$$\mathbf{y}_t = \sum_{j \in \text{top-}k} g_j(\mathbf{x}_t)\, E_j(\mathbf{z}_t), \tag{6}$$

with the result projected back through $\mathbf{W}_{\text{up}}$. Communication volume and weight-load bandwidth both fall by the compression ratio d/ℓ; a representative setting compresses 4096 to 1024 [46]. Kimi K3 uses a stabilised variant as its core sparsity framework [41].

Remark 2 (Attributing the speedup correctly)**.** *The reported 3.5-fold inference speedup at matched accuracy is an end-to-end system figure for a deployed model, not an isolated measurement of the projection, and the accompanying parameter-savings framing is a vendor comparison against a chosen baseline [46]. We treat the compression ratio d/ℓ as the architectural quantity and the speedup as Tier B. The projections cost real arithmetic, $4d\ell$ per token per layer, $2d\ell$ for each of the down- and up-projections, which has to be charged against the bandwidth saved. Both must be charged. At $d = 4096$ and $\ell = 1024$ that is 16.8 MFLOP per token per layer, about one percent of a layer's compute at the active-parameter counts of §2.2 rather than half a percent. The trade is favourable because the regime is memory-bound, and it reverses in a compute-bound one. Finding the crossover needs a measured roofline on real hardware, not an analytic sketch.*

5.3 Load balancing without an auxiliary loss

Conventional mixture-of-experts training prevents routing collapse with an auxiliary load-balancing loss, which by construction fights the primary objective. The auxiliary-loss-free line [78]

replaces it with a per-expert bias added to routing scores and updated from recent load, which preserves causality and avoids gradient interference.

Kimi K3 reports a further step, **Quantile Balancing**, in which expert allocation comes directly from router-score quantiles: a token activates an expert when its score lands in the top quantile. The vendor describes this as deterministic, free of tuning hyperparameters, and producing even utilisation with no dead experts [41, 30]. The method is not the vendor's. It was developed by J. Su and described in a personal blog post in February 2026, months before K3's release [68], and it has been reported independently at 32B total with 5B active, 64 routed experts of which 4 activate, across 32 layers and 326B tokens at about 10^{22} FLOPs, with no auxiliary loss and no loss spikes [54]. Writing $s[i, j]$ for the router logit of token i at expert j, $b[j]$ for the persistent bias, n for expert count and k for the number activated, the alternating quantile solver computes

$$\alpha[i] = \text{quantile}_{1-k/n}\big(s[i,:] + b[:]\big) \qquad \text{over experts,} \tag{7}$$

$$\beta[j] = \text{quantile}_{1-k/n}\big(s[:,j] - \alpha[:]\big) \qquad \text{over tokens,} \tag{8}$$

and sets $b[j] \leftarrow -\beta[j]$ under stopped gradients, so balancing never contaminates backpropagation.

Quantile Balancing works on batch statistics, which leaves per-sequence imbalance untouched. That gap has preprint-grade treatment by a different route: Expert Threshold routing maintains per-expert exponential-moving-average thresholds over the token distribution, which balances load without an auxiliary loss and without any dependence on other tokens in the batch, and reports pretraining results to 2.4B parameters on FineWeb-Edu [72]. The mechanism this architecture adopts instead is the causal counterpart, **Causal Dual Bias**, which tracks expert usage through a dual variable β_t maintained by online dual descent within a sequence, and penalises later tokens by

$$\tilde{s}_t = s_t - \beta_t, \tag{9}$$

with β_t proportional to cumulative imbalance. This balances each sequence without breaking autoregressive causality [12].

Remark 3 (Where these two mechanisms actually come from)**.** *Quantile Balancing originates in a personal blog post [68] and is reported in a vendor technical blog [41], a third-party explainer [30], and an independent at-scale training report [54]. Causal Dual Bias is documented in a personal blog post [12]. Neither mechanism has a peer-reviewed or preprint source at the time of writing, though the causal per-sequence problem Causal Dual Bias addresses does have one by a different route [72], and the equations above are our reconstruction from those descriptions and the surrounding auxiliary-loss-free literature [78, 1], not transcriptions from a paper. Anyone implementing them should treat them as specifications to validate. That validation is a one-accelerator experiment and it is measurement F3.*

6 Attention and context at a million tokens

6.1 Kimi Delta Attention

Quadratic self-attention fails at million-token context twice over: computation grows as $O(N^2)$, and the key-value cache exhausts accelerator memory. Kimi Delta Attention [73] is a hybrid linear-attention module that extends Gated DeltaNet with a channel-wise diagonal gate in place

of scalar decay, so each feature dimension keeps its own forgetting rate. Unlike uniform-decay linear models it can hold a reasoning state in some channels while clearing others quickly. Its chunkwise algorithm uses a specialised variant of Diagonal-Plus-Low-Rank transition matrices, which cuts computation relative to the general form while staying closer to the classical delta rule [73].

The deployed configuration interleaves it with Multi-head Latent Attention in a 3:1 ratio, giving sequential dependency to the linear layers and exact global retrieval to the latent-attention layers. Reported effect: 75% less key-value cache and up to six times higher decoding throughput at one million tokens [73].

6.2 Why recurrence breaks prefix caching

Recurrent layers create a systems problem that quadratic attention does not have. Paged attention engines cache key-value vectors in predictable physical blocks, so any prefix boundary is a valid resume point. A recurrent layer instead carries a dense running state that updates at every token. Storing it at every possible match boundary is too expensive, and storing it only at coarse boundaries means two requests sharing nearly an entire prompt can miss the reusable prefix [76].

Sparse prefix caching [63] formalises the trade: keep exact recurrent states at a sparse set of checkpoints and, on a hit, resume from the deepest stored checkpoint and recompute the rest of the suffix exactly. Checkpoint placement under a distribution over overlap depths has an exact $O(NM)$ dynamic program, outputs are bit-exact, and no new recurrent kernels are needed.

The production integration separates three quantities that used to be one [76]: the **physical block size** at which state is allocated on device, the **scheduler alignment** where execution must stop so cache groups stay consistent, and the **prefix-match unit** at which a shared prefix is hashed. Separating them allows fine-grained matching inside a larger physical state block. The scheduler stops at the right block and hash boundaries so the recurrent state genuinely corresponds to the advertised prefix, and on a match the cached state is copied into a private destination before the request extends it. Full-attention and recurrent caches keep a synchronised computed-token count despite different physical block sizes.

We spend space on this because it is the clearest case in the architecture of a modelling decision creating a scheduler problem. Choosing channel-wise linear attention for its memory behaviour produces a cache-coherence problem with no analogue in what it replaced, and that cost is rarely priced when the mechanism is selected.

6.3 Delta Attention Residuals

In very deep networks, PreNorm residual streams dilute early layers: hidden states accumulate, and the running sum swamps individual sublayer outputs. Attention Residuals [32] replace additive residuals with learned softmax attention over previous layer outputs, giving selective cross-layer routing. But attending over *cumulative* hidden states, which are highly similar between adjacent layers, causes routing collapse deeper in the network: attention weights flatten toward uniform, with a maximum around 0.2 [36].

Delta Attention Residuals [36] attend over the change each sublayer contributes,

$$\mathbf{v}_i = \mathbf{h}_{i+1} - \mathbf{h}_i, \tag{10}$$

combined additively so the baseline residual stream survives intact:

$$\hat{\mathbf{h}}_l = \tilde{\mathbf{h}}_l + \sum_i \alpha_{i \to l}\, \mathbf{v}_i. \tag{11}$$

Because these increments are structurally diverse and sit in different abstraction subspaces, the attention distribution sharpens, with maximum weight around 0.6. Across scales from 220M to 7.6B, the mechanism beats both standard residuals and cumulative Attention Residuals, with validation-perplexity gains of 1.7 to 8.2 percent [36]. The additive form matters for deployment: at initialisation a uniform softmax output is a bounded perturbation on a preserved residual stream, so existing checkpoints can be converted by finetuning rather than retrained.

The thing worth testing independently is the conversion, not the routing contrast. Measuring that a delta-attention implementation produces delta-attention statistics is circular; measuring whether a real checkpoint converts without destabilising is not. The published gains stop at 7.6B and this architecture applies the mechanism at 2.8T. That is measurement F4.

7 Quantization

7.1 The memory arithmetic

A 14T ensemble at FP16 needs $2 \times 1.4 \times 10^{13} = 28$ TB of high-bandwidth memory. Four-bit microscaling is not four bits per parameter: MXFP4 stores E2M1 elements in blocks of 32 under a shared E8M0 scale (§7.2), and that scale is worth $8/32 = 0.25$ additional bits per parameter, so true density is 4.25 bits. The footprint falls to $1.4 \times 10^{13} \times 4.25/8 = 7.4$ TB, which a multi-node cluster can hold. NVFP4's finer blocks of 16 cost 4.5 bits and 7.9 TB. The field, this paper's earlier version included, routinely quotes the flat four-bit figures, which understate by 6.25 percent for MXFP4 and 12.5 percent for NVFP4; we use the true densities and note that even they are floors, because embeddings and output layers are normally kept at higher precision. On the same basis Kimi K3's MXFP4 weights occupy about 1.49 TB rather than the widely quoted 1.4 TB. Proposition 1 makes this the constraint sparsity does not touch, because every parameter must be resident whether or not it activates.

7.2 Closing the MXFP4 gap in software

The Open Compute Project's MXFP4 microscaling format uses blocks of 32 with a shared power-of-two scale, which is cheap in hardware. NVIDIA's NVFP4 uses blocks of 16 with finer scales and is more accurate, with a reported end-to-end accuracy gap of about ten percent [15].

Two software techniques close it [15]. **Overflow-Aware Scaling** increases the effective dynamic range under the same power-of-two constraint by mapping the block maximum into an overflow-aware range, which cuts error on the low-magnitude bulk of the distribution. **Macro Block Scaling** allocates a higher-precision scale at coarser granularity, an outer multiplier that shifts the input distribution into MXFP4's representable range before block quantization. Quantization fidelity is destroyed disproportionately by well under one percent of elements, which is why an outlier-targeted second scale buys so much. Together the two reduce the gap from about ten percent to under one percent on average, at a mean GEMM overhead of 6.2 percent.

Remark 4 (Two numbers we corrected and one we wrongly discarded)**.** *Earlier statements of this architecture attributed a nine percent compute overhead to this mechanism. The published*

figure is 6.2 percent [15], and the nine percent appears to be a conflation with the projection overhead of §5.2, a different mechanism in a different part of the network. Separately, a twelve percent tensor-core die-area saving is sometimes quoted for MXFP4. It is the closing clause of the abstract of [15], the paper cited twice in this remark, which reports "12% relative area savings in tensor cores" for MX against NVFP4. No argument here depends on it.

7.3 Quantization-aware distillation

Post-training quantization fails on large mixture-of-experts models because calibration cannot resolve routing artifacts. Quantization-aware *training* is worse for a model that has been through multi-stage fine-tuning, reinforcement learning, and merging: applying next-token cross-entropy to the low-precision student forces it to relearn base objectives, overwriting alignment installed later.

Quantization-aware distillation [47] aligns the quantized student to the full-precision teacher's output distribution under KL divergence,

$$\mathcal{L}_{\mathrm{QAD}} = D_{\mathrm{KL}}\big(p_{\mathrm{teacher}} \,\|\, p_{\mathrm{student}}\big) = \sum_{y} p_{\mathrm{teacher}}(y \mid x)\, \log \frac{p_{\mathrm{teacher}}(y \mid x)}{p_{\mathrm{student}}(y \mid x)}, \tag{12}$$

and is reported as stable for exactly the multi-stage post-trained models where quantization-aware training is not, and robust to incomplete data coverage, so recovery does not need the original training data [47]. For a tower ensemble that last property decides the matter, since tower post-training corpora may belong to different parties and distillation needs only teacher logits.

7.4 Measured: MXFP4 against FP16, and what survives compression

Everything above this subsection is read from the literature. Three results taken from it have now been run on this work's own instrument, and the run is worth reporting for one property before any of its numbers: it carries **eight seeds**, which is more replication than any other measurement in this paper.

The substrate is Zipf-distributed key recall on a twelve-layer decoder at $d_{\mathrm{model}} = 768$ over a 512-symbol vocabulary, about 8.5×10^7 parameters, trained for 3,000 steps per seed. It was chosen because its Bayes ceiling is computable rather than assumed — with label noise 0.15 over 512 symbols the attainable optimum is 0.8503 — so every arm reports as a fraction of an optimum instead of against an arbitrary target. Quantization is fake-quantize in float32, applied to two-dimensional non-embedding weights only, which measures representational error rather than tensor-core behaviour; the throughput half of the format question needs hardware this work does not have. Nothing here is at tower scale and §15's caveat applies unchanged. What a run this size can do is show whether a number format destroys information a trained network was using.

arm	accuracy	s.d.	percent of Bayes
FP16	0.8246	0.0021	97.0
MXFP4 vanilla	0.8244	0.0024	97.0
MXFP4 + OAS	0.8245	0.0025	97.0
MXFP4 + OAS + MBS	0.8248	0.0025	97.0
NVFP4 (emulated)	0.2036	0.0251	23.9
int4 block (AWQ-style)	0.8244	0.0020	97.0

MXFP4 is indistinguishable from FP16 on this substrate, at 97.0 percent of the ceiling in every arm that is not broken, the block-int4 control included. Four bits and change cost nothing measurable here, which is the assumption §7.1's memory arithmetic rests on and which this paper had been taking on the strength of a single published report [15].

Three verdicts were registered before the run. All three return true and only two of them mean anything.

C1, the gap between MXFP4 with both corrections and NVFP4, passes against a control we broke, and we report it as void. The emulated NVFP4 arm collapsed to 23.9 percent of the ceiling, and a format with finer blocks and a finer scale type cannot be genuinely worse than one with coarser versions of both on the same element grid. The cause is in our emulator and it is specific: the E4M3 block scale is snapped onto the eight-point FP4 grid and rescaled, rather than onto the 256-point grid it stands in for, so almost every block scale rounds to the floor and the block is annihilated. The verdict's own threshold — a relative gap under one percent — is then passed by a margin of -0.88, which is not the quantity being tested but the wreckage of the control. **Nothing in this paper says anything about NVFP4**, and the measurable form of C1 requires an emulator that reproduces the E4M3 scale grid.

C2, that the corrections are detectable at all, holds and is small. Overflow-Aware Scaling and Macro Block Scaling together move the pseudo-perplexity $-\ln(\text{accuracy})$ by 5.6×10^{-4} nats against vanilla MXFP4, paired $t = +2.45$ over eight seeds on a quantity of about 0.193. That is a real effect at three parts in a thousand, detectable only because the design is paired. On this substrate the corrections are not what makes MXFP4 usable, because vanilla MXFP4 was already at the FP16 figure; §7.2's published ten-percent gap is a statement about models and tasks this substrate does not reproduce, and the honest reading is that the corrections are cheap and measurable rather than that they were load-bearing here.

C3 is the verdict with reach past quantization, and it is the one to carry. An aligned arm was built by mixing a sentinel behaviour into a quarter of the training batches — a reserved symbol outside the value range, so that answering it correctly is unambiguous evidence of the installed behaviour rather than of having memorised the recall map. The aligned FP16 model retains it at 0.9775. Both quantized arms were then recovered for 200 steps: quantization-aware *training* on cross-entropy over base data, and quantization-aware *distillation* on KL to the aligned full-precision teacher.

arm	base accuracy	sentinel retention
aligned FP16	0.8183	0.9775
aligned, quantization-aware training	0.5556	0.0022
aligned, quantization-aware distillation	0.5510	0.5452

At base accuracies that differ by 0.0045 — statistically a difference, at $t = -2.30$ in the training arm's favour, and inside the equal-recovery tolerance registered before the run —

quantization-aware training destroyed the installed behaviour and distillation retained more than half of it, a difference of 0.543 ± 0.026 at $t = +21.07$ over eight seeds. This is the largest effect in the run by two orders of magnitude, and it is not a statement about number formats. It is a statement about what survives compression: the behaviour installed last is the behaviour a cross-entropy recovery pass overwrites first, because relearning the base objective is exactly what that pass optimises. §13.4 carries the deployment consequence.

Two qualifications belong with C3 and neither is dispensable. *The arms differ in two ways at once.* The distillation arm is trained against an aligned teacher *and* sees sentinel batches in its recovery mix; the training arm gets base data only. That is the contrast the mechanism predicts, and it is still a confound between objective and data mix, which the ablation that has not been run — cross-entropy recovery with sentinel batches in the mix — would separate. *And both recovery arms are damaged.* Post-training quantization alone, with no recovery whatever, holds 0.8248 on the same weights, where both recovered arms sit near 0.55. A 200-step recovery pass hurt the base task badly in both, so C3 establishes the direction and the size of the gap between two damaged models, not the quality of either.

8 Multi-tenancy

8.1 Adapters over a frozen backbone

No operator can host a dedicated 14T cluster per client. The pattern is to freeze the backbone and map client data in through low-rank adaptation [29], with thousands of adapters batched over frozen experts by segmented gather kernels [14, 62]. Mixture-of-LoRA work [81] treats the adapters themselves as a routed expert set.

8.2 Dimensional collapse under a shared codebook

Discretising continuous states against a shared residual-quantized codebook [33, 58] creates gradient interference under multi-tenant updates: one client optimising the codebook for financial telemetry and another for genomic structure pull the same hierarchical indices in opposite directions.

The resulting failure is a documented mathematical pathology, not a tuning problem. Spectral analysis of vector-quantized autoencoders shows that although the embedding space may be defined at 128 or 256 dimensions, quantization suppresses low-variance components and the model compresses into an effective subspace of roughly **4 to 10 dimensions** [45]. Because this is a structural bias of the commitment objective, rank regularisation forces artificial variance at a cost in loss, and continuous-first warm-up strategies [6] delay rather than prevent it. Performance improves and then degrades as effective dimension rises, with the optimum low [45].

Divide-and-Conquer VQ [45] treats the cause instead of the symptom: split the latent space into several low-dimensional subspaces and quantize each independently. Each subspace then satisfies the model's preference for low dimensionality, while concatenation expands total expressivity combinatorially without triggering variance suppression. Binding subspaces to tenant-scoped routers also isolates client semantics.

8.3 The weakest link, and where we moved it

Here is an assessment the rest of this section does not support, offered because the reader should have it.

The shared codebook is the least motivated component of the architecture as originally drafted. Residual quantization and semantic identifiers are established for generative retrieval and recommendation [58], where discrete hierarchical identifiers *are* the interface. It does not follow that quantizing continuous hidden states through a shared codebook belongs in a language model's serving path, and none of the sources cited above put it there. The draft used a real pathology and a real fix to defend a placement neither source endorses.

We therefore restrict the codebook to two positions where discrete identifiers genuinely are the interface: the retrieval and long-term-memory layer, where semantic identifiers index tenant corpora, and the router's tenant-scoped index. In both, the collapse pathology and its remedy apply as described. In the hidden-state path they answer a question nobody asked. Measurement F5 tests the mechanism on real hidden states rather than synthetic ones, and §12 produces such states as a by-product, since the observation tokenizer there is a residual-quantized codebook sitting in the position we argue it belongs.

9 Decentralized inference

9.1 The split, and what it does not buy

Running a 14T ensemble in one facility concentrates failure and thermal density. The reference deployment distributes layers across nodes owned by independent parties in the manner of Petals [9, 44], keeping tokenization, encoding, and primary routing inside the operator's boundary and dispatching the resulting latent vectors outward for downstream decoding.

Remark 5 (Latents are not confidential)**.** *This split is often justified as protecting proprietary integrity on the grounds that a peer sees only latents. That does not survive contact with the embedding-inversion literature: intermediate representations are routinely invertible to substantial fractions of their input text, and a peer seeing latents across many requests is in a better position than one seeing none. The split does have a real benefit, which is that the router, tokenizer, and codebook are what an adversary would need to* replicate *the system, and they stay inside. It has essentially no confidentiality benefit for user content. We adopt the split for the first reason and drop the second. Any deployment handling regulated data must treat the peer network as an untrusted processor with full visibility, which is a data-protection posture rather than a cryptographic one.*

9.2 Dispatch granularity, and why the Petals topology does not transfer

Two distribution topologies get conflated whenever "Petals-style" is used as shorthand, and they are not interchangeable. What separates them is dispatch granularity, and on a wide-area network it is worth about three orders of magnitude.

A **monolithic sparse mixture-of-experts** routes each token to 16 of 896 experts *per layer*. DeepSeek-V3's published configuration puts 58 mixture-of-experts layers in a 61-layer stack, and taking a count of that order for a 2.8T model gives roughly $16 \times 60 = 960$ expert dispatches per token. That layer count is an assumption rather than a figure, because Moonshot publishes none for K3, but the conclusion is insensitive to it within a factor of two. Which experts fire

is data-dependent, so peer load cannot be predicted from the request, and the resulting traffic pattern is all-to-all rather than pipeline-parallel.

Two distinct quantities follow and they must not be conflated. The 960 figure is a **bandwidth and load** count: that many expert invocations are placed per token. **Latency** is smaller, because dispatch inside a layer is parallel while layers are serial, so a peer round trip is paid once per layer and the serial depth is 60 round trips per token, not 960. At 10 ms per round trip that is 0.60 s per token, or 1.67 tokens per second; at 30 ms it is 1.80 s per token, or 0.56 tokens per second. Neither is an interactive serving rate.

The insensitivity above is worth stating with the sensitivity that sits next to it, because only one of the two is disclosed anywhere else and it is the less consequential one. The conclusion is highly sensitive to the round-trip time. Taking ten tokens per second as an interactive floor, sixty serial round trips per token clears it only when the round trip exceeds $100/60 \approx 1.7$ ms, and measured intra-datacentre round trips for exactly this workload are two orders below that: NVSHMEM reports 24.3 microseconds for mixture-of-experts all-to-all with IBGDA, 18.0 with a CPU proxy and 16.7 device-initiated [48], at which sixty serial round trips is about 1.46 ms per token, or roughly 690 tokens per second. **So the argument below is an argument about wide-area networks and not about sparse dispatch as such**, and a reader is entitled to see how narrow that scope is rather than infer it.

The like-for-like comparison is per response and in serial round trips on both sides. A tower ensemble routes a whole query to one tower: one dispatch, one round trip, then local decode, independent of response length. A peer-sharded monolith pays its 60 round trips for *every* token, so a 500-token response costs $60 \times 500 = 30{,}000$ serial round trips against one. Reading "960 against one" as a latency ratio conflates invocation count with serial depth; separating the two makes the comparison more conservative per token and considerably wider per response.

A **Mixture-of-Towers** routes a whole query to one tower. That is *one* dispatch per query. The round trip is then paid once at admission and decoding runs local to the tower's own pool *provided that pool is one low-latency locality*, and that proviso is a deployment assumption rather than a consequence of the architecture. It has to be stated as a premise, because §3 makes each tower internally a heterogeneous mixture-of-experts and §9 makes the reference deployment a peer network: if a tower's own experts are dispatched to across the same wide-area pool, the tower pays the monolith's sixty serial round trips too and the gap is zero. The comparison below therefore holds when each tower's serving pool is a single locality and the monolith's expert set is not. §11's case 3 is where the premise is least comfortable, because it puts 60 percent of traffic on a 192-accelerator pool that is itself 60 percent of the monolith's footprint.

Sixty serial round trips per token against one per query is the difference that decides serviceability, and nine hundred and sixty expert invocations per token against one is the difference that decides bandwidth.

Petals [9, 44] distributes *dense* transformer layers pipeline-parallel, which sits at the coarse end of this axis and is why its reported behaviour over the open Internet is encouraging. Adopting that topology for sparse expert dispatch without re-deriving its cost model is unjustified, and that is the standing architecture-review finding F-13 in `logos/ARCHITECTURE_REVIEW.md`, which is a review finding and not the measurement F13 of §16. The arithmetic above narrows it and does not close it. Nobody has measured sparse-dispatch cost on real hardware at this scale, and an arithmetic bound is not a measurement.

The components such a deployment would be assembled from are named here as *specified*, and nothing in this paper has been run on any of them. For the pooling role, a worker pool that

scales to zero across hardware nobody standardised, there is a published open-source runtime: Beta9 [8], AGPL-3.0, from Beam Cloud, whose distributed configuration provides scale-to-zero serverless workloads, agent-backed externally attached worker pools, and a Tailscale mesh for reaching them. It fills that role alongside FaaSMoE [23], which is the mechanism from the literature rather than a runtime and is evaluated at 2.7B. For the sharding role there is Petals. One gap belongs in the same breath as the components: a Petals-style deployment of a 2.8T mixture-of-experts additionally needs the model files converted into the shard format such a network serves, and nobody has performed that conversion for a model of this class. The sharding half of the stack is therefore not merely unmeasured, it is unbuilt.

9.3 Pinning the cache

Moving key-value state dominates distributed autoregressive inference. Every dialogue turn extends a state that must persist, and moving gigabytes per turn across a peer network is untenable, while expert computation benefits from spreading out. StateFlow [67] separates the two: pin key-value state at a sticky serving site for cross-turn reuse, and optimise expert dispatch and aggregation placement across the network.

Session start anchors the cache at an owner chosen by

$$o_s \;=\; \arg\min_{v \,\in\, \mathcal{P}(e_s)} \Big[\, \lambda_{\text{lat}}\, \mathrm{d}(e_s, v) \;+\; \lambda_{\text{util}}\, u(v) \;+\; \lambda_{\text{mem}}\, \frac{m_s}{M(v)} \,\Big], \tag{13}$$

minimising over peers $\mathcal{P}(e_s)$ reachable from entry point e_s a weighted sum of network distance, current utilisation, and session memory footprint relative to peer capacity. Intermediate vectors go out to remote experts and results return to o_s for aggregation, so state never migrates between sites. StateFlow reports more than twice the stable dialogue concurrency of distributed baselines and a 53.0 percent reduction in turn-level p95 latency [67].

Equation (13) is our reconstruction of the selection objective from the reported policy, including the weights. An implementation should prefer the published functional form to ours.

Pinning state has a failure consequence the source does not dwell on and we will: because state deliberately cannot move, losing the sticky owner loses the session. The minimum contract is an explicit session restart with the dialogue history replayed from the client, not a silent hang. §11.4 replaces that minimum with content-addressed snapshots under a named ref, which bounds the loss to a checkpoint interval, and states the constraint that construction has to respect.

9.4 Cache management and elastic experts

At the owner, PiKV [34] treats key-value allocation as an expert-sharded, query-driven process rather than a passive block pool, hashing allocations by time and expert index,

$$s(t, e) = (t \bmod N_{\text{tok}}) \oplus (e \bmod N_{\text{exp}}), \tag{14}$$

and combining compression with activity-based eviction of low-utility tokens. The exclusive-or of two independent moduli collides when N_{tok} and N_{exp} share factors. That is true and misleading in equal measure, because it invites an implementer to reach for coprime moduli. Brute-forcing every pair in $[2, 32]^2$ gives collisions in all 584 coprime pairs and all 377 shared-factor pairs: coprimality is not the fix. The minimal case is $N_{\text{tok}} = 2$, $N_{\text{exp}} = 3$, where two of

the four reachable slots each carry two pairs. Nor is the collision forced by slot count, since a disjoint-bitfield construction over the same codomain,

$$s(t, e) = (t \bmod N_{\text{tok}}) + \big(e \bmod N_{\text{exp}}\big) \cdot 2^{\lceil \log_2 N_{\text{tok}} \rceil}, \tag{15}$$

is injective. The right statement is that exclusive-or is the wrong mixing operator here regardless of the moduli, and the original instinct, to check the construction against the paper before implementing, stands.

Remote layers run under FaaSMoE [23], which deploys experts as stateless serverless functions with on-demand, scale-to-zero invocation across tenants and configurable expert granularity per function, trading elasticity against invocation overhead. The prototype uses under a third of the resources of a full-model baseline. It is evaluated on a 2.7-billion-parameter model, roughly a thousand times smaller than a tower, and cold start is the obvious failure mode at interactive latency. That is measurement F7.

9.5 Routing across models, not only inside them

Above the towers sits a Mixture-of-Models problem: how much computation to spend, which models participate, how outputs combine. Static routing on token counts and coarse domain labels ignores within-query difficulty and cross-model complementarity.

A deployed instance is Echo [74], which coordinates open-weight models behind one endpoint and adapts computation per request. The general finding in this line [10, 24] is that complementarity is real: a weaker model can be the right choice on specific sub-problems, so a good router can beat the best single model in its pool. Vendor-reported results place such a system at frontier-comparable quality for about a third of the expected inference cost [74]. That figure is Tier B and self-reported, we found no independent evaluation, and cost-per-quality figures are exactly where the choice of baseline decides the answer.

10 Peer integrity

Distributing activations across independently owned peers admits tampering. Recomputation on trusted hardware defeats the purpose of distributing, and cryptographic commitment over every activation costs too much latency.

The published approach [17] is a threshold-free statistical test using known-answer canaries. The requester picks secret inputs whose correct activations are known in advance and mixes them into ordinary traffic. Peers cannot tell a canary from a live query, so systematic tampering corrupts canaries along with real data.

The difficulty is numerical non-determinism: heterogeneous hardware and non-associative floating-point reduction mean honest peers drift too. The test compares *drift distributions* rather than demanding exact matches. Hardware noise sets a null, and tampering separates from it. Across 408 configurations the detector reaches an area under the ROC curve of 1.0, ranking the malicious shard above every benign shard on every canary [17].

10.1 Why that number will not transfer

Perfect separation across every configuration is a strong result and, read carefully, a narrow one. Three properties of the evaluated setting do not hold here.

(C1) **The adversary is static.** A shard that tampers on every pass corrupts every canary and is trivially separable. One that tampers with probability $p \ll 1$ corrupts a given canary with probability p, so with m canaries the detection probability is $1-(1-p)^m$ and time to detection grows as $1/p$. An adversary who only needs to corrupt a small share of outputs, which is the realistic threat for biasing a code-generation result, can sit below any interactive-latency observation budget.

(C2) **The numerics are homogeneous.** The benign null is set by hardware drift. A network spanning different accelerators, mixed MXFP4 and NVFP4 paths from §7, and independently compiled kernels has a null that is wider, multi-modal, and node-dependent, which is when a single pooled null swallows the tampering signal. This architecture creates that heterogeneity in §7 and then leans on a detector that assumes it away.

(C3) **Tampering is assumed to be in value space.** An adversary running a legitimately quantized variant of the assigned expert produces drift that is in-distribution for the benign null while changing the output.

Position 2 (Detection degrades with adversary duty cycle)**.** *Under a fixed canary budget m and an adversary tampering with probability p, detector AUROC falls monotonically in $1/p$, and there exists $p^\star$ below which it is statistically indistinguishable from chance at any m compatible with interactive-latency overhead. Widening the benign null to cover heterogeneous numerics raises $p^\star$.*

Testing this needs a benign null *measured* across two accelerators of different models with independently compiled kernels, since a matched pair merely reproduces the published setting. It is measurement F8, and it is the one we expect to fire against our own architecture.

10.2 The detector's own verdict rule, run

Position 2 is about an adversary who is trying. A weaker question comes before it, and until now it had not been asked here at all: does this work's own implementation of the detector separate tampering from a benign null, and *where does that separation stop*? The two halves are one question. A detector that fires only on gross corruption has shown nothing, since a shard that scales its outputs by 1.5 or flips their signs is separable by inspection; and a detector that does not publish where it stops is worse than a weak one, because a boundary that is not printed is a boundary the operator assumes lies somewhere else. The instrument is `logos/experiments/canary_trap.py`, and its verdict rule was frozen before the run: supported if and only if the area under the ROC curve is at least 0.99 at every tamper severity at or above 3σ of the benign null, with a collapse below 0.9 nearer the floor admissible *only* when reported as the measured detection boundary.

Nodes are ranked by mean relative-L^2 drift over their canaries, a fixed metric rather than a learned distance, since a learned distance would drift with the thing it is measuring. Twenty-four benign nodes draw from three parametric noise models — non-associative reduction order, bf16-against-fp16 rounding, and varying reduction-tile size — and eight malicious nodes are run per tamper kind, with eight canaries each in dimension 512. Severities are expressed in units of the null's standard deviation, so one grid means the same thing at each of the four null magnitudes swept, from 0.5 to 4 times a base of 10^{-3} in relative-L^2 units. AUROC by tamper kind and severity, identical at all four magnitudes:

tamper kind	32σ	8σ	3σ	1σ	0.5σ	boundary
bias	1.0000	1.0000	1.0000	0.6667	0.0000	3σ
scale	1.0000	1.0000	1.0000	1.0000	0.0000	1σ
sign	1.0000	1.0000	1.0000	1.0000	1.0000	0.5σ
top-k	1.0000	1.0000	1.0000	1.0000	1.0000	0.5σ
subtle	1.0000	1.0000	1.0000	0.7500	0.0000	3σ

The rule is satisfied, and the question the module was written to answer is settled: AUROC is 1.0000 at every severity at or above 3σ, for all five tamper kinds, at all four null magnitudes. **The boundary column is the half worth keeping.** Sign flips and top-k reordering are caught at 0.5σ, well under the noise floor, because they change the direction of an answer rather than its magnitude while the null is a magnitude perturbation; rescaling is caught at 1σ. The two kinds that correspond to a careful adversary — an additive bias, and the deliberately subtle arm the module exists for — stop at 3σ, and they stop abruptly rather than gracefully: AUROC 1.0000 at 3σ, 0.6667 and 0.7500 at 1σ, and exactly 0.0000 at 0.5σ. A zero is not a failure to detect. It is a perfectly inverted ranking, every tampered node scored below every benign one, which is what happens when the tamper is smaller than the noise it hides in and the honest nodes' drift dominates the statistic. **Below 3σ of the benign null this detector is not weak but anti-informative**, and an operator reading a low drift score in that regime is reading the tampering rather than its absence. Reporting the boundary per tamper kind rather than pooling the five into one number is what makes that visible; a mean over the five kinds would have printed a comfortable figure and hidden the inversion inside it.

What this discharges is the module's stated rule and nothing past it, and the distance to F8 is not a formality. The null here is *parametric*: a noise model of hardware non-determinism, not drift recorded across accelerators of different models with independently compiled kernels, which is the condition (C2) says will widen it. The adversary tampers on every canary it is given, a duty cycle of $p = 1$, which is precisely the static adversary (C1) identifies as trivially separable. And the sweep is one seed at four magnitudes, which is a robustness check on the noise model rather than a replication. **F8 is untouched, its prediction stands, and the table above should be read as the floor a detector has to clear before F8 is worth running against it** — not as a reading of the adversary F8 describes. The second tier of the same module, which grounds the null in measured reduction-order drift on real decoder activations, is built and has not been run.

11 Cost, evaluation, and what breaks

The architecture has argued affordability throughout without pricing anything, and has offered no way to tell a well-routed ensemble from a collection of models behind a lookup table. This section supplies both.

11.1 What it costs to serve

Start from resident memory, since Proposition 1 makes it the constraint sparsity does not move. A 14T ensemble at MXFP4's true 4.25 bits per parameter is 7.4 TB of weights. On 80 GB accelerators with roughly 70 GB usable for weights, the floor is about **106 accelerators**, before

any key-value cache or activation memory, and before the higher-precision embedding and output layers that the floor also ignores.

Real deployments run above that floor, but how far above is not documented and we should not pretend otherwise. A figure of sixty-four or more accelerators for Kimi K3 circulates widely. We could not trace it to a primary document, and the vLLM preview this paper cites for K3 serving does not contain it: it gives a sixteen-accelerator data-parallel and expert-parallel configuration for the MXFP4 mixture-of-experts path and defers hardware requirements to the open-weights release [76]. Taking sixty-four at face value gives 23.2 GB of weights per accelerator, which is 33 percent of a usable 70 GB, 19 percent of a usable 125 GB, and 14 percent of a usable 170 GB. Those figures use K3's 1.4875 TB at 4.25 bits; dividing a flat-four-bit 1.4 TB instead gives 21.9 GB at 31, 18 and 13 percent, and that substitution is the density error §7 names. The source does not say which part it means, and the reading inverts with the answer: on the largest parts, holding weights to that ratio would mean leaving 86 percent of high-bandwidth memory for cache and activations, which nobody does. Extrapolating the ratio to 14T gives 320 accelerators, or forty eight-accelerator nodes, but that extrapolation assumes the sixty-four is a memory ratio at all, and node granularity is at least as parsimonious an explanation, since $64 = 8 \times 8$ and $320 = 40 \times 8$.

So the statement has to be a bracket: between about 106 accelerators, the memory floor above, and about 320, the ratio-preserving extrapolation. We do not know where in it a real deployment lands, and the gap is key-value cache, activations, expert-parallel communication headroom, and throughput in unknown proportion. Settling it needs a measured serving deployment on named hardware, which is a measurement and not a calculation. The rest of this section uses 320 because it is the conservative end.

Now the comparison the architecture needs and never made. Only one tower activates per query, so a query touches only that tower's pool. Five towers of 64 accelerators is 320 accelerators. **Five replicas of a single 2.8T *generalist* of the same per-instance size is also 320 accelerators.** The word is deliberate: a replica fleet built from a domain tower would serve one domain and would not be a competitor to an ensemble at all, so the comparator has to be a model that any query can be admitted to. The hardware bill is the same. The equality is real, and it does not justify the ensemble on traffic entropy alone, because that inference treats the two fleets as interchangeable stock when they differ in the property that decides the answer: **replicas are fungible and towers are not**. Any replica can serve any query, so an idle replica is capacity available to every domain. A Code request cannot consume an idle Life-Sciences accelerator, because the weights resident on it are the wrong weights, and moving 1.49 TB to fix that is not a request-time operation. So the comparison has three cases, not one, and only the third is an argument for the architecture.

Case 1, uniform traffic and uniform sizing. Five domains at 0.20 of traffic each, five pools of 64. Each pool's share of offered load equals its share of capacity, so both fleets saturate at the same aggregate rate and utilisation is uniform in both. Same cost, same throughput, no serving-cost advantage either way. The ensemble still buys domain coverage, which is a quality argument and is priced nowhere in this section.

Case 2, skewed traffic and uniform sizing. The ensemble is worse. Take shares $(0.60, 0.20, 0.10, 0.05, 0.05)$, a plausible mix for an operator whose users mostly write code. Write C for the aggregate service capacity of the 320 accelerators, so a 64-accelerator pool carries $C/5$. The replica fleet admits any query to any pool and saturates at offered load $L = C$. The

ensemble saturates when its busiest pool does, at $p_{\max} L = C/5$, that is

$$L_{\max} \;=\; \frac{C}{5\,p_{\max}} \;=\; \frac{C}{5 \times 0.60} \;=\; \frac{C}{3},$$

one third of the replica fleet's throughput on identical hardware. Mean utilisation at that point is $\sum_i p_i/(5p_{\max}) = 1/3$, so two thirds of a 320-accelerator fleet is idle while the Code pool queues. This is the loss of statistical multiplexing, it is the standard argument against partitioned capacity, and it applies here in full. **The shares above are stated over the five-way partition of Table 1 and not over the four-way partition §3.5's criterion returns.** On four towers with Mathematics and Logic merged, shares $(0.60, 0.20, 0.15, 0.05)$, the same skew gives mean utilisation $1/(4 \times 0.60) = 0.417$, that is 58 percent idle and a throughput penalty of 2.4 rather than 3. Both arithmetics are correct for their own partition and neither skew is measured; they must not be interleaved.

Case 3, skewed traffic and asymmetric sizing. The ensemble buys per-domain parameter allocation, and it does not recover case 2's throughput. A replica fleet is n copies of one model, so every domain gets the same parameter count whether it carries 60 percent of traffic or 5. An ensemble has no such constraint: at a fixed 14T budget, tower sizes can be set proportional to demand. At the same shares that gives towers of 8.4, 2.8, 1.4, 0.7 and 0.7 trillion parameters. At MXFP4's true 4.25 bits the weights are 4.46, 1.49, 0.74, 0.37 and 0.37 TB, totalling 7.44 TB, which is the same total and therefore the same 106-accelerator memory floor as the uniform arrangement. Allocating the 320 accelerators in the same proportion gives pools of $192, 64, 32, 16, 16$, and because both the memory floor and the allocation are proportional to N_i, every pool sits at the same $320/106 = 3.02$ times its own floor. **That equality does not make capacity share equal load share, and it does not recover the multiplexing loss of case 2. The reason is arithmetic rather than judgement.** Memory-floor proportionality is a statement about bytes resident; capacity share is a statement about tokens per second, and the step between them does not hold under this paper's own conventions. Proposition 1 fixes the sparsity $s = N_{\text{act}}/N_{\text{total}}$ *per tower*, so tower i activates sN_i parameters, and by Eq. (3) its pool of $A_i = 320\,p_i$ accelerators at F FLOP/s serves

$$R_i \;=\; \frac{A_i F}{2sN_i} \;=\; \frac{320\,p_i\,F}{2s \times 1.4 \times 10^{13} \times p_i} \;=\; \frac{320F}{2s \times 1.4 \times 10^{13}},$$

which is *independent of* i. Every pool serves at the same rate whatever its share, the busiest pool still carries 0.60 of the load, and the fleet still saturates at $L_{\max} = C/3$: exactly case 2's answer, with mean utilisation still 1/3. The same cancellation survives the memory-bandwidth-bound reading of §5.2, since bytes moved per token in a sparse forward pass is also proportional to activated parameters, and it survives spending the headroom above the floor on replicas of tower i, each serving at a rate proportional to $1/(sN_i)$.

Case 3 would recover the throughput under the opposite convention, activated parameters held *constant* across towers so that the 8.4T tower is three times sparser. That is a legitimate mixture-of-experts design, more experts at unchanged top-k, but it is not the convention Proposition 1 states, it is not what §2.2's bracket is swept under, and adopting it would change what the case says: "a tower three times the size" would then buy resident knowledge at unchanged compute per token rather than more compute per query. We keep the fixed-s convention and take the weaker conclusion. **What case 3 does buy is per-domain parameter allocation at throughput parity with case 2**: the 60 percent of traffic that is code is served by

a tower three times the size of anything the replica fleet can offer it, and that effect a fixed-size replica fleet structurally cannot reproduce, because to give code queries an 8.4T model it would have to train one and then replicate *it*, at three times the memory bill on every domain including the ones carrying 5 percent. Whether resident parameters per domain is worth a factor of three in throughput is a quality question this section does not price, and the companion `logos/LADDER_ARCHITECTURE.md` §5.3 reaches parity rather than a strict win on its own four-domain instantiation, which is the same answer arrived at independently.

Four caveats bound case 3. The proportionality is exact only because both the memory floor and the allocation scale linearly in N_i; introduce a per-pool fixed overhead and the small towers stop fitting. The throughput conclusion above is conditional on the fixed-s convention and inverts under constant activated parameters, which is the sensitivity to state. Nothing here says a 0.7T Administration tower is good enough at its job, and §3.5's criterion does not speak to size. And the traffic mix has to be stable, since re-sizing a tower is a training run and not a scheduling decision, which is the sense in which this ensemble is less elastic than the replica fleet even where it is cheaper.

We spend this much space on the arithmetic because it is checkable and almost nothing else supporting the architecture now is. §2.2 removed the compute motivation, §2.3 removed the token-fragmentation motivation, and §11.5 leaves the data-wall motivation alive only when the residency-bound fraction of each tower's corpus is below about 0.02, that is only when the shared core is essentially the whole corpus. A cost argument can be checked against an invoice, and being checkable is a property that cuts both ways: the throughput half of case 3 was added and removed inside a single revision of this paper, on arithmetic anyone can redo, which is the behaviour a checkable argument is supposed to have.

One workload is priced here and one is not. §12's loop is not the serving workload: it shows each observation to $k \geq 2$ towers and admits only the fraction q on which they disagree. All towers are resident in the 320 either way, so the hardware bill does not change and the earlier statement that a two-tower step doubles the footprint would be wrong. What does change is the cost per *admitted* trajectory, which scales as k/q. Neither symbol appears anywhere in this cost model, and §12 notes that q falls by design as the environment is explored, so cost per admitted trajectory rises as the loop succeeds. That is unpriced, and §4's ε-exploration slice has the same problem: it is charged in quality and never in accelerators.

That yields a decision rule, and it is not the rule this section used to give, nor the rule the previous revision replaced it with. Writing $H(\text{domain})$ for the entropy of the domain distribution over incoming traffic, the oldest rule was that the ensemble's advantage grows with H and vanishes as $H \to 0$. That rule is case 1 against case 2 and it holds only under *uniform* tower sizing: an operator whose traffic is eighty percent code and whose towers are all 2.8T should serve code replicas rather than an ensemble. The revision that replaced it said the rule *inverts* under demand-proportional sizing, on the strength of case 3's since-removed throughput recovery. What survives the correction is weaker and is stated as such. Demand-proportional sizing does not buy throughput back; it buys *resident parameters where the traffic is*, at the same saturation point case 2 reaches, and a fungible replica fleet has no mechanism for doing so at any H. At maximal H there is nothing to allocate and case 3 collapses into case 1. Reduced to one sentence, *on throughput alone the ensemble never beats a fungible fleet of equal cost, and its serving-side case is that a skewed, stable, pre-known traffic mix lets it spend the same hardware on more parameters per unit of demand.* That is a quality argument wearing a cost argument's clothes, and this section does not price the quality. It still gives Position 1 something to bite on: the

cost of unfiltered frontier deployment is dominated by resident memory per served domain, not by activated compute.

The correction matters beyond this subsection, because the paper's other motivations have been eroded rather than reinforced. §9.2 supplies a second justification independent of all of them, and it is the sturdier of the two because it does not depend on the traffic mix at all. **A tower ensemble is serviceable over a wide-area heterogeneous pool where a monolithic sparse model of equal total size is not**, because the ensemble pays one peer round trip per query and the monolith pays one per layer. Three things that statement is not. It does not say towers produce better answers, since it is a statement about deployability under a given topology and says nothing about quality. It is not unconditional: it holds under §9.2's stated premise that each tower's serving pool is one low-latency locality, which case 3's 192-accelerator Code pool strains. And it is not measured: §9.2 is arithmetic over an unverified layer count, its crossover sits at a round trip of roughly 1.7 ms so it is an argument about wide-area networks rather than about sparse dispatch as such, and the sparse-dispatch cost model it argues against has never been measured on real hardware either.

For price anchoring, reported blended serving costs per million tokens in mid-2026 are $2.31 for Kimi K3, $0.90 for GLM-5.2, and $0.18 for DeepSeek V4 Pro [39]. All Tier B, and the spread across three trillion-scale sparse models is more than an order of magnitude, which says the dominant term is serving strategy rather than parameter count.

11.2 What a tower costs in parameters, and what routing buys for it

The subsection above prices accelerators. This one prices parameters, and it comes out against the architecture as specified at small N. The numbers are measured on the upcycled rig of §4.5 rather than derived, and the limits on how far they travel are stated at the end rather than the beginning, because the shape of the result does not depend on them.

Not every parameter can be replicated per tower. Embeddings, the shared trunk and the shared head are common by construction, so upcycling multiplies the total parameter count by $1+\tau(N-1)$ rather than by N, where τ is the **expert-able fraction**: the share of the parameter count a tower actually owns. On Qwen3-0.6B, $\tau = 0.3695$. Embeddings alone are 26 percent of that model and the trunk and head take most of the remainder. At $N = 4$ the result is a $2.11\times$ model rather than a $4\times$ one, which is exactly the 1.257×10^9 total against 5.96×10^8 active that §4.5 reports.

The comparison that decides whether that is worth doing is not against the parent but against a dense model of the same *total* parameter count, since that is the model an operator could have held in the same memory. Priced in the effective parameter count that the routed-model scaling literature fits [18], the ratio of effective-parameter gain to total parameters falls monotonically in N: 0.853 at $N = 2$, 0.221 at $N = 32$. **There is no N at which per-segment top-1 routing breaks even against a dense model of the same total parameters.** What sparsity buys here is activated compute, which Proposition 1 has already said is not the binding constraint, at a price in resident memory, which Proposition 1 says is.

One quantity explains why per-segment routing sits on the wrong side of that comparison, and it is checkable in a line. A routing decision carries at most $\log_2 \binom{N}{k}$ bits, and what that is worth per token depends entirely on how often it is taken. Mixtral routes top-2 of 8 at each of 32 layers, which is $32 \log_2 \binom{8}{2} = 153.8$ bits per token [52]. The rig here routes top-1 of 4 once per 1024-token segment, which is $2/1024 = 0.00195$ bits per token. **Per-segment tower routing carries roughly five orders of magnitude less routing information than per-**

layer expert routing, and §15.5 has measured what it buys in exchange: a reduction in bytes crossing a device boundary of six-sevenths of the routed depth. That is a real trade rather than a defect. The objection is that this paper has been counting the memory-traffic side of it for several sections without ever pricing the information side.

The run these figures come from also sat at $D/N = 0.0186$ tokens per parameter against Chinchilla's twenty [28], three orders of magnitude short. Everything measured on it is measured far from convergence, which is §15.4's regime error again rather than a second one: mixture specialisation is a response to capacity pressure, and at that token ratio there was none to respond to.

Two limits bound how far τ travels, and they cut in opposite directions. The measurement is on a 6×10^8-parameter model whose embedding table is 26 percent of it; at 2.8×10^{12} the embedding share is negligible, τ rises toward the feed-forward share, and the "N towers is N times the parameters" intuition is less wrong at tower scale than it is here. But the break-even statement is against total parameters, and nothing about scale changes the fact that a routed model of $1 + \tau(N - 1)$ times the parameters is being compared with a dense model of the same total. **This subsection is negative for the architecture at the scale it was measured at, and nothing measurable from here says it stops being negative at the scale the architecture proposes.**

11.3 Measuring whether the router earns its place

The naive metric for an ensemble, the best tower's score, is exactly what a good router beats and a bad one does not, so it cannot distinguish them. Routing evaluation already has the right primitive: an *oracle ceiling*, the score of a policy that always sends each query to the best model for it, computed from a fully observed reward matrix on an evaluation set [7, 60]. Serving every evaluation query on every tower costs k times inference, which is affordable on an evaluation set and not in production.

With a ceiling and a floor, the tempting quantity is where the router lands between them:

$$\eta \;=\; \frac{S_{\text{router}} - S_{\text{best single tower}}}{S_{\text{oracle}} - S_{\text{best single tower}}}. \tag{16}$$

At $\eta = 0$ the router adds nothing over picking one tower and keeping it. At $\eta = 1$ it is said to route perfectly. η is not the primary metric here, and F10's criterion is not stated in terms of it, for four reasons that are properties of the definition rather than of any deployment.

(E1) **The denominator can be zero, on a non-negligible event.** If the best single tower already solves every item any tower solves, η is undefined. Three towers at accuracy 0.90, 0.20, 0.20 on a 50-item per-domain slice put that probability at 0.161 over 200,000 Monte Carlo trials, and the mandate below to report per domain shrinks the slice and raises it further. The two-axis partition of §3.5 shrinks it again.

(E2) **η is unbounded below.** A router worse than the fixed best tower scores negative with no floor and no stated scale on which to read the magnitude.

(E3) **The floor is biased high, and the bias grows with tower count.** $S_{\text{best single tower}}$ is a post-hoc maximum over k noisy scores measured on the same evaluation set, so it carries a winner's curse. With towers of genuinely equal skill at 0.700 on 500 items, the expected maximum computed exactly from the order statistics of the binomial exceeds the truth

by 1.16 points at $k = 2$, 2.10 at $k = 4$, 2.90 at $k = 8$ and 3.31 at $k = 12$. That biases η downward for exactly the wide ensembles it exists to judge, and §3.5's domain-crossed-with-size partition is what pushes k toward the high end of that list.

(E4) **The ceiling is inflated by chance, and even without chance it is unreachable.** An oracle that counts an item solvable if *any* tower is right counts lucky guesses. On four-choice multiple choice with towers of literally zero skill, $S_{\text{best single tower}}$ is about 0.250 while $S_{\text{oracle}} = 1 - 0.75^k$ reaches 0.763 at $k = 5$, manufacturing a denominator of 0.513 out of independent guessing. The benign case is no better: five genuinely skilled independent towers at $p = 0.70$ give $S_{\text{oracle}} = 1 - 0.3^5 = 0.99757$, so a router at $\eta = 1$ would have to identify, per item, which tower happens to be right. $\eta = 1$ **does not mean "routes perfectly". It means "is an oracle"**, and a metric whose upper anchor is unattainable by any router at any budget is not a scale anyone can calibrate against.

What to report instead, and it is not ours

The quantity an operator actually chooses between configurations on is a cost-quality curve rather than a normalised gap, and the field already has named, benchmarked formulations of it. **We adopt them. Nothing in this subsection is a contribution, and the correct description of what follows is that the paper stops using a metric it invented and starts using the standard ones.** Proposing a cost-quality metric here would repeat, inside the repair, exactly the missing-related-work defect §12 was corrected for.

The primary metric is **AIQ**, average improvement in quality, from RouterBench [49]:

$$\text{AIQ}(R_\theta) \;=\; \frac{1}{c_{\max} - c_{\min}} \int_{c_{\min}}^{c_{\max}} \tilde{R}_\theta \, \mathrm{d}c, \tag{17}$$

the area under a router's cost-quality curve normalised by the span of the cost domain. RouterBench supplies the supporting constructions with it: a non-decreasing convex hull over operating points, which is the Pareto frontier object under its published name, a Zero Router reference built from the individual models' collective hull, and an Oracle Router [49]. The secondary pair is from RouteLLM [53], which defines the performance gap recovered between a designated weak model M_w and a designated strong model M_s,

$$\text{PGR}(M_R^\alpha) = \frac{r(M_R^\alpha) - r(M_w)}{r(M_s) - r(M_w)}, \qquad \text{APGR} = \int_0^1 \text{PGR}(M_R^\alpha) \, \mathrm{d}\, c(M_R^\alpha), \tag{18}$$

together with $\text{CPT}(x\%)$, the minimum percentage of calls to the strong model needed to reach $\text{PGR} = x\%$. Both directions of the trade were reported before either metric existed, by FrugalGPT, as quality at fixed cost and cost at fixed quality [13]. **We report AIQ as the primary scalar and APGR with** $\text{CPT}(50\%)$ **as the secondary pair.**

The reason for the swap is specific, and it is the four defects above rather than a general preference.

(E1) AIQ avoids the zero denominator **outright**: it contains no ratio and no oracle, so a dominating single tower is simply a flat curve with a well-defined area. APGR does *not* avoid it structurally, and we say so rather than present the pair as clean: $r(M_s) - r(M_w)$ is zero if the designated endpoints score alike. What changes is that the endpoints are designated *a priori* rather than selected post hoc, so the condition is diagnosable before evaluation instead of emerging from per-domain slicing. Weakened, not eliminated.

(E2) AIQ inherits the quality scale, so with quality normalised to $[0, 1]$ its value lies in $[0, 1]$ and reads as average quality across the cost range. RouterBench does not state that bound as a theorem, and it follows from normalised quality rather than from the paper (`logos/PRIOR_ART_v03.md` §7, item 11). APGR is stated as bounded in $[0, 1]$, though a single-α PGR can go negative.

(E3) Neither contains a maximum over k noisy scores. AIQ's hull is a reference *curve* over the individual models rather than a maximum entering a denominator, so selection bias in the hull moves the comparison line and not the metric value. APGR takes $k = 2$ endpoints fixed in advance.

(E4) This is the one caveat, and it needs stating precisely. **RouterBench's Oracle Router is itself a per-item maximum over k and carries the chance inflation in full**, manufacturing $1 - 0.75^k$ of apparent headroom on four-choice items with zero-skill towers, which is the 0.763 at $k = 5$ computed above. The difference that matters is that the oracle is a reported reference *line* and not a term inside Eq. (17), so AIQ's value is unaffected by it. Report the oracle line only chance-corrected, and say in the caption that it has been corrected. APGR uses no oracle and avoids the defect entirely.

So η is retired rather than kept as a diagnostic. The one thing it was for, the fraction of available headroom a router recovered, is what APGR measures, with endpoints designated in advance instead of maximised over, which is the same idea done correctly by people who published it two years earlier. There is no residual question η answers that APGR does not answer better.

One property matters more than the rest for this paper. AIQ **does not fragment under the two-dimensional partition of §3.5**. Cost and quality are both per-query quantities, so a domain slice or a size slice is read on the same axes, and slices aggregate by summing cost and averaging quality under the traffic weights, which is the operator's own objective. η is a ratio of differences and does not aggregate at all: a weighted mean of per-domain η is not the η of the pooled set, and with domains crossed by size tiers there is no level at which the ratio is both defined and decision-relevant.

Measurement F10's criterion is restated on AIQ, since as written on η it was unmeasurable whenever η was negative or undefined. A warm-started router fails F10 if, on any domain carrying at least five percent of evaluation traffic, its post-swap AIQ falls more than 0.02 below its pre-swap AIQ, or its CPT(50%) rises by more than ten percentage points of calls. Three conditions make that computable. They are not optional: the two metrics are *not* both defined on every evaluation set, the second failing on this subsection's own showing at (E1).

(i) **A common cost domain.** Eq. (17) normalises by the span $c_{\max} - c_{\min}$, and a tower swap changes the pool's price range, so pre- and post-swap AIQ are otherwise averages over different intervals and a router whose behaviour did not change can move. Both sides are integrated over the *union* of the pre- and post-swap operating ranges, fixed before the swap.

(ii) **A defined denominator.** CPT(50%) is defined through PGR and PGR is undefined when $r(M_s) = r(M_w)$. The limb is evaluated only on slices where the designated endpoints differ in score, which (E1) already says must be checked in advance; where they do not, F10 is decided on AIQ alone and the exclusion is reported.

(iii) **A slice size and a multiplicity rule.** 0.02 is below the noise floor of a small slice: at quality near 0.7 on a 50-item slice the standard error of one quality estimate is $\sqrt{0.7 \times 0.3/50} = 0.065$, more than three times the threshold, and with §3.5's two-dimensional partition there are enough slices that an unchanged router would fail somewhere with high probability. F10 requires a minimum of 5,000 evaluation items per reported slice, a paired test across the swap at a one-sided $\alpha = 0.05$, and a Bonferroni correction across the reported slices; or, alternatively, the 0.02 is replaced by a threshold set from a measured noise floor on the same slices. Without one of those two, F10 is a research task rather than a measurement.

Per §4, AIQ must be reported per domain. A good pooled AIQ hiding a collapsed one on the unverifiable domains is the expected failure, not a surprise, and it is the failure the verifiability asymmetry of §4 predicts. The design question this puts to measurement is the one that matters: if the router's curve never separates from the best single tower's point, the ensemble is a collection of models with a lookup table, and the move is to ship the best tower.

The capability gate is scored over trajectories, and the split is half the measurement

AIQ and APGR price a router against a pool of models. Neither says whether the model got better at the thing its corpus was collected for, and the runs of §15.6 needed a second gate for that. Four properties of the one used there are worth recording, because three of them are where this kind of evaluation usually fails.

The split is whole-episode. Held-out items are trajectories, split by a hash of the trajectory path, never by chunk. A chunk-level split of an agentic corpus puts segments of one episode on both sides of the line, and because an episode repeats its system prompt, its tool schemas and frequently its file contents at every step, a chunk-level split shares documents between train and held and reports memorisation as generalisation. This is the discipline the project-disjoint null of §15.1 enforces on a different measurement, adopted here for the same reason.

The metrics are grounded in what the recorded trajectory did next, not in a preference or a rating. Four are reported. `gold_novel`: the model calls the tool the recorded trajectory called next, scored on the subset where that differs from the model's own first call. `gold_match`: the same without that restriction. `emits_call`: whether a call was emitted at all. `valid_name`: whether the name emitted exists. Every one of them is decided against a fact about the recording.

The `gold` family measures imitation, not correctness, and the difference is not pedantic. `emits_call` and `valid_name` are both 1.00 for the base model, so every call it makes is already *plausible*: well-formed, and naming a tool that exists. What `gold` adds is whether that plausible call is the one the expert made. But a recorded trajectory is one path through a task rather than the only correct one, so "not gold" conflates *wrong* with *different and perfectly fine*, and a base model at 4.3 percent agreement (§15.6) is not thereby 95.7 percent wrong. That is a property of the instrument rather than a defect in it, so long as the instrument is not asked the question it cannot answer.

A second environment computes its ground truth by construction. A generated self-grading environment produces tasks whose correct answer is known before the model is run, which removes the recorded trajectory itself as a possible source of error and gives the gate one arm that no corpus artifact can reach. The two arms are complementary rather than redundant, and the division of labour between them should be stated in the terms of the paragraph above. `gold_novel` measures **imitation**: cheap, deterministic, no environment needed, scored against a

recording. The generated environment measures **outcome** against a solver whose answer exists before the model runs, so a novel-but-correct path scores as correct there and as a miss on `gold`. **Only the second can tell right from merely different**, and any statement of the form "the model is wrong x percent of the time" has to come from it.

There is no judge model anywhere in this. No language model scores another language model's output at any point in the gate. A judge is an unpriced model dependency and a channel through which one distribution grades itself, and §12's entire argument is that what makes a correction worth anything is that it came from outside.

One of these interacts with §3.6's rule, and the interaction is stated here rather than left for a reader to get wrong. `gold_novel` is a label, and a human-provided one in the sense that somebody's recorded trajectory supplied it. It is a label on the *model's output*, not on the *router's decision*, and nothing in the gate asks which tower ought to have served a segment. §3.6 forbids scoring the router against a name. It does not forbid scoring the model against the world, and §12 is a long argument that scoring the model against the world is the only thing that ultimately works.

The gate discards 13.5 times its own data, and the fix is named rather than deferred

This is a design point about the instrument, not a complaint about it, and it is recorded here because the correction §15.6 has to apply by hand is a symptom of it. An agentic trajectory contains many assistant-to-tool-result transitions and the gate scores exactly one of them: the first. Measured over 200 held trajectories, the count of assistant→tool-result pairs has mean 13.5, median 14 and maximum 25; exactly one trajectory of the 200 contained a single pair, and 156 contained ten or more. **The gate takes 200 decisions where** 2,692 **are available.**

Continuing the replay in sequence rather than stopping at the first transition buys three things, and the second is the one that repairs a result rather than sharpening it.

(i) **Sample.** n_{novel} rises from about 20 to about 270, which moves the granularity of a reported rate from four or five percentage points to about one. Every figure in §15.6 is currently quoted at a resolution the sample does not support.

(ii) **A denominator the model cannot move.** Replaying each recorded transition independently against the *recording's* own prefix makes every trajectory contribute a fixed number of decisions whatever the model emits. That removes the selection effect of §15.6 at the source, rather than working around it with a second fixed-denominator statistic.

(iii) **Depth.** Accuracy at transition 1 against accuracy at transition 10 is the actual multi-step question this corpus was collected to ask, and it is currently unmeasurable, because only transition 1 is ever scored.

One design constraint governs the fix and it is the easy thing to get wrong. **The recording's call must be fed forward, not the model's.** Otherwise the model receives a tool result answering a question it did not ask, and every transition after the first is scored on a prefix that never existed. Two costs go with that and neither should be hidden. The probe costs about 6.75× what it costs now. And teacher-forced replay measures whether a model can *continue* an expert's trajectory rather than whether it can *produce* one — exposure bias in its standard form — which is precisely the gap the free-running generated environment exists to cover, and one more reason the two arms of this gate are not substitutes for each other.

11.4 Failure semantics, collected

Three failure paths, one contract each.

Tower unreachable. The three-step ladder of §4, with a degradation flag on the response at step two. No silent substitution.

Sticky owner lost. Session restart with client-side history replay, surfaced as an explicit error. State cannot migrate by design, so pretending otherwise produces a hang. This is the weakest of the three contracts and the next paragraphs replace it with a better one.

Straggler in expert dispatch. All-to-all dispatch sets p99 by the slowest peer. The mitigation is a deadline per dispatch round with a degraded top-k fallback, which lowers quality by a known amount instead of raising latency by an unknown one.

The general principle: at every failure the caller learns that quality dropped. The architecture's own §1 example of demotion without notification is the pattern to avoid.

Losing the sticky owner should not cost the session

Architecture-review finding F-10 (`logos/ARCHITECTURE_REVIEW.md`, not the measurement F10 of §16) states the gap precisely: there is no failure semantics for the sticky key-value owner going away mid-session, because StateFlow pins state so that it never moves, and pinning state means that losing the owner loses the session. Restart-and-replay is a contract for that loss, not an answer to it. Here is an answer.

Separate the state from its name, on the model that content-addressed version control and container registries already use: immutable objects under a content hash, plus small mutable *refs* that name one of them. Git's object store and its branch refs are the reference implementation; a container tag resolving to an image digest is the same object in a different domain. Applied here, a session's key-value state is checkpointed as an **immutable, content-addressed snapshot**, and the session is a **named mutable ref** pointing at the most recent one.

Two properties follow, and the second is the one that matters for a peer network. Immutable snapshots replicate freely, because a content hash makes a copy self-verifying and there is no coherence problem between replicas of an object that cannot change. So snapshots can be pushed to several peers and across several failure domains, and the more peers in the pool the more places a given snapshot can live. **Churn resilience therefore improves with pool size rather than degrading with it**, which inverts the usual property of a volunteer network. The ref itself is a pointer of a few tens of bytes and can sit wherever the operator already keeps consistent metadata. Losing the owner then costs the delta accumulated since the last named snapshot, which is bounded by the checkpoint interval, and not the session.

Remark 6 (Key-value state does not merge, and this constraint is load-bearing)**.** *The analogy to version control stops at exactly one place and the design has to stop with it. Text three-way-merges because a document is a sequence of independently meaningful lines with a common ancestor, so two divergent edits can be reconciled by a rule. Attention state is not that. Two consumers who advance the same session's key-value cache along different continuations hold states that no rule reconciles, and there is no merge function to write. So a session name must advance* ***fast-forward only, under a single designated writer****. Every other consumer that wants to continue from a snapshot* forks*: it takes the snapshot, starts its own ref, and diverges*

by design, which is the intended and cheap operation. Multi-writer access to one name is not a merge conflict, it is a silent-corruption path, because nothing errors. The second writer's update simply overwrites a pointer, the first writer's continuation is orphaned, and the surviving state is coherent-looking and wrong. A system that permits it will not produce a diagnosable failure, which is worse than the hang the restart contract was written to avoid.

One limitation belongs with the answer, because it is what makes the answer specific to this architecture rather than general. Replication rescues the *tower* topology, where a session's state is an artifact belonging to one tower's pool and any sufficiently provisioned peer can host it. It does not rescue a single model sharded across peers. There, what would have to survive a peer's departure is not an artifact but the shard geometry itself, the global assignment of layers or experts to machines, and a global layout is not a thing that can be replicated to a peer the way an object can. This is the same distinction §9.2 draws on latency, arriving from the direction of failure rather than of throughput, and it is one more reason the two topologies should not be discussed as if they were one.

11.5 Overlapping corpora and data residency

Proposition 2 requires tower corpora to overlap, and §13 treats towers as clean residency partitions. Both cannot hold, and the architecture as usually stated contains both without noticing.

The resolution is to split the corpus by residency class rather than by tower. A **shared core** of public, licensed, non-personal data may be consumed by every tower, and Proposition 2's overlap benefit applies to it in full. **Residency-bound shards** are consumed only by towers certified for that jurisdiction, and contribute to $\left|\bigcup_i \mathcal{C}_i\right|$ without being shareable.

The consequence is that the headroom shrinks, and saying only that the amount "depends on how much of the corpus is residency-bound" stops one step short of the number that matters. That is not a quantification, and one line of algebra shows why it matters. Proposition 2's saving applies only to the shared core, so the effective peak requirement is $\max_i\big(|\text{core}|+|\text{shard}_i|\big)$ rather than $\max_i D_{\text{opt}}(N_i)$ over an unconstrained corpus. Write f for the residency-bound fraction of each tower's corpus. Each tower still needs $|\text{core}| + |\text{shard}_i| \geq D_{\text{opt}}(N_i) = 5.6 \times 10^{13}$, so $|\text{core}| = (1-f) \times 5.6 \times 10^{13}$ and $|\text{shard}_i| = f \times 5.6 \times 10^{13}$, and the total unique supply the system has to find is

$$(1-f)\, D_{\text{opt}}(N_i) \;+\; 5f\, D_{\text{opt}}(N_i) \;=\; 5.6 \times 10^{13}\,(1+4f). \tag{19}$$

At $f = 0$ this is Proposition 2's benefit in full. At $f = 1$ it is 2.8×10^{14}, which is exactly what a 14T monolith needs with no decomposition at all, so the benefit is not merely smaller but zero. The benefit is linear in f, $f = 1$ is reachable rather than pathological for an operator serving only regulated jurisdictions, and nothing in this paper bounds f.

Reading Eq. (19) against the token-supply estimate of §2 is worse than we would like. Against the central unique-stock figure of about 6×10^{13}, the decomposed requirement is already at the estimate at $f = 0$ and exceeds it for f above about 0.02. Against the 90 percent upper bound of 2×10^{14} it survives to $f \approx 0.64$ and fails beyond. So the statement at §2.3 that tower decomposition "converts an impossible unique-data requirement into a possible one" holds only at the optimistic end of a supply estimate and the low end of an unmeasured f. Measuring f is measurement F12, and it needs no accelerators.

12 The observation bound

Everything so far removes a constraint. Sparsity removes the compute limit. Tower decomposition raises the data ceiling. Four-bit serving handles memory. Adapters handle tenancy. None of it touches the quantity that caps the system once human text runs out, and this section argues that quantity is the real subject of the paper.

12.1 What happens when the tokens are gone

Past exhaustion there are three moves. *Repeat* the corpus, which is bounded: up to about four epochs of repeated data costs almost nothing against unique data, and degrades after [43]. *Synthesise* from existing text, which is derivative, since it resamples a distribution the ensemble already holds. Or *ground*: generate tokens by acting on something and seeing what happens.

The obvious form of the third move is to have the towers talk to each other. That is where the literature has something uncomfortable to say.

12.2 Four results that say the same thing

Four separate lines of work, using different methods on different questions, arrive at one conclusion.

(1) **Debate is a martingale, and informational diversity does not break it.** Under a Dirichlet-Categorical belief model, standard multi-agent debate induces a martingale over agents' belief in the correct answer, so expected correctness does not improve over rounds, and the result holds for homogeneous agents under unweighted belief updates [16]. The same authors extend it: collective belief remains a martingale with heterogeneous agents. Deliberation reinforces a shared prior rather than correcting it, and holding informatively different views does not by itself change that. Two mechanisms are known to break the martingale, and both are internal to the protocol rather than external to the model. Weighting each agent's contribution by a calibrated confidence that is positively correlated with correctness turns the belief process into a strict submartingale, so expected correctness strictly increases [83]. Diversity-aware initialisation raises the prior probability that the debate starts from a correct answer, and its own authors are explicit that it does so *without changing the update dynamics*, which remain a martingale [83]. Empirically, multi-agent debate does not reliably beat self-consistency [38].

(2) **Reinforcement learning with verifiable rewards sharpens rather than expands.** Probing the reasoning boundary with pass@k at large k, base models catch up with their reinforcement-trained versions across every benchmark and model family tested, and eventually surpass them [79]. The training improves sampling efficiency; it does not enlarge the set of problems that can be solved.

(3) **Recursive self-training without grounding provably degenerates.** Formalised as a discrete-time dynamical system, self-training exhibits two failure modes: entropy decay, where finite sampling monotonically destroys distributional diversity, and variance amplification, where the absence of persistent grounding produces drift by a random walk. If the fraction of exogenous, externally grounded signal vanishes asymptotically, degeneration follows [80]. Model collapse is the empirical face of this [64], though it is not automatic:

collapse is established when synthetic data *replaces* real data, and accumulating both together avoids it [26].

(4) **Self-play works when something external checks the answer.** The strongest zero-human-data result trains a model to propose and solve its own tasks, and reaches state-of-the-art coding and mathematical reasoning without external data, using *a code executor* to validate proposed tasks and verify answers [84]. The executor is not a detail. It is the entire source of correction.

Silver and Sutton frame the destination [65]: experiential data will come to dwarf human data, with agents learning from continuous streams of action, perception, and feedback. The four results above say something more specific about why, and what the limiting resource is.

12.3 The bound

Putting them together gives a statement we take as the paper's thesis.

> **At LOGOS scale, capability is bounded by observation bandwidth, not by parameters or compute.** Towers arguing under unweighted belief updates gain nothing, whether or not they hold the same information. Reinforcement learning on a fixed problem set sharpens what is already reachable. Self-training without grounding decays. The only mechanism that reliably adds capability is a channel to something outside the model that can say the model is wrong, and the rate of improvement is the rate at which that channel delivers corrections.

One route around the martingale is documented and it does not escape the bound. Confidence-weighted debate improves correctness only under the assumption that confidence is positively correlated with correctness, and the calibration that buys that correlation is itself purchased with external supervision: reinforcement-learning calibration on a hand-curated harder subset with known answers [83]. The exogenous signal moves from the debate into the calibrator; it is not removed. That is the pattern the bound predicts.

The same pattern is what a calibrated typed head is. A `DEC` tower (§3.7) answers with probabilities trained by a proper scoring rule against outcomes the environment settles, and its temperature is fitted afterwards on held-out rows. That fit is where the exogenous signal enters: the head's confidence correlates with correctness exactly to the extent that settled outcomes were paid for, and a head fitted on labels averaged from other models has bought nothing. The calibrator is built in, and the bound is unchanged.

This is a synthesis of established results, and we want to be exact about which parts are ours. The martingale result, the pass@k boundary result, the degeneration theorem, and the code-executor existence result all belong to their authors. What we add is three things.

First, the bound is the answer to this paper's own question. Sections 2 to 11 lift every other constraint on a 10T system, and this one is left standing. An operator who has solved compute, corpus, and memory has a system whose ceiling is set by how fast the world corrects it.

Second, and this is the specific contribution, we take a position against the result we have just cited, and we would rather do that in the open than hide it in a paraphrase. **Towers pretrained on disjoint corpora under different objectives with different alignment histories differ in a way that persona-level heterogeneity does not capture, and that difference is exploitable.** The cited authors say otherwise. Choi et al. extend the martingale

explicitly to heterogeneous agents, and Zhu et al.'s diversity result changes only the starting distribution [16, 83]. Both evaluate one base model under different prompts, personas or priors. Our position is that corpus-level difference is a different object from prompt-level difference, because it changes what an agent can condition on rather than how it is asked to respond, and that the martingale extension has not been tested against it. This is Tier C, it contradicts a published theorem's stated extension, and it is measurement F13.

The position is stated at $\lambda = 0$ in the sense of §3.3, and the architecture of §3 cannot supply that, because Branch-Adapt-Route needs a common seed. So the object under test here and the object the reference architecture can build are not the same object, and the gap between them is whether a divergent continued-pretraining branch is a large enough perturbation to do what an independent pretraining run would. That is measurement F14, and until it returns, this subsection is about independently pretrained models rather than about the towers of this paper.

We are also careful about what the position would buy if it held. Even a broken martingale is a protocol result about debate. It does not on its own supply the exogenous correction the bound names, which is why the loop below adjudicates against an environment rather than against a majority.

Third, the bound gives an admission rule. If exogenous signal is the scarce resource, trajectories should be kept in proportion to how much of it they carry.

12.4 Where this sits in the literature

Three adjacent positions should be named, because a bound stated as a terminal contribution has to say what it is not. Genewein et al. enumerate four frictions on the path from general to superhuman capability, a data wall, an abstraction barrier, an embodied bottleneck, and multi-agent trust, and explicitly decline to say which of them binds [25]. Ding and Wang add environment determinism as a complementary binding axis, with a bound on chain-task success and a floor on reward optimisation [20]. Sun et al. state the closest empirical version of our thesis, and they state it nine months earlier: in agentic self-learning, generative-reward-model verification capacity is the main bottleneck, and freezing the verifier induces reward hacking and stalls progress [71].

Our position is narrower than the first and different from the second. We do not survey the frictions; we name one as the residual after §§2 to 11 lift compute, corpus and memory for one specific 14T architecture, and we instantiate it as a loop that runs on a single accelerator. Against Sun et al. we say less than the framing above might suggest. Their verification-capacity bottleneck and our observation bound are, as far as we can tell, the same quantity measured on different substrates, and where the two differ is scope rather than content: they measure it inside a search environment with a learned verifier, and we argue it survives when the verifier is replaced by an environment nobody trained. If that difference turns out not to matter, the bound is theirs and the contribution here is the architecture-specific derivation and the loop.

12.5 The loop

Define the yield of a trajectory τ as the surprisal of the observed outcome under the ensemble's own prediction before it acted:

$$\text{yield}(\tau) \;=\; -\log P_M\big(o_{\text{observed}} \mid \text{context},\, a\big). \tag{20}$$

A trajectory where the towers agreed and were right scores near zero. It is self-confirmation, which is the entropy-decay path of result (3). A trajectory where the towers disagreed and the environment settled it scores high.

The loop, which the companion specification `logos/LOGOS_HARNESS.md` develops as `logos-harness`, runs:

1. Show an observation to the substrate's towers independently. Each returns what its kind returns in §3.7. Independence here is a property of the gate's reading: the pass probability step 2 scores is read from `DEC` without `SIM`'s prefix. The acting `DEC` tower does read the `SIM` tower's prediction, because `SIM` informs `DEC`.
2. Score the divergence between their predictions, for example the `DEC` tower's pass probability against the outcome the `SIM` tower predicts. Discard candidates where they already agree: a settled disagreement is where the environment's verdict carries information the ensemble did not already hold, and an agreed prediction that turns out right is self-confirmation.
3. Act on the environment.
4. Let the environment settle the disagreement.
5. Score Eq. (20). In the four mechanics of §3.7, P_M in Eq. (20) is what a `SIM` tower emits, the prediction over the next observation; the divergence and the yield are read off it, and the `DEC` readout on every node is the calibrated probability that the trajectory will be confirmed, scored against the same settlement.
6. Admit the trajectory weighted by yield, and accumulate rather than replace, per result (3).
7. Retrain, and repeat. Disagreement shrinks where the environment has been explored, which pushes generation toward the frontier without being told to.

Steps 2 and 6 do the work. The loop structurally cannot train on its own confident agreement, which is the distribution-narrowing dynamic behind collapse, and step 6's accumulation is the condition under which collapse is known to be avoidable [26].

Step 2 has a consequence for evaluation that has to be stated, because it bites exactly where this loop meets the forecasting measurements of §14. Gating on disagreement selects a subsample on which the ensemble is uncertain, and uncertainty correlates with difficulty, so a Brier or skill score computed on the admitted trajectories is not comparable to a persistence, climatology or market baseline computed over a full question set fixed in advance, and the direction of the bias is not known in advance. Any use of this loop to score forecast skill needs an ungated arm run over the whole roster alongside the gated one. The gate is an admission rule for training data. It is not an evaluation protocol, and using it as one would manufacture a result.

12.6 Two environments, chosen because they fail differently

An emulated game is the cheap test. A Game Boy emulator run at high throughput [69] generates trajectories with exact ground truth in memory. They feed the pretraining loop above as process data; reinforcement learning is at most a later post-training step and has no part in this test. A raw-frames-only benchmark [82] evaluates the trained model in the same terms used for frontier vision-language models. Observations are tokenized by a residual-quantized codebook into a vocabulary shared with text, and the model is a small early-fusion decoder in

the Chameleon and Emu3 line [11, 21], with loss masked on observation tokens so it learns to read observations rather than generate them [57]. The decisive experiment holds out a vocabulary: terms scrubbed from the entire text stream, acquirable only through grounded trajectories, probed against a matched control set that does appear in text. Adjudication takes milliseconds, ground truth is exact, and the semantics under test are printed on screen, so the observation channel's fidelity can be checked pixel by pixel.

Social reality is the test that decides, and the observation channel is the companion psychohistory program's own pipeline [61]: block states, belief-dispersion observables, community partitions, operator-concentration statistics. Towers propose competing readings of a developing episode with a forecast at a fixed horizon, and reality settles it.

The difference between the two matters more than it looks. An emulator is a deterministic program, and a code executor is too. A sufficiently large model can in principle internalise either, at which point yield legitimately goes to zero and the well runs dry. That is the limit of result (4): the strongest self-play result uses an environment the model could eventually learn to simulate. Social reality cannot be internalised, which is the companion program's central negative finding about out-of-model agents restated as a resource. Environments that resist simulation are the renewable ones.

Remark 7 (Social reality does not scale as a token source)**.** *A generator producing a few thousand good trajectories per quarter falls further than three orders of magnitude short of relevance to a 2.8T tower; three is the flattering reading. The companion specification recomputes the shortfall from measured quantities of the pipeline named above (`logos/LOGOS_HARNESS.md` §4.3). The window length and the window count are properties of the analysis rather than estimates; the two remaining inputs, the tokens per trajectory and D_{opt}, are estimates and are named as such below, both running conservative against this paper's own conclusion. The analysis window is $[\text{onset} - 90\ d,\ \text{onset} + 22\ d)$, so no verdict exists until 22 days of post-onset reality have accrued, and nothing in the design shortens that floor. The verdict is one bit per trajectory, `fires_vs_shuffle`, plus two scalars over the whole window, which is not the dense step-wise signal a training loop consumes. The continuous record the pipeline runs on spans 1764 days, which at that window length is fifteen non-overlapping adjudications on the entire dump, and thirty-nine even at the repository's looser 45-day separation. And the measured lifetime yield, across every substrate and every run the companion programme has performed, is 93 adjudicated windows. A figure of 94 circulates in the companion and does not sum from its own breakdown. Against a 2.8T tower's $D_{\text{opt}} = 5.6 \times 10^{13}$ tokens, itself the order-of-magnitude placeholder §2.3 declares measurement F1 against, and pricing a trajectory generously at 10^4 tokens, three thousand trajectories per quarter is 3×10^7 tokens and therefore **6.3 orders of magnitude short**; at the measured lifetime yield it is 7.8 orders. Call it six to eight, not three. The emulator is for volume and reality is for validity; conflating them voids the result. If grounded trajectories help on the emulator but not when reality adjudicates, the mechanism was learning emulator artifacts. Retrospective backtesting against held-out rosters looks like a partial relief at the usual price in hindsight contamination. That price is not payable here: every stand-in tower has read about the episodes on the roster, 82 of 83 roster rows carry onsets before 2025-01-01 so knowledge-cutoff-based model selection leaves one usable row, and entity scrubbing raises the floor without stripping structural recognisability. What it puts in place instead is the block-label shuffle and the matched-calm arm, which generate episodes that never happened and therefore cannot have been memorised.*

12.7 The ordering F9 measures

The bound predicts an ordering: grounded trajectories beat disagreement-gated self-play, which beats unfiltered self-play, which beats nothing. Measurement F9 runs the four arms at matched token counts on one consumer accelerator, and it is the cheapest experiment in this paper.

13 Governance

13.1 The threshold

Under the EU AI Act, a general-purpose AI model is presumed to present systemic risk when cumulative training compute exceeds 10^{25} floating-point operations [22]. Meeting the presumption triggers the obligations in the General-Purpose AI Code of Practice [27]: a state-of-the-art safety and security framework, continuous lifecycle risk assessment covering automated cyber-offence, biological and chemical proliferation, and loss of control, a safety and security model report, board-level accountability, and whistleblower protection. Exposure for non-compliance runs to €35 million or seven percent of global annual turnover.

Supervision is national. In the Netherlands the Autoriteit Persoonsgegevens coordinates oversight with attention to transparency and high-risk applications, alongside the Rijksinspectie Digitale Infrastructuur for technical market surveillance [5, 59]. Turning obligations into enforceable mechanism is lifecycle governance work, and the Artificial Intelligence Governance Professional certification [31] is the current credential for it.

13.2 How many models is a Mixture-of-Towers?

The threshold is written against *a model*. A Mixture-of-Towers is five separately pretrained and separately post-trained models joined by a router that is itself trained. Three readings follow, all defensible from the text:

(R1) **Per-tower.** Each tower is assessed alone. A sparse 2.8T tower at 2.2 to 3.1×10^{25} FLOPs clears 10^{25} by a factor between 2.2 and 3.1. On the bracket of §2.2 that is a comfortable in-scope verdict rather than a marginal one, so the per-tower reading no longer offers an escape and R3 carries correspondingly more of the arbitrage. The margin would still close for a tower an order of magnitude sparser, or trained on a tenth the tokens, so the threshold remains sensitive to design choices a provider controls.

(R2) **Composed system.** The ensemble is one model whose cumulative training compute is the sum over towers plus the router, about 1.1 to 1.5×10^{26} FLOPs, clearly in scope on any reading.

(R3) **Router only.** The training the provider performed *as a composition* is the router, whose compute is negligible. On this reading the composed system is an integration of third-party models rather than a model the integrator trained.

R3 is the arbitrage, and it is not obviously wrong. Branch-Adapt-Route's whole economic proposition is that towers can be sourced, swapped, and upgraded independently [42], and an integrator who obtains five open-weight towers and trains only a router has performed a very small amount of training. Pricing that training is harder than it looks, and this paper does not derive a point figure: §4 specifies the router qualitatively and prices exploration in quality rather

than in FLOPs. A bracket is the defensible form. Charging the router's own signal-gathering at the order of 10^{11} FLOPs per token that a tower forward pass costs, §11.3's oracle-ceiling evaluation over 10^6 queries of 2000 tokens on five towers is about 10^{21} FLOPs; §4's exploration slice at $\varepsilon = 0.05$ over 10^{13} production tokens is about 5×10^{22}; at $\varepsilon = 0.10$ over 10^{14} it is about 10^{24}. The bracket spans three orders and its top end is still ten times under 10^{25}, which makes Position 3 stronger rather than weaker, because the arbitrage survives the most generous accounting we can construct. Whether tower inference spent generating router training signal counts as the integrator's training compute at all is exactly the accounting question at issue, and whether the presumption attaches to the compute a provider expended or to the compute embodied in what it places on the market is not resolved by the text or the Code of Practice as far as we can determine.

Position 3 (Modular composition is a compute-threshold arbitrage surface)**.** *Any capability threshold defined on cumulative training compute per model is evadable by modular composition, because the compute attributable to the composing step can be made arbitrarily small while the capability of the composed system is bounded below by that of its strongest component. A threshold robust to this must attach to deployed system capability, or to embodied compute including sourced components.*

Naming an arbitrage surface makes it easier to use and easier to close. We judge disclosure correct here because the surface follows immediately from reading the threshold next to any modular-composition paper, and because the parties who need it are regulators rather than victims who can be harmed by it. This is the disclosure calculus the companion psychohistory program applies to its control layer [61], and we keep to it.

13.3 Routing as the compliance mechanism

Residency law forbids sensitive regional data from crossing foreign infrastructure. Modularity is what makes the prohibition enforceable rather than aspirational: the operator restricts domains by region, and when a prompt touching sovereign data would otherwise route through an uncertified path, a mask forces it elsewhere:

$$G_{\text{masked}}(t)_e = \begin{cases} G(t)_e & \text{if } e \in \mathcal{A}_{\text{safe}}, \\ -\infty & \text{if } e \notin \mathcal{A}_{\text{safe}}. \end{cases} \tag{21}$$

Per §4 the mask applies after routing, which is what makes it non-bypassable. Structurally it is the same object as the classifier demotion of §1: a capability-ordered routing decision, differing only in the objective the ordering encodes.

Supply-chain integrity is enforced before dispatch, with the router hashing the target module against an immutable ledger and aborting on mismatch, so a substituted tower cannot be reached. And per §1, forensic capability must stay local, so that incident analysis cannot be revoked by a third party's classifier mid-incident.

Remark 8 (What the mask does not guarantee)**.** *Masking the expert set constrains* which weights execute*. It does not constrain what the permitted weights know, and it does not prevent an unmasked tower from having trained on data a masked tower was forbidden. Residency enforcement at the routing layer controls data in flight, not capability. Treating it as the latter is an error the architecture invites, §11.5 is where the corpus side of the problem is actually handled, and §13.5 is the only mechanism in this paper that produces evidence about it after the fact.*

13.4 Alignment does not survive compression by default

Every deployment this architecture describes ships four-bit weights, because §7.1 shows the 14T ensemble does not fit otherwise. That makes a measurement from §7.4 a governance fact rather than an engineering one, and it is filed here as well as there because the two audiences are different and only one of them reads the quantization section.

An aligned model was compressed and then recovered by two routes at matched base quality. Recovery by cross-entropy on base data — quantization-aware *training* — retained the installed behaviour at 0.0022 against the full-precision model's 0.9775. Recovery by distillation against the full-precision aligned teacher retained 0.5452. **The compression step did not remove the behaviour; the recovery step did.** The mechanism is stated in §7.4 and it generalises past the substrate it was measured on: a cross-entropy recovery pass optimises the base objective, so what it overwrites first is whatever was installed last, and in any multi-stage pipeline that is the alignment.

Three consequences follow for a provider, and the first two are cheap.

(1) **Safety evaluations do not transfer across the quantization boundary.** A safety and security model report under the Code of Practice [27] that evaluates a full-precision checkpoint says nothing about the four-bit artifact actually served, and the gap is not a rounding effect but a factor of several hundred on the measurement above. Whatever behaviour a report attests to has to be re-measured on the served weights.

(2) **The recovery objective is a governance-relevant configuration choice**, on the same footing as the safety mask of Eq. (21), and it should be disclosed with the artifact rather than left inside a compression pipeline.

(3) **A sourced tower carries this risk into the composed system.** Under the reading of §13.2 where the integrator trains only a router, the integrator does not perform the quantization either, and an upstream provider's recovery objective then determines how much of an alignment property survives into a system the integrator places on the market. That is the same accounting gap Position 3 names, reached from the deployment side instead of the compute side.

The measurement behind this is at 8.5×10^7 parameters on a synthetic substrate, one behaviour, one recovery budget, and the caveats in §7.4 — including a confound between the two arms' objectives and their recovery data — apply here in full. What it supports is that alignment retention under compression is a quantity a deployment must measure, not that any particular fraction of it is lost.

13.5 Provenance attestation, and why it needs no labels

The mask is a control and §11.5 is a corpus policy. Neither leaves a trace. An operator who says a tower never consumed a residency-bound shard is making a claim about training that finished months ago, and the architecture as specified gives an auditor nothing to check it against. A residency guarantee that cannot be tested after the fact is a promise, and the section that most needs one is this one.

The second governance primitive is **attestation**: a test, run against the deployed model rather than against the pipeline that produced it, of which corpus version it was trained on. It matters here for a reason beyond its own utility. §4 rules that the router is never scored against

a human label, and names governance as the one deliberate exception, safety being the place where a name has to be imposed. Attestation sits beside that exception without needing one of its own, and the distinction that buys it is worth stating precisely.

A canary tests provenance, not specialisation. Provenance is a fact about the training set: which corpus version was placed in front of which tower, fixed before any gradient was taken, and true or false independently of what the model made of it. Specialisation is what the router found, and it is the thing §4 forbids anyone from grading, because grading it means deciding in advance what the router ought to have found. A canary never asks what a tower learned. It asks only what a tower was shown. So the audit it supports is label-free in the sense that matters: it can return a verdict about a tower whose function nobody can name.

The mechanism

The construction is a key hierarchy, and it is standard. Write H for HMAC-SHA256, ν for a corpus version name and π for a probe identifier. A master key K_{master} of 32 random bytes is held offline. Each corpus version derives its own sub-key, $K_\nu = H(K_{\text{master}}, \nu)$ truncated to 32 bytes, derived on demand and never persisted. Each probe derives a token: the literal `CANARY_` followed by $H(K_\nu, \pi)$ truncated to sixteen hexadecimal characters, that is 64 bits. A commitment $\text{SHA256}(K_\nu \| \nu)$ is published with the corpus and pinned to a source revision, so a verifier later handed K_ν can confirm the version was not silently re-keyed after release. Leaking one K_ν burns one version; leaking K_{master} burns all of them.

What makes it a test rather than a label is the split of the probe identifier space. A published pool of identifiers appears in the released corpus; a held-back pool never does. Both are derivable by the key holder and neither is derivable by anyone else, because forging a token for an unseen identifier is forging an HMAC. A model trained on a leaked copy of the public release reproduces the published pairs from memory and fails the held-back pool. A model trained on the private corpus, or on weights derived from it, answers both. The delta between the two recall rates is the provenance signal, and it separates three verdicts rather than two: trained on the dataset, key leak suspected, and inconclusive. The third verdict is not a hedge; Remark 9 is why it has to exist.

In the implementation this paper draws on, probes are injected as a two-message turn pair at a random message boundary, three to five pairs into a configurable fraction of rows defaulting to one percent, and recorded in row metadata for audit. The trainer preserves that metadata column and does not teach on it: only the inline assistant turn carrying the token is in the training target, so the audit record is not itself a thing the model can learn to recite. The modules are `aginto/canary/` and a `verify_dataset_provenance` command in the harness codebase this work is developed in, which is not the public companion repository, and the mechanism is described here rather than shipped with the paper.

None of that is novel. Hierarchical keyed hashing is what HMAC is for, held-back probe sets are the standard shape of a membership test, and the composition is engineering rather than a contribution. It is stated at this length because the part that follows is not standard, and does not make sense without it.

The mixture-specific consequence

Towers may be trained on different corpora. That is the whole of the residency split of §11.5, where a shared core is consumed by everything and residency-bound shards are consumed only

by certified towers, and it is also what $\lambda \leq 1$ means on the lineage ladder of §3.3, where towers branch from a common seed over corpora that diverge. Wherever the corpora differ, the version key can be derived per (tower, corpus version) pair rather than per corpus version alone.

Per-tower canaries make a deployed ensemble auditable for which tower saw which corpus. The probing path is a mechanism the architecture already has: Eq. (21) restricts the reachable expert set, and an auditor who can force it to a single tower is querying that tower rather than the ensemble. Recall rates on the published and held-back pools of a given corpus version, taken tower by tower, return the assignment of corpora to towers. That is precisely the object the residency claim needs and the mask cannot supply — the mask governs data in flight, a canary attests to data at rest in the weights — and it is exactly what a sovereignty or right-to-audit claim has to be able to check, since "this jurisdiction's data never entered a tower outside it" is a statement about corpora and towers and about nothing else.

The point worth holding onto is what the audit does *not* report. It says a tower consumed a corpus. It says nothing about what the tower does with it, what it specialises in, or whether it specialises at all, and it could not be made to say so, because the token is uncorrelated with the content it was injected among. Provenance per tower, silence on function: that is the only combination compatible with §4's rule, and it is why this is a second governance primitive rather than a second exception.

One structural condition belongs with the claim rather than in a footnote. Forcing the mask to a single tower is a capability the operator has and the auditor does not. A right to audit provenance is therefore a right to compel single-tower routing, and if the deployment does not expose that, the mechanism degrades to a statement about the ensemble as a whole, which recovers nothing the operator could not have asserted. That is a governance requirement on the deployment contract, not a property of the cryptography, and no amount of key hygiene supplies it.

Remark 9 (What a provenance canary does not show)**.** *Four limits, and the mechanism is oversold without all four.*

It is one-sided. *A positive result means a corpus version was present in training. A negative result does not mean it was absent. Exposure does not guarantee memorisation of a 64-bit string carried on a small fraction of rows, and quantisation, subsequent fine-tuning, or an intervening reinforcement-learning stage can erase what was memorised. The operating threshold in the implementation is a match rate of* 0.8 *rather than* 1*, which is an admission of exactly this in the form of a constant.*

It is defeatable by an ordinary data pipeline, adversarial or not. *Deduplication, perplexity filtering, or any filter that strips high-entropy tokens removes probes while retaining the corpus. The probe format compounds it: a user turn reading* `canary probe #<id>:` *followed by an assistant turn holding the token is trivially found by anyone looking, and can be stripped without touching the data that has value. So the mechanism detects unwitting or careless reuse, and does not detect laundering.* ***It is not a watermark and carries none of the robustness a watermarking claim asserts.*** *Note that this is the same property §10's canary treats as a design commitment rather than an accident: there, canaries are drawn from the same distribution as live traffic, on the reasoning that a canary a node can identify is one it can answer honestly while corrupting everything else. Transposed to provenance, the same principle says probe rows should be indistinguishable from ordinary corpus rows. The format above plainly is not, and that is a defect in the implementation rather than a limit of the idea.*

The key is a single point of failure in both directions. *Compromise of* K_{master} *voids*

every downstream attestation retroactively. Worse, it voids them forward as well: whoever holds K_ν can mint valid tokens for held-back identifiers and manufacture a positive against a model that never saw the corpus, so a compromised key does not merely stop attesting, it starts attesting to false things. The three-verdict logic above exists for this reason, since high held-back recall is genuinely ambiguous between "trained on our data" and "our key leaked". Here too §10 has the stricter discipline: its implementation treats the known-answer pool as verifier-side state and raises on any request for it from node-facing code, on the grounds that a leaked answer pool converts the trap into theatre. The provenance implementation keeps held-back identifiers and the master key out of version control by policy and by ignore rules, which is a weaker thing than a guard that fails closed, and should be read as the weaker thing it is.

It is deliberate contamination. *Injecting probes means training the model to emit a meaningless high-entropy string on request. That is a capability tax and an alignment oddity, both small at one percent of rows and neither zero, and the released corpus is one we have knowingly degraded in order to make it auditable. Anyone presenting the cost as free has not priced it.*

Three mechanisms, one word

This paper now uses "canary" for three unrelated tests, and they answer three different questions. They are separated here so that a reader does not carry a property of one across to another.

Grounding: did this trajectory do the work? An unkeyed random token planted in a generated task's workspace, so that a model which never opened the file cannot produce it. Per-task, discarded afterwards, and used by the task generators of the agentic harness §12 draws its trajectories from.

Provenance: what was this model shown? The keyed construction above, with its held-back probe pool. Per corpus version, and per tower in the extension this section proposes. Its verdict concerns the training set, never the model's function.

Integrity: is this node computing honestly? The known-answer drift test of §10, indistinguishable from live traffic by construction and evaluated against a benign hardware-noise null (`logos/experiments/canary_trap.py`). Its verdict concerns a peer's runtime behaviour, and says nothing about what any model was trained on.

The only property they share is that a secret the subject cannot derive is planted in a place the subject must pass through. Everything downstream of that — what a positive means, what a negative means, and how hard the test is to evade — differs, and Position 2's pessimism about the third transfers to the second not at all, since a provenance probe faces a corpus filter rather than an adversary with a duty cycle.

Nothing in the paper's case for towers depends on any of this. It is offered as the one thing §13 otherwise lacks, which is a way for someone outside the operator to check a claim the operator makes, and it costs no accelerators to test.

14 Relation to the psychohistory program

This paper shares a repository with, and is a deliberate counterpart to, the bounded-psychohistory program [61]. The connection runs in both directions and neither is decorative.

That program's central negative result is that its forward engine fails against *out-of-model agents*, whose type its hypothesis space never contained and against which more data does not help. It also names *major players*, agents large enough to break mean-field averaging, who must be modelled explicitly. LOGOS-class systems instantiate both. A 10T system executing autonomously over long horizons is a major player by construction, and one that discovers novel exploit chains without source access [55] is out-of-model on the record.

The Mixture-of-Towers is also a near-decomposability design [66] for a machine: dense interaction inside a tower, weak interaction between towers, with an aggregate law over coarse-grained units. It is the same architectural bet that program makes about societies, and it fails the same way, when inter-unit coupling grows and the units stop being independent. Socially that is a critical transition; here it is router-induced correlation, where a router that consistently co-activates towers has rebuilt a monolith with worse ergonomics. Whether the social diagnostic transfers is genuinely open. That program's measure of independent units is a different estimator on a different substrate, and borrowing the name would not license borrowing the validation, so we record it as a hypothesis rather than a result.

The more useful direction runs the other way. Section 12 needs an observation channel the ensemble cannot fake, and social reality is one, for exactly the reason that program identifies as its own limit. Meanwhile that program's forward measurements are not all blocked on the same thing, and the coupling is narrower than it first appears. Smooth-regime skill is not blocked: it has a one-block pilot over 45 scored steps that beat climatology and tied persistence, which is a measured negative rather than an unstarted test. Of the three that remain unstarted, fixed-point reliability needs a lodged announcement set with outcomes, regime occupancy needs a regime monitor operationalised on a named live series, and only parameter invariance across regime change needs the coupled engine. All three additionally need a multi-regime social reanalysis corpus that does not exist, which that program's own ledger names as its structural blocker and as a data-access problem rather than a compute one. The loop of §12 could supply the multi-block forward engine that parameter invariance needs. It does not supply the corpus, the question set, or the monitor. So the dependency is real, narrow, and one-directional, and the earlier framing of "the same missing piece from two sides" overstated it in the direction that flattered both papers.

There is a final convergence, and it is not comfortable for either paper. Psychohistory ends at an observability trilemma: complete predictability would require total observability, which is surveillance and the abolition of independent agents, and that program rejects the limit. This paper ends at an observation bound: capability past the token wall is set by the rate at which the world corrects the model. Both papers therefore terminate on the same quantity, approached from opposite sides. One says you cannot predict people without watching them; the other says you cannot improve past human text without watching something. When the something is people, the two limits are the same limit, and the governance problem of §13 is not a separate section but the place where the argument was always going to end up.

15 What has been measured

Every section before this one is specification. This one is not. It reports the measurements that have been run against a working harness, `logos-harness`, on one 24 GB consumer accelerator, and it exists because a partition criterion that is only argued for is the thing routing was invented to avoid.

The scale caveat comes first and applies to everything below. **Three rigs are reported, all on one 24 GB consumer accelerator, and none is a small version of the machine this paper specifies.** The first is a set of byte-level decoders of about 1.7×10^7 parameters, built to make one measurement answerable; §15.1 through §15.5 run on it. The third is a 8.5×10^7-parameter decoder on a synthetic recall substrate with a computable Bayes ceiling, used for the quantization measurements of §7.4, which are reported in the quantization section because that is where they belong; a fourth instrument, the integrity detector of §10.2, needs no accelerator at all. The second, reported in §4.5 and §15.6, is a Qwen3-0.6B base upcycled into four towers on one RTX 3090: 1.257×10^9 total parameters against 5.96×10^8 active, trained for 0.096 epochs of an 84.9 M-token corpus of recorded agentic trajectories, which is roughly 8×10^6 tokens seen and a token-to-parameter ratio of 0.0186 against Chinchilla's twenty. Nothing here was measured at 10^{12} parameters, nothing here was trained to convergence, and no result below transfers to tower scale by any argument we can make. What runs this size can do is kill an operationalisation, or show that a mechanism the specification assumes is absent from the artifact implementing it. Those are different and much cheaper services: a criterion that fails on four adjacent programming languages at 17M parameters does not become sound at 2.8T, and a router whose gradient is exactly zero at initialisation for structural reasons is exactly zero at any width. The results reported here are of those two kinds.

15.1 One criterion, three operationalisations, one survivor

§3.5's P1 asks whether two candidate towers' corpora are different enough to justify separate parameters. Three ways of asking it were built and run against the same real corpora — source trees in Python, JavaScript, TypeScript and Markdown, drawn from independent open-source projects. Two of the three are dead.

The null is the whole measurement. Before any of them, one discipline decides whether the numbers mean anything. A separation between two domains is only interpretable against a *within*-domain baseline: how far apart do two samples of the same domain sit? The first baseline built here split each corpus by document, and it was wrong in a way that inflated every result. Documents from one project landed on both sides of the split — shared authorship, house style, licence headers, occasionally copied blocks — so the baseline measured within-*project* redundancy and made every cross-domain gap look larger than it was. Under a project-disjoint split, in which no project appears on both sides, the measured separation of the compression variant fell from +0.0746 to +0.0168. The correction is not a detail; it is the difference between a positive result and a null one, and it was found only because the baseline was built to be attacked.

P1 as written: token-type Jaccard. Dead. Python against English prose measures $J = 0.3246$, above P1's cut of 0.30, so the criterion as written merges code and prose into one tower. Moving the cut does not help, because the problem is range rather than placement: four *mutually unrelated* corpora span only 0.26 to 0.32 end to end. The measure has less spread across unrelated domains than the partition needs between related ones, and any threshold placed inside that band is separating noise. The cause is structural — lexical overlap between English-derived technical corpora is dominated by shared identifier and punctuation vocabulary, which is present whatever the corpora are about — so this kills the operationalisation rather than P1.

Normalized compression distance. Dead. Compression distance is model-free, needs no accelerator, and on the first null it appeared to vindicate the criterion, which is why the project-disjoint null was built to attack it. It fails the negative control. Under NCD lower means more similar, and Markdown sits to Python at 0.9661 — closer than JavaScript sits to *itself* across independent projects, at 0.9814. A measure that places two different domains nearer each other than one domain is to a fresh sample of itself cannot support any partition, at any cut point, in either direction. The aggregate separation of +0.0168 is the same fact stated less sharply. Distances are computed with the compressor and dictionary size pinned in the result record, since an unpinned window silently changes what is being measured.

Gradient conflict. Survives. The surviving operationalisation abandons model-free measurement. One model is trained on the mixture of all domains; at checkpoints, the gradient of each domain's loss is taken with respect to every parameter, and pairs of domains are compared by the cosine between those gradients. The question it asks is the one the architecture actually cares about — *do these domains pull the shared parameters in different directions* — rather than whether their surface statistics differ.

It also removes the failure mode that killed the other two, because **it has no cut point to place**. Within-domain replicates are drawn from *project-disjoint* batches, so the within-domain cosine is the measured spread of "two unrelated codebases in the same language", and a pair is called separable only when its between-domain cosine falls below the weakest such baseline. The threshold is a property of the data, recomputed at every checkpoint. Nothing about the verdict can be tuned by choosing a number, which is why the two paragraphs above spend their time on range and controls rather than on where 0.30 should have been.

step	within	between	separation	complete	
0	0.7963	0.6560	0.1402	no	(untrained control)
200	0.3007	0.1308	0.1699	yes	
600	0.2548	0.0870	0.1678	yes	
1500	0.2105	0.0765	0.1340	yes	
3000	0.1595	0.0387	0.1207	yes	

Complete separation — every between-domain pair below every within-domain baseline — holds from step 200 through step 3000. The per-pair verdicts are also ones a reader can check against their own judgement, which neither predecessor managed: prose separates from all three programming languages (Markdown against Python at +0.4808), the programming languages merge with each other, and the pair with the highest agreement is JavaScript against TypeScript at +0.8024, which genuinely should merge.

The untrained control earns its place in the table. At step 0 the measure reports *not* separable, with uniformly high cosines, because early gradients are dominated by generic structure — byte frequency, whitespace, the shape of the loss surface. A single reading at initialisation returns the wrong answer, which is why a trajectory is reported rather than a checkpoint, and it is also what shows the measure is not trivially separating whatever it is handed.

What this costs, and what it leaves open. The cost class of F11 has changed and §16 is corrected accordingly. The two operationalisations that needed no accelerator are the two that

failed; the survivor requires training a model on the mixture, which puts F11 on the consumer-accelerator list rather than off the hardware ledger entirely. Two things remain open and are flagged rather than extrapolated. Separation narrows monotonically with training, from 0.1699 at step 200 to 0.1207 at step 3000, as both cosines decay; whether it survives a full training run is unmeasured, and a criterion that dissolves at convergence would be no criterion. And the measure has been applied to four programming languages, not to the five reference domains of §3.5. Until it is, the "four towers, not five" verdict remains an argument rather than a measurement, and F11 is not discharged.

15.2 The criterion, run on the five reference domains

§15.1 applied gradient conflict to four programming languages, and noted that until it was pointed at the five domains §3.5 names, "four towers, not five" remained an argument rather than a measurement. It has now been pointed at them. Corpora are drawn from the Pile's components — GitHub, PubMed Central, arXiv and DM Mathematics, StackExchange, FreeLaw and USPTO — at 12 MB per domain, with the within-domain null built from disjoint document buckets.

within-domain null		between-domain		
code	0.0893	logic | mathematics	−0.0159	separable
administration	0.1766	administration | mathematics	−0.0153	separable
logic	0.1708	code | mathematics	−0.0078	separable
life sciences	0.1887	life sciences | mathematics	+0.0019	separable
mathematics	0.2119	code | life sciences	+0.0418	separable
		administration | code	+0.0555	separable
		life sciences | logic	+0.0811	separable
		code | logic	+0.0851	separable
		administration | life sciences	+0.1047	**MERGE**
		administration | logic	+0.1102	**MERGE**

Complete separation is *false*: two pairs sit above the weakest within-domain baseline, which is code at 0.0893. That is already a difference from the source-tree run, where every pair separated — the five reference domains overlap more than four programming languages do, which is the reverse of what the corpus work assumed.

How to read it, decided before the run. The null here is weaker than the source-tree null and its error has a known direction. The Pile's test shard carries no metadata beyond the component name, so there is nothing to group by and documents are dealt into buckets by content hash; one author's several papers can land on both sides, which inflates the within-domain cosine, lifts the threshold, and admits more pairs as separable. The bias therefore runs toward **over-separation**. A **merge** verdict was reached against it and is robust; a **separable** verdict runs with it and is not. This asymmetry was recorded before the measurement, and it is what makes an imperfect null readable rather than merely regrettable.

Which of these verdicts is a property of the corpora. The table above is the final checkpoint, and §15.1 already committed this measurement to the opposite discipline — "a single reading at initialisation returns the wrong answer, which is why a trajectory is reported

rather than a checkpoint". Applied to its own run, that rule removes verdicts. Across the four trained checkpoints the pair verdicts are:

pair	200	600	1500	3000	
administration | life sciences	M	M	M	M	**stable**
administration | logic	M	M	s	M	unstable
code | logic	s	M	s	s	unstable
life sciences | logic	M	s	s	s	unstable
the other six pairs	s	s	s	s	stable

One merge verdict survives every checkpoint, and it is Administration against Life Sciences. The three that change all involve Logic, and they change because they sit within a few thousandths of the bar: at step 3000 code | logic is 0.0042 below it and life sciences | logic 0.0082 below. A second bias compounds this and is separate from the metadata argument above. The threshold is the *minimum* of five per-domain means, and a minimum over noisy estimates lands low, so the bar is biased downward and pairs are admitted as separable for that reason alone. Both biases push the same way, which is why the asymmetry of §15.2 is if anything understated. The run that produced this table serialised only the means of its replicate cosines, so a margin of 0.0042 cannot yet be compared against replicate spread; the harness now persists the per-replicate values it was discarding, and the question is open until a run using it is read.

The verdict contradicts §3.5 in both directions. That subsection argues Mathematics and Logic merge and that Administration separates cleanly on all three axes. Under the reading above, the defensible half of the contradiction is the robust one: **Administration merges with Life Sciences**, the one verdict that is both reached against the metadata bias and stable across training. The companion Administration–Logic merge printed in the table is *not* stable and does not count, which weakens the finding from two merges to one without changing its direction. The Mathematics–Logic result is the most opposed pair measured, at -0.0159, and Mathematics also has the *highest* within-domain coherence at 0.2119 — a consistent picture of a genuinely distinct distribution — but it is a separable verdict and therefore in the direction the bias favours, so we report it as unsupported for merging and no more.

The consequence: Administration was never a tower. Administration is defined by what it is **for** — declining, citing policy, complying — not by what its text looks like, and a court opinion and a clinical paper are both formal English prose that a model wants the same updates for. No corpus partition produces it, at this scale or any other, and the measurement above shows it directly.

The repair is not a better partition. **Safety and compliance are router-level decisions, not a destination to route to.** A refusal is a classification of the request, trained on safety-labelled data against a classification objective; it is not a distribution to be modelled by seven layers of tower depth, and neither towers nor the corpora that would feed them carry the nuance that training needs. The architecture already has the right mechanism and states it in the wrong terms: Eq. (21) masks router logits after routing, which is exactly a router-level safety decision. What has to go is the parallel framing in which Administration is a tower that requests are handed to. §4.2 and §13 are corrected accordingly, and the reference partition drops to four domains — Code, Life Sciences, Mathematics and Logic — with safety leaving the tower axis entirely.

15.3 Dense against expert routing against tower routing

The comparison §3 needs is a dense model, per-layer expert routing and per-segment tower routing trained on one mixture at matched per-token cost. It has now run, at $d_{\text{model}} = 384$, twelve layers, four units, a trunk-tower-head split of 3–7–2, on a 26.9 MB four-domain source mixture for 26,000 steps.

arm	params	mean bpb	javascript	markdown	python	typescript
dense	16,722,816	**1.1562**	0.8440	1.8155	1.1264	0.8389
moe	73,368,960	1.1616	0.8272	1.8348	1.1685	0.8158
mot	45,644,928	1.2497	0.9297	1.9273	1.2262	0.9154

The expert-routing row is void and is printed only so the run is reported whole. That arm carried a defect found after it completed: its attention blocks used a module that carries its own feed-forward, and the routed expert output was added on top, so every layer ran two feed-forward networks. It was a dense model with a redundant routed FFN, its active parameters were 3.56×10^7 against the dense arm's 1.67×10^7, and "matched per-token FLOPs" was false in its favour. Tower routing was never affected, because a tower is legitimately a stack of complete blocks, so **the dense-against-tower comparison stands and the expert column does not**.

Tower routing is worse than dense by 0.0935 bits per byte. Three conditions of the run were recorded before this number existed, and they bound what it can support. The towers were trained from scratch, which is $\lambda = 0$, where §3.3 specifies $\lambda = 1$. The mixture is 26.9 MB seen 4.0 times, so every arm is far from its compute-optimal point and the sparse arm furthest, since at matched per-token cost it carries more total parameters. And the four domains are ones §15.1 reports as separable but closely related — JavaScript against TypeScript at +0.8024 is the pair it says genuinely should merge — so the mixture offers fewer effective domains than there are towers to divide it. **The result is therefore that depth-coherent specialisation did not pay here, under conditions the architecture does not propose**; it says nothing about whether it pays under conditions it does.

A fourth condition, found afterwards, and what happened when it was repaired. The three above were recorded before the number existed. This one was not, and it is a defect rather than a caveat. The optimiser split matched the substring `head` to decide which parameters to exclude from Muon. The dense decoder names its unembedding `head`, which is correctly excluded; a Mixture-of-Towers additionally names its shared output stack `head`, so fourteen ordinary transformer matrices were excluded too and trained under AdamW with weight decay while the dense arm's equivalent layers stayed on decay-free Muon. **Two of the tower arm's twelve layers were on a different optimiser from the dense arm's, in the run that produced** 0.0935**.** It is also substring containment used as a classifier, which this work bans elsewhere on exactly the grounds that it fails silently.

The split now matches on parameter role rather than on a substring. A rerun with the repair in place, at pilot scale on a different corpus, puts the tower arm last again — dense 1.3842, per-layer experts 1.3727, towers 1.5213, so +0.1371 against dense — and a 60-step pilot before the repair gave +0.0855. Three runs spanning both sides of the repair therefore agree on the direction. **They do not agree on a magnitude and should not be read as though they did**: the step counts are 60, 3000 and 26,000, the corpora differ, and the expert arm's

parameter count moved from 73,368,960 to 45,052,800 between them, so the model code changed as well. The repaired comparison at the step count that produced 0.0935 has not been reported here because it has not finished. What can be said is that the optimiser asymmetry was real, is fixed, and did not turn out to be the explanation.

Two quantities from the same run do not depend on the quality comparison. The mutual information between domain and tower assignment is **0.4237 of the 2.0000 bits** available. This was read as showing that segment routing is not domain-blind — a point against treating Mixtral's token-level null as settling the question at every granularity, and the first datum for F15. **That reading was wrong, on two independent grounds.**

First, the router in that run was never trained. The gate parameter that carries tower competence back to the router defaults to off, and the three-way harness never passed it. With it off the assignment reaches the loss only as an index, indexing is not differentiable with respect to the index, and the router therefore takes no gradient from the language-modelling objective at all. Whatever produced 0.4237 bits, it was not learned routing.

Second, and more damaging to the measure itself, the auxiliary load-balance term produces this quantity on its own. A control was run in which the routing gradient is provably absent and the balance term is the only thing touching the router: on four disjoint, equally sized input ranges it reached 1.551 of 2.000 bits. The mechanism is not subtle — when the classes are balanced, splitting the input along its most salient axis *is* the load-balanced solution, so a router optimising balance alone recovers the classes while learning nothing whatever about which tower predicts better. The same signature appears on the real corpus: the two arms whose routers provably received no language-modelling gradient score MI/H of 0.291 and 0.248, *above* the gated arm's 0.201.

Mutual information between a domain label and a tower assignment therefore does not measure routing, and nothing in this work should rest on it without an ungated control on the same corpus. The measure was the wrong one to reach for in any case: towers here are unlabelled and N is a capacity hyperparameter, so nothing in the design says a tower should correspond to a subject, and the routing analysis this paragraph cites found assignment tracking syntax and token identity rather than subject matter. A router doing its job perfectly can score near zero. What the router is for is lowering the loss, and the label-free measure of that — the gap between learned routing and oracle routing — is what §15.4 now uses instead. And on one real training step, expert routing moved 113,061,888 bytes in twelve dispatches against tower routing's 6,291,456 in one, a factor of **17.97**, which is the figure of §15.5 measured on the comparison rather than on an instrumented forward pass.

15.4 Does the router earn its place, and does each tower?

F15 and F16 were run on a trained checkpoint and reported as not firing, on a substitution test: replace the learned assignment with an arbitrary one, and with a constant one, on identical weights. Learned routing cost 0.1604 bits per byte less than random and 0.1395 less than constant. **Both readings were wrong, and the test itself is retired.**

The substitution margin measures proximity to router collapse, not specialisation. Six arms now share the instrument, and the two whose routers collapsed onto a single tower — mutual information exactly zero, routing entropy exactly zero — return the two largest margins in the set, +0.5826 and +2.4591. (§4.5 corrects a separate "routing entropy exactly 0.000" reading on the other rig as an artifact of a four-sample histogram. The collapse readings here are on a different instrument and stand, but the sample count behind any such figure is

now recorded alongside it, and a reader should ask for it.) The mechanism is mechanical: under collapse most towers were never trained, so randomising the assignment routes most of the evaluation into untrained weights, and the model's own loss measures that damage rather than the value of routing. Among healthy routers the margin is not merely uninformative but unstable, spanning +0.0721 to +0.3195 across four seeds of one configuration — fourteen times the spread of the loss itself. F16 fails the same way: a tower's disable cost tracks whether it received traffic during training.

The instrument that survives is oracle routing, and it needs no labels. Score every held-out segment through every tower, keep the best: that is the ceiling any router could reach on those weights, and the gap between it and the learned router is exactly the loss the router leaves on the table. **[PENDING: oracle gap, four seeds.]** Earlier arms could not measure it: the routing projection was trained by Muon, whose update is the nearest orthogonal matrix and so has a fixed norm whatever the gradient was, which drove the router to a point mass ($p_{\max}$ median 1.0000). A forced route is then gated by the probability of a tower the router did not choose, around 2×10^{-3}, so every substitution arm returns the learned arm's own number. With the projection on AdamW the router stays at $p_{\max} \approx 0.3$ and the comparison has range again. Mutual information against a domain label is not an alternative and should not be reported as one — §15.3 gives the control showing it is produced by the load-balance term alone.

Why none of this says anything about the architecture. Every arm above ran at 0.68 **epochs**. Mixture specialisation arises from capacity pressure *at convergence*: a model that has converged and still cannot fit its data has a reason to divide labour, because division of labour is then the loss-minimising configuration. A model that has not converged has no such reason — it is early, not capacity-bound, and additional towers are capacity it is not yet using. The measurements were taken in a regime where the phenomenon under test cannot appear.

This is a regime error and not a scale error, which matters because the two have different remedies. A larger model at 0.68 epochs fails identically. It also accounts for the one result that no undertraining story fits: at eight towers the held-out loss got *worse* and every tower became free to remove, which is what adding unused capacity to an unconverged model looks like.

What would actually test it. Not a larger model, and not more seeds on this rig. A task small enough to train to genuine convergence, with capacity requirement that can be dialled, and the tower count held against it: below some capacity demand one tower fits everything and the mixture pays dispatch for nothing; above it, one tower cannot and N towers at identical per-token cost can. **The location of that crossover is the number to read.** If no setting produces one — if a single tower wins at every capacity demand at matched per-token cost — the design does not pay, and unlike anything reported above that would be a result rather than an artifact of the instrument.

15.5 Dispatch traffic, counted rather than derived

§9.2 argues that an ensemble routing a query once is serviceable over a wide-area heterogeneous pool where a monolith paying a peer round trip per layer is not. That argument is arithmetic. It has now also been instrumented: a ledger counts bytes crossing a device boundary from the routing decisions a forward pass actually makes, with home placement by index and no affinity scheduling, which is the pessimistic case for the design being argued for. Against per-layer expert routing, per-segment tower routing moves 3.43× less at 4 routed layers, 10.28× less at 12,

and 27.44× less at 32. The ratio is **six-sevenths of the routed depth**, not the routed depth: the three figures are 0.8575, 0.8567 and 0.8575 of L, uniformly, and an earlier version of this sentence rounded that to L. The deficit is the trunk and head, which every segment traverses and which therefore dispatch on both sides. What the measurement supports is that the saving is *proportional to* routed depth and improves as towers deepen; the constant is 6/7, not 1.

The first version of that measurement was wrong by three orders of magnitude, and the error is worth recording because it is the kind an unchecked derivation invites. A segment dispatch moves the *whole segment*, T token-activations, not a single d_{model} vector; counting it as one vector reported a 1318× advantage where the true figure is the routed-layer count. The corrected ledger is a regression test.

The same instrumentation corrected an argument this paper's companion specification had been making. It argued that towers buy N-fold parameters at 1× key-value cost and presented that as an advantage over per-layer expert routing. Measured at equal total depth, the two cache *identically*, and the equality is now itself a test. The error was forgetting that expert routing leaves attention dense and shared — only the feed-forward blocks are sparse — so it already buys N-fold feed-forward parameters at 1× cache. What the cache argument does separate towers from is *separately served models*, which share neither a cache nor a trunk. That is a real distinction, and it argues against a router-over-independent-models design rather than against sparse experts. §9.2's argument is unaffected, because it never rested on cache size.

15.6 Upcycling a real base model: the corpus works, the mixture does not

Everything above runs on byte-level decoders of 1.7×10^7 parameters. This subsection does not. The rig is a Qwen3-0.6B base under its own tokenizer, upcycled into four towers on one RTX 3090 and run against a dense control at matched tokens, 2,000 steps, 0.096 epochs of an 84.9 M-token corpus of recorded agentic trajectories. Three results came out of it, pointing in three directions. §4.5 reports the fourth, which is that the mixture arm's router received no gradient at all, and it should be read before the second result below.

The corpus works, and this is the positive result. Both arms learned to choose the tool the recorded trajectory chose next. **On a fixed denominator the gain is a factor of** 6.5: across 60 held episodes the base model produced a correct second-step call in 2 of them and the mixture arm in 13 after 2,000 steps, with the dense control at 14. The conditional rate the gate reports directly, `gold_novel`, moves from 0.043 to 0.650, which is a factor of fifteen. The distance between those two figures is a selection effect, and it has to be stated rather than left sitting inside a denominator.

arm	step	emitted	n_{novel}	correct	`gold_novel`	correct per 60
mot	0	60	46	2	0.043	0.033
mot	2000	28	20	13	0.650	0.217
dense	2000	35	21	14	0.667	0.233

An episode reaches scoring only if the model emitted a first call at all, so as `emits_call` fell from 60 of 60 to 28 the scored pool shrank with it, and the denominator of `gold_novel` is under the model's own control. **A model that speaks only when it is confident scores better on `gold_novel` without being better overall**, which is why the 6.5× figure leads here: it is

denominator-independent and the fifteenfold one is not. Both are real and they answer different questions. `gold_novel` asks how often the model is right *when it commits to a call and the situation calls for a different tool than its own first choice*; the per-60 rate asks how often it produces a correct second step *across all episodes*. §11.3 names the change to the instrument that removes the effect rather than correcting for it by hand. The conclusion that the two arms do not differ is unchanged on either measure — 13 correct against 14, out of 60.

Two qualifications belong with the headline. **The base model emits a well-formed tool call on essentially every prompt — `emits_call` and `valid_name` are both** 1.00 **— and picks the tool the recording picked** 4.3 **percent of the time.** That is agreement with one expert path, not correctness: §11.3 bounds what the `gold` family can say, and 4.3 percent agreement is not 95.7 percent wrong. And no part of the architecture is implicated in the gain, because the dense control collected it too. What this measures is the corpus.

A metric design point falls out of that and it generalises past this run. The rate at which the model emitted a call *different* from its own first choice fell from 0.783 to 0.367 over exactly the training that raised correct second steps by a factor of 6.5. **Shape-based metrics — did it emit a call, did it emit a different call, is the name well-formed — moved against correctness here rather than with it.** A reader tracking only those would have concluded the run made the model worse. An evaluation of tool use that scores form rather than agreement with a recorded decision is not merely weak on this evidence; it points the wrong way.

The mixture did not beat its dense control. At matched tokens, 2,000 steps, four towers at 1.257×10^9 total and 5.96×10^8 active against one dense model:

arm	held loss	`gold_novel`	`gold_match`
mot	0.8596	0.650	0.317
dense	0.8646	0.667	0.333

The loss gap is 0.0050 in the mixture's favour and both capability figures are in the dense arm's favour. The reading that survives is that the two arms are statistically identical here, with the dense control marginally ahead on the numbers that describe capability rather than fit. **The** 2.11× **parameters of §11.2 bought nothing.** That is what §4.5 predicts rather than a surprise: with no gradient reaching the router and 89.6 percent of segments landing on one tower, the mixture arm's forward pass was a trunk, one tower and a head — a dense model carrying three sets of unused weights in memory. **This does not say a Mixture-of-Towers cannot beat a dense control. It says this one was never given the mechanism by which it would have.**

Composition degrades monotonically, and unconditional chaining does not hold. A separate question is what happens when a tower's slice is applied more than once, which is what any chained or cascaded tower arrangement asks for. Held loss with the pretrained middle slice applied r times and *no* training, so that at initialisation all towers are identical and the measurement is exact rather than a sample over which towers happened to be trained:

r	held loss	change from $r = 1$
1	2.0062	—
2	2.3977	+0.3915
3	2.9550	+0.9488
4	3.4675	+1.4613

The degradation does not diverge — the per-application increments are 0.39, 0.56 and 0.51 nats rather than a runaway — but it is monotone, and it is large against what training buys: one extra application of the slice costs a substantial fraction, on the order of a third to a half, of everything 2,000 steps of training gained. **A range rather than a ratio is deliberate, and the reason is a property of the two measurements rather than caution.** The chain probe and the training run report loss on different held samples — four chunks of 512 tokens scored from position 128 for the probe, sixty-four chunks of 1024 for the run — so the two numbers are not on a common scale and no precise ratio between them is warranted. The comparison is indicative, and stated as such. **The consequence for the specification is that unconditional tower chaining throws away the capable initialisation that is the whole justification for sparse upcycling.** A cascade is viable only if its second stage is gated to a no-op at step 0. That is the same constraint Proposition 3 states from the other side, and §4.5 shows it is not free: a stage gated to a no-op at step 0 is a stage whose gate receives no gradient at step 0, and something has to break the tie deliberately.

15.7 Status

Eleven results across three rigs and one accelerator-free instrument, and none at a scale that settles anything about a ten-trillion-parameter machine. Of the seven reported first, one is a survivor of two failed operationalisations, three are corrections to statements made elsewhere in this work, and one is positive: the corpus moves the rate of correct second steps by a factor of 6.5, and does so on a dense model as readily as on a mixture. The sharpest of the seven is structural rather than empirical: the router in the only implementation of this architecture that exists received a gradient of exactly zero, and Proposition 3 says two design choices the specification makes independently forced it to.

The remaining four were run afterwards, and they change the ledger in three places. **One stated verdict is discharged**: the integrity detector's own verdict rule, which required an AUROC of at least 0.99 at every tamper severity above 3σ of a benign null and an honestly printed boundary below it, was passed on both halves (§10.2). It is not F8, which needs a null measured across heterogeneous accelerators and an adversary with a duty cycle below one, and F8 is not advanced by it. **One result is positive and one verdict inside it is void on its own control**: MXFP4 is indistinguishable from FP16 at 97.0 percent of a computable optimum over eight seeds, which is the most replicated figure in this paper and the assumption §7.1's memory arithmetic rests on, while the MXFP4-against-NVFP4 comparison in the same run passed against an emulator this work broke and §7.4 reports that comparison as wreckage rather than as a result. The same run carries the one measurement here with a governance consequence, which §13.4 states: a quantized model's alignment survives a distillation recovery and does not survive a cross-entropy one. **And two findings are about the router and are negative in the useful way.** Routing entropy at initialisation differed forty-three-fold between two unseeded runs, which turns every single-run routing figure in this paper into a statement of what a configuration can produce rather than a measurement; and a gate initialised off zero

produced a live routing gradient that moved entropy by a factor of 3.2 over 500 steps while two of four towers held exactly zero traffic throughout, which says a live gradient is necessary and not sufficient for a mixture to use its towers.

16 Open measurements

Each entry is a position, a measurement, and the hardware the measurement needs. Three of the sixteen have been run. F11 has been run in the scoped form of §15 and is partially discharged; F15 and F16 were evaluated on a single trained checkpoint and reported as not firing; §15.4 shows that reading does not hold, because the substitution test both rely on measures proximity to router collapse rather than specialisation and returns its largest margins on the arms that have none. **F15 is therefore unrun** and its row below needs rewriting around oracle routing before it is worth running again. A precondition on both has since been found and is recorded here because it is cheap and was missed twice: §4.5 reports a router that received a gradient of exactly zero at initialisation, and F15 asks whether *learned* routing earns its place, so an arm whose router provably took no gradient is not an instance of F15 at all. A gradient check on the routing parameters belongs in the harness ahead of the measurement rather than in the post-mortem after it. **F16 fires**: a tower-count sweep at constant per-token cost finds all eight towers of an $N = 8$ mixture free to remove, with every tower having carried traffic, so the confound that voids the F15 reading does not reach it. The other thirteen have not been run. This table is the complete list of the paper's *numbered* measurements. It is not the complete list of open measurement: the work ledger at `logos/GAPS.md` carries further measurements that are budgeted but carry no F-number, together with items that are data questions rather than experiments.

One such measurement has since been discharged and it is worth saying plainly what that does and does not do to this table. The integrity detector of §10.2 carried its own stated verdict rule — AUROC at least 0.99 at every tamper severity above 3σ of the benign null, with any collapse nearer the floor reported as the measured boundary — and both halves hold. **F8 is not thereby advanced.** That run used a parametric null rather than one measured across heterogeneous accelerators, and an adversary tampering on every canary, which are the two conditions §10.1 says will not transfer and the two the F8 row is written around. A verdict discharged outside the ledger does not move an entry inside it, and the count of numbered measurements run remains three.

A second addition to the ledger is a precondition rather than an entry, and it applies to F15, F16 and any future row that reads a routing distribution. **No routing figure in this work is admissible on fewer than three seeds.** §4.5 measures a forty-three-fold spread in routing entropy between two *untrained* routers whose only recorded difference was an unseeded draw and two uncontrolled factors, and §15.4 reports a substitution margin spanning +0.0721 to +0.3195 across four seeds of one configuration. Both are larger than any effect either measurement is trying to resolve. The requirement is cheap on this hardware and it is the difference between a measurement and an anecdote.

An earlier version also stated that two measurements, F11 and F12, need no accelerators at all. That is now true of F12 only. Both of F11's accelerator-free operationalisations were built, run, and failed, and the criterion that survives requires training a model on the domain mixture (§15.1). What changed is the cost class of the measurement, not the wording of the ledger.

Four of these could be discharged by simulation on synthetic inputs, and that route is closed

as circular: measuring that an implementation of an equation satisfies that equation measures nothing.

The ledger also has a budget ceiling, and it is worth stating here rather than leaving it to be inferred from the hardware column. F2 is the measurement that would test the architecture as specified, and its right-hand column says "frontier budget for the real thing". No such budget is available to this work, and none of the larger configurations the paper reasons about, at 5T, 10T, 100T or 1000T total parameters, is reachable either. What is reachable is the *bottom rung* of F2's own ladder: the composition gap $\Delta(N)$ between a modularly composed ensemble and a matched jointly post-trained monolith, at about 1B and no higher. The programme `logos/LADDER_ARCHITECTURE.md` specifies a 1B-to-70B sweep and that specification is wrong for this hardware: at $D = 20N$ and $6ND$ FLOPs, 1B costs about 1,100 GPU-hours per model, 7B about six GPU-years, and 70B about **620 GPU-years**, against two models per rung. The ladder document needs rescoping to the rung that exists. That curve is a result whether or not a 14T ensemble is ever built, and §12's bound is the reason it is not a consolation prize: if observation bandwidth rather than parameter count is what binds at the top of the ladder, then the rungs this work cannot buy are also not the rungs where the interesting quantity lives. The seven measurements that fit one consumer accelerator and the two that need none are the rest of the reachable programme, and they are the whole of what can proceed now.

ID	Position	Reading that would change it	Needs
F1	Sparse towers are not governed by $D_{\text{opt}} = 20N_{\text{total}}$ (§2.3)	A fitted mixture-of-experts scaling law giving compute-optimal D within 20% of $20N_{\text{total}}$ at $N_{\text{total}} \geq 10^{12}$ and 98% sparsity	Scaling sweep
F2	Branch-Adapt-Route composes at tower scale with router quality high enough that misrouting costs less than monolithic alignment damage	Composed ensemble scoring below a matched jointly post-trained baseline at any scale from 7B upward	8-GPU node (7B to 70B proxy); frontier budget for the real thing
F3	Quantile Balancing gives even utilisation with no dead experts, and Causal Dual Bias removes per-sequence imbalance, without an auxiliary loss (§5.3). The first half is already reported at 10^{22} FLOPs [54], 83 times F3's budget, so F3 is an independent replication rather than first evidence; a disagreement between the two would locate the effect in the vendor-adjacent configuration rather than in the mechanism	Dead experts under Quantile Balancing, or worse balance and downstream loss than the auxiliary-loss-free bias or than Expert Threshold routing [72], in a real 1B, 64-expert run	One consumer GPU

ID	Position	Reading that would change it	Needs
F4	An existing checkpoint converts to Delta Attention Residuals by finetuning without destabilising	Conversion diverging, or failing to recover baseline perplexity, on a real open checkpoint	One consumer GPU
F5	Shared-codebook quantization collapses to 4 to 10 effective dimensions on real hidden states, and subspace partitioning fixes it	Monolithic quantization retaining near-full effective rank on real states, or partitioning failing to recover it	One consumer GPU (by-product of F9)
F6	The post-routing compliance mask of Eq. (21) does not disturb quantile-balanced routing for other users in the batch	Measurable shift in other tokens' expert assignment when one request is masked	One 8-GPU node
F7	Scale-to-zero serverless experts are viable at interactive latency for tower-scale experts	Cold-start p95 exceeding the per-token latency budget at tower-scale expert sizes	One 8-GPU node
F8	Canary drift detection holds against a realistic adversary (Conj. 2 says it does not). **Not advanced by** §10.2, which discharges the detector's own weaker verdict — perfect separation at and above 3σ, with the boundary falling to 3σ for bias and subtle tampering and to an *inverted* ranking below it — against a parametric null and an adversary at duty cycle $p = 1$, the two conditions this row varies	AUROC staying above 0.95 as duty cycle $p \to 0.01$ under a null measured across heterogeneous accelerators	Two different GPUs
F9	Grounded trajectories beat disagreement-gated self-play, which beats unfiltered self-play, at matched token counts, without collapse (§12)	Unfiltered self-play matching grounded trajectories, or the collapse monitor firing on the grounded arm	One consumer GPU
F10	Router quality survives a tower swap via descriptor warm-start, so update cost stays linear (§4)	On any domain carrying at least 5% of evaluation traffic, the warm-started router's post-swap AIQ [49] falling more than 0.02 below its pre-swap AIQ, or its CPT(50%) [53] rising by more than ten percentage points of calls (§11.3)	One consumer GPU

ID	Position	Reading that would change it	Needs
F11	**Scoped to a deliberately partitioned corpus** (§3.6): where an operator chooses branch corpora by hand, the criterion of §3.5 returns *four* towers rather than the five Table 1 prints, because Mathematics and Logic fail all three axes. It says nothing about the end-to-end case, where the router learns the split and no tower is the Mathematics tower; F15 and F16 are the measurements there. Asserting the five-way partition here instead makes the reading in this column identical to the paper's own conclusion, so no run could change it. **Partially run** (§15.1): the lexical operationalisation is dead, its total range across unrelated corpora being narrower than the gap it must resolve, and so is the compression-based replacement, which under a project-disjoint null rated two different domains closer than one domain is to a fresh sample of itself. The criterion that survives is *gradient conflict*, whose threshold is the measured within-domain spread rather than a number chosen by anyone	Gradient conflict failing to hold complete separation over a full training run, rather than the 3000 steps measured, since separation narrows monotonically over that range ($0.1699 \rightarrow 0.1207$); or, applied to the five reference domains rather than the four programming languages measured so far, the criterion returning five towers after all	One consumer GPU. **This row needs an accelerator**: both accelerator-free operationalisations failed, and the survivor requires training a model on the mixture

ID	Position	Reading that would change it	Needs
F12	Tower decomposition converts an impossible unique-corpus requirement into a possible one under real residency constraints (§11.5)	A measured residency-bound fraction f high enough that Eq. (19)'s $5.6 \times 10^{13}(1 + 4f)$ exceeds the **central** unique-token supply estimate of §2, 6×10^{13}, that is f above about 0.02. The supply point is named because the verdict is a function of it, and leaving it unstated hides that: at the 90 percent lower bound of 2×10^{13} the condition holds at $f = 0$ and F12 fires unconditionally, and at the upper bound of 2×10^{14} it needs $f > 0.64$. At $f = 1$ the benefit is exactly zero on any supply figure	No accelerator; corpus residency classification against a named jurisdiction set

ID	Position	Reading that would change it	Needs
F13	Corpus-level difference between towers is exploitable by debate in a way persona-level heterogeneity is not, contradicting the heterogeneous-agent extension of the martingale (§12). The position has two limbs, and they need different instruments	**(a)** Debate between models of genuinely different pretraining lineage tracking the martingale as closely as debate between personas of one model. **(b)** Calibrated-confidence weighting lifting ensemble accuracy without any environment adjudication, *once the calibrator's own exogenous supervision is charged to the same ledger and held identical across arms*, which would locate the gain in the protocol rather than in the observation channel. Without that condition the reading in this column would be the content of [83]'s Theorem 1, a published result §12 cites and accepts, which would make the limb a restatement rather than a test. A null here is an equivalence statement and is declared only under the two-one-sided-tests procedure, at the margin stated in `logos/F9_MEASUREMENT_PLAN.md` §10	**(b)** One consumer GPU, an arm of F9, 36.3 GPU-hours at the planning roster (18.1 to 290.1 across candidate proposer rosters). **(a)** One consumer GPU, but *not* an arm of F9 and not with 350M stand-ins: existing open-weight models of distinct pretraining lineage, quantized and stepped sequentially, 17.4 GPU-hours at a four-model 7-to-8B-class roster and 2.2 at the 1B-class roster F9's own generation ledger plans on, so the two figures are not addable without saying which roster each is at. The 17.4 assumes prefix caching across debate rounds; without it the same protocol is 34.2

ID	Position	Reading that would change it	Needs
F14	A common seed with divergent continued pretraining preserves the informative difference F13 needs, so the lineage value Branch-Adapt-Route requires ($\lambda = 1$) and the value F13's position is stated at ($\lambda = 0$) are interchangeable (§3.3)	Debate between two continued-pretraining branches of one base checkpoint tracking the martingale measurably more closely than debate between two independently pretrained models of matched size and matched benchmark quality, under an identical protocol, where "measurably" means outside the two-one-sided-tests equivalence margin stated in `logos/F9_MEASUREMENT_PLAN.md` §10. If it does, the architecture cannot have both Branch-Adapt-Route and the diversity position, and §12's position is being measured on an object the reference architecture cannot build. **F14 is interpretable only conditional on F13 limb (a) first returning a nonzero lineage effect**, and the two run in that order: if limb (a) finds no lineage effect at all, both of F14's arms track the martingale equally by construction, the difference is zero, and F14 passes automatically. That pass is not support. It would mean the two lineage settings are interchangeable because neither has any diversity to preserve, and a null F14 under a null F13(a) is reported as **uninformative**	One consumer GPU, the same instrument as F13 limb (a) and costed with it at 17.4 GPU-hours at the 7-to-8B-class roster. Its binding constraint is not compute but availability: it needs a published continued-pretrain of a base that is also on the proposer roster, and we have not checked that one exists

ID	Position	Reading that would change it	Needs
F15	**The learned router earns its place.** Segment-level routing into deep towers recovers something that assignment without it does not (§3.6). This is the architecture's central bet and, until now, the only load-bearing position in this paper with no measurement attached — an omission, not a judgement that it needed none	Learned routing failing to beat *random* tower assignment, evaluated on identical trained weights with nothing retrained, so the difference is attributable to the routing decision rather than to capacity. A router that cannot beat random has produced a lookup table with extra steps. A *constant* assignment arm runs alongside it and asks the weaker question of whether more than one tower is used at all. Note the direction of interest: Mixtral's routing analysis found no topical specialisation among experts on Pile subsets [52], so at token-level granularity the domain reading of routing is already negative, and F15 asks only whether routing helps, not whether it helps *for the reasons the five-way illustration suggests*	One consumer GPU, **and at least three seeds**: §4.5 measures a forty-three-fold spread in routing entropy between two untrained routers, so a single-run routing figure is not admissible here. Instrumented in `logos/experiments/learned_partition.py`; the controls need no training beyond the arm itself

ID	Position	Reading that would change it	Needs
F16	**Towers are non-redundant.** Each tower carries competence the others do not, so the tower count is doing work rather than duplicating it (§3.6). This is the emergent-case replacement for the question F11 asks of a hand-partitioned corpus, and it is measured rather than predicted: a partition criterion infers redundancy from corpus statistics, whereas disabling a tower and reading what is lost *is* the redundancy	Disabling any single tower at inference — letting the router fall through to next-best per §4.2 — costing nothing on the material that tower was serving. It needs no threshold, no within-domain null and no assumption about how independent the corpora are, and it asks about tower three rather than about a named domain, which is what makes it usable when towers carry no labels. The reading is asymmetric and should be stated as such: "costs nothing" is a clean merge signal, while "costs something" is ambiguous between a tower that was necessary and a rerouting the model was never trained to perform	One consumer GPU, no training: an ablation pass over an already-trained checkpoint, **over at least three seeds** — a tower's disable cost depends on whether the router ever sent it traffic, and §4.5 measures two of four towers at exactly zero share under a live gradient

Nine of the sixteen run on one consumer accelerator: F3, F4, F9, F10, F15 and F16 as independent experiments, the last two on an already-trained checkpoint and therefore the cheapest of them, with F5 and F13's limb (b) falling out of F9, the latter at 36.3 GPU-hours, and F14 sharing an instrument with F13's limb (a). F13's limb (a) runs on that accelerator too, but not as an arm of F9 and not on 350M stand-ins. What it needs is models whose pretraining corpora, objectives and alignment histories genuinely differ, and that is a property of how a model was trained rather than of the hardware it runs on. Models with that property already exist, having been pretrained by different organisations on different data under different objectives, and they are arguably a better instrument than five towers from one laboratory, which would share data-collection pipelines and filtering decisions and be less independent than they look. Distinct lineage is the treatment variable: two models from one laboratory, or two finetunes of one base checkpoint, do not count as distinct. So instrumented, limb (a) measures the position at the level of independently trained open-weight models rather than at tower scale inside one architecture, and the ensemble under test is not a Mixture-of-Towers; that limitation is real and we state it rather than let the limb read as more than it is. Limb (a) is priced at 17.4 GPU-hours, derived from the proposal volume and the roster's inference cost rather than assumed, at a four-model 7-to-8B-class roster and under an assumption of prefix caching across debate rounds; the same protocol at F9's own 1B-class generation roster is 2.2 GPU-hours and without prefix caching it is 34.2, so the figure is only meaningful with its roster and its caching assumption attached. F14

runs on the same instrument but is read second, because a null F14 is uninterpretable unless limb (a) has first returned a nonzero lineage effect. The five-tower ensemble itself remains out of reach on any hardware we have, and that is F2, not F13. F12 needs no accelerator; it is a corpus-residency classification. F11 was budgeted the same way and is not: §15.1 reports both of its accelerator-free operationalisations failing, and the criterion that survived them needs a model trained on the domain mixture, which moves F11 onto the consumer-accelerator list, which makes ten of the sixteen run there. It remains among the cheapest tests of the paper's motivation for towers now that §2.2 has removed the compute motivation — the run reported in §15.1 is minutes of one consumer card, not hours — but "cheapest" and "free" are different statements and only the first holds. F9 is the one we would run first, because it tests the constraint that none of the architecture relaxes and it can return a decisive negative on a desktop.

17 Conclusion

A ten-trillion-parameter system is buildable, and not because anyone solved the data wall. The data wall was never the constraint the dense arithmetic said it was. Sparsity separates the compute bill from the parameter count. Modular composition separates the peak unique-corpus requirement from the total parameter budget. Four-bit serving separates the memory footprint from training precision. What is left binding is a shorter and less romantic list: unique high-quality tokens, resident memory, and the quality of a router that must adjudicate between frontier-grade specialists on far less training signal than any of them received.

We have tried to be exact about which joints are load-bearing and untested. The extrapolation of Branch-Adapt-Route from 7B to 2.8T is the central bet and it is unhedged. The shared codebook was in the wrong place and we moved it. The confidentiality story for peer dispatch does not hold and we withdrew it. The peer-integrity result is real, its detection boundary is now measured rather than assumed — perfect separation at and above 3σ of a parametric benign null, an inverted ranking below it, and both halves of a rule fixed before the run (§10.2) — and we predict it will not survive an adversary who is trying. Four-bit serving is not free of alignment consequences either: on this work's own instrument the compression step preserved an installed behaviour and the cross-entropy recovery step destroyed it, which makes the recovery objective a disclosure item rather than a pipeline detail. The architecture's own partition criterion says it wants four towers rather than five, on a domain axis crossed with a size axis that costs 2.75 percent of the parameter budget. Serving five towers costs the same as five replicas of one generalist of the same per-instance size, but the two fleets are not interchangeable, because replicas are fungible and towers are not: under skewed traffic and uniform sizing the ensemble is strictly worse, losing two thirds of a 320-accelerator fleet to idleness on the five-way partition, or 58 percent of it on the four-way partition §3.5's criterion actually returns. Sizing towers in proportion to a traffic mix known before they are trained does *not* recover that throughput, though it looks as though it should: under Proposition 1's fixed sparsity every pool serves at the same rate regardless of its size, so demand-proportional sizing buys resident parameters where the traffic is and leaves the saturation point exactly where the uniform arrangement put it. That is one axis, and §9.2 adds a second that does not depend on the traffic mix: an ensemble routes a query once where a monolith of equal total size pays a peer round trip per layer, so the ensemble stays serviceable over a wide-area heterogeneous pool and the monolith does not, provided each tower's own pool is one low-latency locality. That axis is arithmetic, unmeasured, conditional on that premise, and scoped to networks whose round trip exceeds about 1.7 ms. The update-

economics argument, the alignment-isolation argument and the informative-difference position of §3 are independent of both and are not priced here.

The regulatory finding is the one we would most like to be wrong about. A capability threshold written against *a model* does not survive an architecture whose economic proposition is that models are components. Modular composition makes the compute a provider expends arbitrarily small while the capability it places on the market is bounded below by its strongest sourced part. Closing that gap means attaching the presumption to deployed capability, or to embodied compute, and the sooner it is settled the smaller the incentive to build this architecture specifically to exploit it.

But the finding that changed our own view is the last one. Every mechanism in this paper lifts a constraint, and after all of them are lifted one remains, which none of them touches. Debate among towers is a martingale whether or not they know the same things. Reinforcement learning on a fixed problem set sharpens what was already reachable. Self-training without grounding decays. Self-play works when a code executor checks the answer, and the executor is doing the work. At this scale the limiting resource is not parameters or compute or even data, but observation: the rate at which something outside the model can tell it that it is wrong. Scale buys the capacity to be corrected quickly. It does not buy the corrections.

That is also where the tower architecture stops being an accounting convenience, and where we take our one position against the literature rather than from it. The debate theorem does not say that improvement needs participants who know different things; it says the opposite, and extends the martingale to heterogeneous agents. We hold that towers pretrained on disjoint corpora differ in a way that heterogeneity of prompt or persona does not, and that this difference is exploitable. It is measurement F13, and it is the one we would run second. §3.3 makes it worse before it makes it better: Branch-Adapt-Route needs a common seed, so the towers this architecture can actually build are branches of one checkpoint rather than independent models, and whether a branch is a large enough perturbation to carry the position is measurement F14. Neither of its limbs needs hardware we do not have. The confidence-weighting limb is an arm of F9 at 36.3 GPU-hours, and the diversity limb needs no towers built for the purpose: models of genuinely different pretraining lineage already exist, and the test is to use them, at the price of measuring the position at the level of independently trained open-weight models rather than at tower scale.

One last thing about status, stated here because the abstract states it and the two should not disagree. The machine this paper specifies is not buildable at any budget behind this work, and the larger configurations it reasons about are further away still. That is not a hedge and it is not an apology, because it is the same sentence as the paper's own conclusion: if the binding quantity at the top of the ladder is observation bandwidth rather than parameters, then an unreachable rung is not merely unaffordable, it is the wrong thing to be reaching for. What remains reachable is the part that was going to carry the information anyway. The composition gap $\Delta(N)$ *at about 1B* is measurable on hardware that exists here — about 1,100 GPU-hours per model and two models, so roughly a quarter of a card-year for the one rung. It is measurement F2's own quantity evaluated at the only scale where it can be evaluated here, and a single point is not a curve. It is still useful in both directions: it tells an operator with a frontier budget whether modular composition holds at scale, or it tells them not to spend the run. F9 and the other consumer-accelerator measurements are reachable. F12 needs no accelerator; F11 was thought not to and does, for the reason §15.1 gives, and it has been run on the hardware that exists here. That is the programme, and none of it waits on the 14T ensemble.

Almost nothing here is a result. It is a specification with its arithmetic checked, its citations audited against primary sources, its weakest components relocated, and sixteen open measurements stated with the hardware each requires. Ten of them fit on a desktop and one needs no accelerator. The exception is §15, which reports the three measurements that have been run: two ways of asking F11's question are dead, a third survives on four programming languages at 1.7×10^7 parameters and has since been applied to the five reference domains (§15.2), where complete separation failed; and F15 and F16, which were reported as not firing on one trained checkpoint and whose reading §15.4 now corrects, because the substitution test they share returns its largest margins on routers that collapsed to a single tower. **F15 reverts to unrun, and the instrument that replaces it — oracle routing — is being measured now** (§15.4); no value is quoted until those arms land. **F16 fires on one run**, a tower-count sweep in which all eight towers of an $N = 8$ mixture are free to remove despite every one of them having carried traffic; the same replicates show the count of towers that earn their place is seed-dependent at $N = 4$, taking the values 2, 3 and 4, so that firing is a reading on its run and not a discharge. §15 now also reports a second rig, and its findings are about the specification rather than about the measurement ledger. **The router in the only implementation of this architecture that exists received a gradient of exactly zero**, and Proposition 3 says exact parent-identity at initialisation and a live routing gradient cannot both be had, so one of them has to be given up on purpose. That router was also pooling its features from the first 128 tokens of each segment, which on a corpus of agentic trajectories is the shared system prompt and carries a quarter of the between-segment spread the very next window does — so routing granularity has a *sequence-position* dimension this paper had not named, and the choice an implementation makes by default is the worst one available. At 2.11 times the parameters that arm did not beat its dense control, which is what the first two findings predict rather than an independent disappointment. The single positive result on that rig belongs to the corpus and not to the architecture, and the dense arm collected it too: a factor of 6.5 in the rate of correct second steps, on a denominator the model cannot move. A full training run has not been reached. The architecture itself remains untrained and unserved at every scale, and the measurements are instruments for the numbered questions rather than instances of the machine.

AI contribution declaration

This paper was developed in close collaboration with Claude, a large language model from Anthropic. The AI contributed to drafting, to verifying every citation against a primary source, to the arithmetic corrections of §2, and to the synthesis of the four results in §12. The human author, Wingston Sharon, directed the work and contributed its central positions: the Mixture-of-Towers composition, the compute-economics reading of restricted deployment (Position 1), the regulatory-arbitrage argument (Position 3), and the proposal that towers can bootstrap past the token wall by generating and adjudicating their own trajectories, which §12 develops. He made all final judgments and takes responsibility for the content, including any errors. An AI system cannot bear accountability for a publication and so cannot be a formal author; this declaration preserves the transparency a joint byline was meant to express.

References

[1] A theoretical framework for auxiliary-loss-free load balancing of sparse mixture-of-experts in large-scale AI models. *arXiv:2512.03915*, 2025.

[2] Anthropic. Claude Fable 5 and Claude Mythos 5. https://www.anthropic.com/news/claude-fable-5-mythos-5, June 2026.

[3] Anthropic. Project Glasswing: securing critical software for the AI era. https://www.anthropic.com/glasswing, 2026.

[4] Anthropic. Redeploying Claude Fable 5. https://www.anthropic.com/news/redeploying-fable-5, July 2026.

[5] Autoriteit Persoonsgegevens. Wetgevingstoets Uitvoeringswet AI-verordening. https://www.autoriteitpersoonsgegevens.nl/, June 2026.

[6] Continuous first, discrete later: VQ-VAEs without dimensional collapse. *arXiv:2605.06870*, 2026.

[7] Learning to route LLMs from bandit feedback. *arXiv:2510.07429*, 2025.

[8] Beam Cloud. Beta9: ultrafast serverless GPU inference, sandboxes, and background jobs. https://github.com/beam-cloud/beta9, AGPL-3.0, 2026. **Named as a specified component. Nothing in this paper has been deployed on it. Scale-to-zero workloads, agent-backed external worker pools and the Tailscale mesh setting are read from the upstream repository's documentation and default configuration.**

[9] A. Borzunov et al. Petals: collaborative inference and fine-tuning of large models. *Proceedings of ACL 2023 (System Demonstrations)*, pp. 558–568, 2023.

[10] Brick: spatial capability routing for the Mixture-of-Models paradigm. *arXiv:2606.13241*, 2026.

[11] Chameleon Team (Meta FAIR). Chameleon: mixed-modal early-fusion foundation models. *arXiv:2405.09818*, 2024.

[12] J. Chang. Causal routing bias for aux-loss-free MoE training. https://jonathanc.net/blog/causal-routing-bias, 2026. **Blog post; no peer-reviewed source.**

[13] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: how to use large language models while reducing cost and improving performance. *arXiv:2305.05176*, 2023.

[14] L. Chen et al. Punica: multi-tenant LoRA serving. *Proceedings of MLSys*, 2024.

[15] J. Chhugani, G. Jeong, B.-Y. Su, Y. Pan, H. Yang, A. Ankit, J. Yu, S. Deng, Y. Chen, N. Satish, and C. Kim. Unveiling the potential of quantization with MXFP4: strategies for quantization error reduction. *arXiv:2603.08713*, 2026.

[16] H. K. Choi, X. Zhu, and S. Li. Debate or vote: which yields better decisions in multi-agent large language models? *arXiv:2508.17536*, 2025; *Advances in Neural Information Processing Systems*, 38, 2025 (Spotlight). **Source of the Dirichlet-Categorical martingale result and of its heterogeneous-agent extension.**

[17] M. Cihangiroglu and A. Nocera. Integrity of peer-to-peer distributed LLM inference under malicious nodes. *arXiv:2607.19490*, 2026.

[18] A. Clark, D. de Las Casas, A. Guy, A. Mensch, M. Paganini, J. Hoffmann, et al. Unified scaling laws for routed language models. *arXiv:2202.01169*; *Proceedings of ICML*, 2022. **Source of the effective-parameter-count formulation used in §11.2.**

[19] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah. Hybrid LLM: cost-efficient and quality-aware query routing. *International Conference on Learning Representations*, 2024; *arXiv:2404.14618*.

[20] L. Ding and X. Wang. Grounded scaling: why agentic AI needs deterministic environments. *arXiv:2606.22495*, 2026.

[21] Emu3 (BAAI). Multimodal learning with next-token prediction for large multimodal models. *Nature*, s41586-025-10041-x, 2025; preprint *arXiv:2409.18869*.

[22] European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), Article 51 and Annex XIII. *Official Journal of the European Union*, 2024.

[23] FaaSMoE: a serverless framework for multi-tenant mixture-of-experts serving. *arXiv:2604.26881*; *MobiSys Workshops '26*, 2026.

[24] Towards generalized routing: model and agent orchestration for adaptive and efficient inference. *arXiv:2509.07571*, 2025.

[25] T. Genewein, M. Franklin, A. Lerchner, L. Orseau, S. Albanie, A. Bales, C. Wyeth, S. Chan, I. Gabriel, J. Z. Leibo, A. Dafoe, M. Hutter, T. Graepel, and S. Legg. From AGI to ASI. *arXiv:2606.12683*, 2026.

[26] M. Gerstgrasser et al. Is model collapse inevitable? Breaking the curse of recursion by accumulating real and synthetic data. *arXiv:2404.01413*, 2024.

[27] European Commission AI Office. General-Purpose AI Code of Practice, final version. https://code-of-practice.ai/, 2025.

[28] J. Hoffmann et al. Training compute-optimal large language models. *arXiv:2203.15556*; *Advances in Neural Information Processing Systems*, 35, 2022.

[29] E. J. Hu et al. LoRA: low-rank adaptation of large language models. *arXiv:2106.09685*; *ICLR*, 2022.

[30] K. Huang. Demystifying Kimi K3: the three algorithms behind the top frontend coding model. https://kenhuangus.substack.com/, 2026. **Third-party explainer.**

[31] International Association of Privacy Professionals. Artificial Intelligence Governance Professional (AIGP) certification. https://iapp.org/certify/aigp, accessed 2026.

[32] Kimi Team. Attention Residuals: technical report. *arXiv:2603.15031*, 2026.

[33] D. Lee et al. Autoregressive image generation using residual quantization. *Proceedings of CVPR*, 2022.

[34] D. Liu et al. PiKV: KV cache management system for MoE architecture. *arXiv:2508.06526*, 2025.

[35] J. Ludziejewski et al. Joint MoE scaling laws: mixture of experts can be memory efficient. *arXiv:2502.05172*, 2025.

[36] C. Luo, Z. Cai, and J. Hu. Delta Attention Residuals. *arXiv:2605.18855*, 2026.

[37] Z. Ma et al. Mixture of Heterogeneous Grouped Experts for language modeling. *arXiv:2604.23108*, 2026.

[38] A. Smit, P. Duckworth, N. Grinsztajn, T. D. Barrett, and A. Pretorius. Should we be going MAD? A look at multi-agent debate strategies for LLMs. *arXiv:2311.17371*, 2023; *Proceedings of ICML*, 2024.

[39] Kimi K3 vs DeepSeek V4 Pro vs GLM-5.2: open trillion-scale MoE models compared on benchmarks, license, and serving cost. MarkTechPost, July 2026. **Tier B, secondary.**

[40] Holistic scaling laws for optimal mixture-of-experts architecture optimization. *arXiv:2603.21862*, 2026.

[41] Moonshot AI. Kimi K3 technical blog: open frontier intelligence. https://www.kimi.com/blog/kimi-k3, 2026. **Vendor deployment report. Not the primary source for Quantile Balancing, which originates with J. Su months earlier; see su2026qb.**

[42] J. Morrison, S. Adhikesaven, A. Bhagia, M. Zaharia, N. A. Smith, and S. Min. Train separately, merge together: modular post-training with mixture-of-experts. *arXiv:2604.18473*, 2026.

[43] N. Muennighoff et al. Scaling data-constrained language models. *arXiv:2305.16264*; *Advances in Neural Information Processing Systems*, 36, 2023.

[44] A. Borzunov et al. Distributed inference and fine-tuning of large language models over the Internet. *Advances in Neural Information Processing Systems*, 36, 2023.

[45] Dimensional collapse in VQVAEs: evidence and remedies. *Advances in Neural Information Processing Systems*, 38, 2025. Introduces Divide-and-Conquer VQ.

[46] LatentMoE: toward optimal accuracy per FLOP. *arXiv:2601.18089*, 2026.

[47] Quantization-aware distillation for NVFP4 inference accuracy recovery. *arXiv:2601.20088*; NVIDIA Nemotron technical report, 2026.

[48] NVSHMEM device-initiated communication for mixture-of-experts all-to-all dispatch: IBGDA against CPU-proxy round-trip latency. *arXiv:2604.00317*, 2026. **Source of the 16.7 to 24.3 microsecond intra-datacentre round trips quoted in §9.2. Read for the round-trip figures only.**

[49] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay. RouterBench: a benchmark for multi-LLM routing systems. *arXiv:2403.12031*, 2024. **Source of AIQ, and of the non-decreasing convex hull used here as the cost-quality frontier. Its Oracle Router is a per-item maximum over k models and carries the chance inflation of §11.3 in full; it is a reported reference line, not a term inside AIQ, so AIQ's value is unaffected.**

[50] The Myth of Expert Specialization in Mixture-of-Experts: Why Routing Reflects Geometry, Not Necessarily Domain Expertise. *arXiv:2604.09780*, 2026. **Routers are linear maps, so hidden-state similarity is necessary and sufficient for expert-usage similarity; specialisation is a property of the representation space, and the patterns resist human interpretation.**

[51] The Illusion of Specialization: Unveiling the Domain-Invariant "Standing Committee" in Mixture-of-Experts Models. *arXiv:2601.03425*, 2026. **Expert committees emerge domain-invariantly, against the assumption of domain-specific routing.**

[52] A. Q. Jiang et al. Mixtral of Experts. *arXiv:2401.04088*, 2024. **Routing analysis (§5) reports no topical specialisation among experts on Pile subsets (ArXiv, PubMed, PhilPapers): assignment tracks syntax, token identity and positional locality, and the distribution over experts is similar across subsets.**

[53] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: learning to route LLMs with preference data. *arXiv:2406.18665*, 2024.

[54] L. Dial (Open Athena). Mixture of Experts Quantile Balancing: validated at 32B-A5B (1e22 FLOPs) scale. https://openathena.ai/blog/quantile-balancing/, April 2026. **Independent report of Quantile Balancing at 32B total / 5B active, 64 routed experts, 326B tokens. Blog post; no peer-reviewed source.**

[55] OpenAI. Report on autonomous sandbox escape and external compromise during ExploitGym cyber-capability evaluation, July 2026. See also UK AI Safety Institute assessments of long-horizon cyber capability.

[56] OpenRouter. Model routing: the Auto Router. https://openrouter.ai/docs/features/model-routing, 2026. **Deployed commercial system; vendor documentation, read directly.**

[57] L. Beyer et al. PaliGemma: a versatile 3B VLM for transfer. *arXiv:2407.07726*, 2024.

[58] S. Rajput et al. Recommender systems with generative retrieval (TIGER). *Advances in Neural Information Processing Systems*, 36, 2023.

[59] Rijksinspectie Digitale Infrastructuur. Market surveillance under the AI Regulation. https://www.rdi.nl/, accessed 2026.

[60] Dynamic model routing and cascading for efficient LLM inference: a survey. *arXiv:2603.04445*, 2026.

[61] W. Sharon. Conditions for predictable social dynamics: conservation, decomposition, and control at criticality. Draft v0.5, position paper (in review), 2026.

[62] Y. Sheng et al. S-LoRA: serving thousands of concurrent LoRA adapters. *Proceedings of MLSys*, 2024.

[63] M. Shirokikh and S. Nikolenko. Sparse prefix caching for hybrid and recurrent LLM serving. *arXiv:2605.05219*, 2026.

[64] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal. AI models collapse when trained on recursively generated data. *Nature*, 631:755–759, 2024.

[65] D. Silver and R. S. Sutton. Welcome to the era of experience. Google DeepMind, 2025.

[66] H. A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.

[67] Multi-turn distributed inference with mixture of experts for 6G edge and cloud networks (StateFlow). *arXiv:2607.02522*, 2026.

[68] J. Su. Blog post introducing Quantile Balancing for mixture-of-experts load balancing (in Chinese). https://kexue.fm/archives/11619, February 2026. **Personal blog; originating description of Quantile Balancing, predating the Kimi K3 release. The page**

returns HTTP 403 to automated retrieval, so we record the attribution as corroborated by [54] and by a public statement from P. Liang, rather than as read by us.

[69] J. Suarez. PufferLib: making reinforcement learning libraries and environments play nice. *arXiv:2406.12905*, 2024. Pokémon Red environment: `PufferAI/pokegym`.

[70] S. Sukhbaatar, O. Golovneva, et al. Branch-Train-MiX: mixing expert LLMs into a mixture-of-experts LLM. *arXiv:2403.07816*; *COLM*, 2024.

[71] W. Sun, X. Cheng, J. Fan, Y. Xu, X. Yu, S. He, J. Zhao, and K. Liu. Towards agentic self-learning LLMs in search environment. *arXiv:2510.14253*, 2025.

[72] H. Sun, Y. Liu, Y. Wu, and L. Sun. Expert threshold routing for autoregressive language modeling with dynamic computation allocation and load balancing. *arXiv:2603.11535*, 2026.

[73] Moonshot AI (Kimi Team). Kimi Linear: an expressive, efficient attention architecture. *arXiv:2510.26692*, 2025. Introduces Kimi Delta Attention.

[74] Tracer AI. Echo: adaptive coordination of open-weight models. https://tracerml.ai/, 2026. **Vendor-reported; no independent evaluation located.**

[75] P. Villalobos, A. Ho, J. Sevilla, T. Besiroglu, L. Heim, and M. Hobbhahn. Will we run out of data? Limits of LLM scaling based on human-generated data. *arXiv:2211.04325v2*, 2024; *Proceedings of ICML* (Position track), 2024.

[76] vLLM Project. A preview of production-scale Kimi K3 support on vLLM. https://vllm.ai/blog/2026-07-22-kimi-k3-preview, July 2026.

[77] A. Wang et al. HMoE: heterogeneous mixture of experts for language modeling. *arXiv:2408.10681*, 2024.

[78] L. Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv:2408.15664*, 2024.

[79] Y. Yue et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *Advances in Neural Information Processing Systems*, 38, 2025. Best Paper Runner-up.

[80] H. Zenil. On the limits of self-improving in large language models: the singularity is not near without symbolic model synthesis. *arXiv:2601.05280*, 2026.

[81] H. Zhang et al. MiLoRA: efficient mixture of low-rank adaptation for large language models fine-tuning. *Findings of EMNLP*, 2024.

[82] A. L. Zhang, T. L. Griffiths, K. R. Narasimhan, and O. Press. VideoGameBench: can vision-language models complete popular video games? *arXiv:2505.18134*, 2025.

[83] X. Zhu, C. Zhang, Y. Chi, T. Stafford, N. Collier, and A. Vlachos. Demystifying multi-agent debate: the role of confidence and diversity. *arXiv:2601.19921*, 2026. **Derives the confidence-weighted submartingale (Thm. 1); its diversity result changes the initial distribution, not the update dynamics. No author named Choi is on this paper; it is not `choi2025debatevote`.**

[84] A. Zhao et al. Absolute Zero: reinforced self-play reasoning with zero data. *arXiv:2505.03335*; *Advances in Neural Information Processing Systems*, 38, 2025.

[85] Zhipu AI. GLM-5.2 open-weight release, June 2026. https://huggingface.co/zai-org/GLM-5.2 (MIT licence).